

O'REILLY®

第三版

NGINX 完全指南

实现高性能负载均衡的进阶实操指南



来自



NGINX®
Part of F5

Derek DeJonghe

NGINX 完全指南

NGINX 是当今使用最广泛的 Web 服务器之一，部分原因在于它可以用作 HTTP 和其他网络协议的负载均衡器和反向代理服务器。本修订版完全指南通过一些简单易懂的例子解析了应用交付中真实存在的问题。实用的实操指南可帮助您设置开源或商业产品，并利用它们解决各种用例中的问题。

对于了解现代 Web 架构（例如 n 层或微服务设计以及 TCP 和 HTTP 等常见 Web 协议）的专业人士来说，本书的这些实操指南为安全和软件负载均衡以及 NGINX 应用交付平台的监控和维护提供了经过验证的解决方案。您还将了解到免费的 NGINX 开源版以及许可授权版 NGINX Plus 的高级功能。

您将获取以下方面的实操指南：

- HTTP、TCP 和 UDP 高性能负载均衡
- 通过加密流量、安全链接、HTTP 身份验证子请求等保护访问
- 将 NGINX 部署到 Google、亚马逊云科技（AWS）和 Azure 云服务
- 将 NGINX Plus 配置为 SAML 环境中的服务提供商
- HTTP/3（QUIC）、OpenTelemetry 和 njs 模块

Derek DeJonghe 是 AWS 认证专业人员，专门研究基于 Linux/Unix 的系统和 Web 应用。他在 Web 开发、系统管理和网络方面拥有扎实的专业背景，积累了丰富的云经验，主要涉及基础架构管理、配置管理及持续集成。他还开发 DevOps 工具，并维护多个多租户 SaaS 产品的系统、网络及部署。

免费试用 F5 NGINX Plus 和 F5 NGINX App Protect

为微服务提供高性能应用交付和安全防护



NGINX Plus 是一款软件负载均衡器、API 网关和微服务代理。



NGINX App Protect 是一款轻量级、快速 Web 应用防火墙 (WAF)，基于 F5 业经验证的可靠技术，专为现代应用和 DevOps 环境而设计。



节省成本

较之硬件应用交付控制器和 WAF，可显著节省成本，并提供您所期望的所有性能和功能。



降低复杂性

唯一一款将负载均衡器、API 网关、微服务代理和 Web 应用防火墙集于一身的多合一产品，有助于减少基础架构蔓延。



面向企业

NGINX Plus 和 NGINX App Protect 可与 DevOps 和 CI/CD 环境集成，满足企业对安全性、可扩展性和弹性的要求。



高级安全防护

保护您的应用和 API 免受不断扩大的网络攻击和安全漏洞的影响。



第三版

NGINX 完全指南

实现高性能负载均衡的进阶实操指南

Derek DeJonghe

Beijing • Boston • Farnham • Sebastopol • Tokyo

O'REILLY®

NGINX 完全指南

作者：Derek DeJonghe

Authorized Simplified Chinese translation of the English edition of NGINX Cookbook, Third Edition (ISBN 9781098158439) © 2024 O'Reilly Media, Inc. This translation is published by permission of O'Reilly Media, Inc., which owns or controls all rights to publish and sell the same.

出版商：O'Reilly Media, Inc., 1005 Gravenstein Highway North, Sebastopol, CA 95472。

所购的 O'Reilly 图书可用于教育、商业或促销用途。其中多数图书也有网络版 (<http://oreilly.com>)。有关更多信息，请联系我们的企业/机构销售部：800-998-9938 或 corporate@oreilly.com。

策划编辑：John Devins

开发编辑：Gary O'Brien

制作编辑：Clare Laylock

文字编辑：Piper Editorial Consulting, LLC

英文版校对：Kim Cofer

索引：Potomac Indexing, LLC

内部设计：David Futato

封面设计：Karen Montgomery

插画：Kate Dullea

中文版校对：林静、熊平、易久平、孔令子

相关发布信息请参见：<http://oreilly.com/catalog/errata.csp?isbn=9781098158439>

O'Reilly 徽标是 O'Reilly Media, Inc. 的注册商标。《NGINX 完全指南》、封面图片和相关商业包装是 O'Reilly Media, Inc. 的商标。

本书所述均为作者的个人观点，不代表出版商的立场。虽然出版商和作者已尽力确保本作品中所包含信息和说明的准确性，但双方均不对其中的错误或遗漏承担任何责任，包括但不限于因使用或依赖本作品而导致的损害赔偿。使用本作品中包含的信息和说明需要您自行承担风险。如果本作品包含或描述的任何代码示例或其他技术受开源许可或他方知识产权的约束，您有责任确保您的使用符合此类许可和/或权利。

本作品是 O'Reilly 和 NGINX 的合作成果之一。详情请参见我们的[编辑独立性声明](#)。

目录

前言	x
序文	xii
1. 基础知识	1
1.0 简介	1
1.1 在 Debian/Ubuntu 上安装 NGINX	1
1.2 通过 YUM 软件包管理器安装 NGINX	2
1.3 安装 NGINX Plus	3
1.4 验证安装	3
1.5 关键文件、目录和命令	4
1.6 使用 include 简化配置	6
1.7 提供静态内容	7
2. 高性能负载均衡	9
2.0 简介	9
2.1 HTTP 负载均衡	10
2.2 TCP 负载均衡	11
2.3 UDP 负载均衡	13
2.4 负载均衡方式	14
2.5 NGINX Plus 之 Sticky Cookie	17
2.6 NGINX Plus 之 Sticky Learn	18
2.7 NGINX Plus 之 Sticky Routing	19
2.8 NGINX Plus 之连接清空	20
2.9 被动健康检查	21

2.10 NGINX Plus 之主动健康检查	22
2.11 NGINX Plus 之慢启动	24
3. 流量管理.....	25
3.0 简介	25
3.1 A/B 测试	25
3.2 使用 GeoIP 模块和数据库	27
3.3 基于国家 / 地区的访问限制	30
3.4 查找原始客户端	31
3.5 限制连接数	32
3.6 限制速率	34
3.7 限制带宽	35
4. 大规模可扩展的内容缓存.....	37
4.0 简介	37
4.1 缓存区	37
4.2 缓存哈希键	39
4.3 缓存锁定	40
4.4 使用陈旧缓存	40
4.5 绕过缓存	41
4.6 NGINX Plus 之缓存清除	42
4.7 缓存切片	43
5. 可编程性和自动化.....	45
5.0 简介	45
5.1 NGINX Plus API	45
5.2 使用 NGINX Plus 的键值 (Key-Value) 存储功能	49
5.3 在 NGINX 中使用 njs 模块暴露 JavaScript 功能	51
5.4 使用通用编程语言扩展 NGINX	54
5.5 使用 Ansible 安装	56
5.6 使用 Chef 安装	58
5.7 使用 Consul 模板自动进行配置	59
6. 身份验证.....	63
6.0 简介	63
6.1 HTTP 基本身份验证	63
6.2 身份验证子请求	65
6.3 使用 NGINX Plus 验证 JWT	66

6.4	创建 JSON Web Key	68
6.5	使用 NGINX Plus 通过现有的 OpenID Connect SSO 验证用户身份	69
6.6	使用 NGINX Plus 验证 JSON Web Token (JWT)	70
6.7	使用 NGINX Plus 自动获取和缓存 JSON Web Key Set	71
6.8	将 NGINX Plus 配置为 SAML 身份验证的服务提供商	72
7.	安全控制	77
7.0	简介	77
7.1	基于 IP 地址的访问	77
7.2	允许跨域资源共享 (CORS)	78
7.3	客户端加密	79
7.4	高级客户端加密	81
7.5	Upstream 加密	83
7.6	保护 location	84
7.7	使用 secret 生成安全链接	84
7.8	使用过期日期保护 location	85
7.9	生成过期链接	86
7.10	HTTPS 重定向	88
7.11	在 NGINX 之前卸载 SSL/TLS 后重定向到 HTTPS	89
7.12	HTTP 严格传输安全协议	89
7.13	基于国家 / 地区的访问限制	90
7.14	提供多种安全方法	92
7.15	NGINX Plus 动态应用层 DDoS 防护	92
7.16	安装和配置 NGINX Plus 的 NGINX App Protect WAF 模块	94
8.	HTTP/2 和 HTTP/3 (QUIC)	99
8.0	简介	99
8.1	启用 HTTP/2	99
8.2	启用 HTTP/3	100
8.3	gRPC	102
9.	复杂的媒体串流	105
9.0	简介	105
9.1	传输 MP4 和 FLV 格式的文件	105
9.2	使用 NGINX Plus 的 HLS 模块进行流式传输	106
9.3	使用 NGINX Plus 的 HDS 模块进行流式传输	107
9.4	使用 NGINX Plus 限制带宽	108

10. 云部署.....	109
10.0 简介	109
10.1 自动配置	109
10.2 将 NGINX 虚拟机部署到云端	111
10.3 创建 NGINX 机器镜像	112
10.4 无需云原生负载均衡器将流量路由到 NGINX 节点	113
10.5 负载均衡器 Sandwich	115
10.6 通过动态扩展 NGINX 服务器实现负载均衡	117
10.7 创建 Google App Engine 代理	118
11. 容器 / 微服务	121
11.0 简介	121
11.1 使用 NGINX 作为 API 网关	122
11.2 在 NGINX Plus 中使用 DNS SRV 记录	126
11.3 使用官方 NGINX 容器镜像	127
11.4 创建 NGINX Dockerfile	128
11.5 构建 NGINX Plus 容器镜像	132
11.6 使用 NGINX 中的环境变量	133
11.7 来自 NGINX 的 NGINX Ingress Controller (Ingress 控制器)	134
12. 高可用性部署模式.....	137
12.0 简介	137
12.1 NGINX Plus HA (高可用性) 模式	137
12.2 通过 DNS 实现负载均衡器的负载均衡	140
12.3 在 EC2 上实现负载均衡	141
12.4 NGINX Plus 配置同步	142
12.5 与 NGINX Plus 的状态共享和区域同步	144
13. 高级活动监控.....	147
13.0 简介	147
13.1 启用 NGINX stub 状态	147
13.2 启用 NGINX Plus 监控仪表盘	148
13.3 使用 NGINX Plus API 收集指标	150
13.4 在 NGINX 中使用 OpenTelemetry	153
13.5 Prometheus Exporter 模块	157

14. 利用访问日志、错误日志和请求跟踪进行调试和故障排除	159
14.0 简介	159
14.1 配置访问日志	159
14.2 配置错误日志	161
14.3 转发日志到 Syslog	162
14.4 调试配置	163
14.5 请求跟踪	164
 15. 性能调优	 167
15.0 简介	167
15.1 使用压测工具实现测试自动化	167
15.2 控制浏览器缓存	168
15.3 保持客户端长连接	169
15.4 保持上游长连接	169
15.5 响应缓冲	170
15.6 访问日志的缓冲	171
15.7 操作系统调优	172
 索引	 175

前言

欢迎阅读 2024 版《NGINX 完全指南》。O'Reilly 已连续九年出版《NGINX 完全指南》，我们不断进行更新以跟上 NGINX 做出的诸多改进。如今，NGINX 是全球最受欢迎的 Web 服务器。该产品于 2004 年首次发布，并不断发展，以满足现代应用扩展、保护和交付的需求。精心打造的 NGINX 具备卓越的灵活性和可扩展性，其功能从 Web 服务扩展到负载均衡、反向代理和 API 网关。主要公有云和 CDN 提供的许多负载均衡服务实际上均基于 NGINX 的原代码，这充分彰显了 NGINX 的重要价值。

NGINX 还在不断挖掘更多可能性，增添重要功能。为 Kubernetes 打造的 NGINX Ingress Controller 与 NGINX 原生集成，可为管理东西向和南北向流量提供重要功能——这是 Kubernetes 环境的一项关键要求。NGINX 还一直在扩展其身份验证和安全防护功能——但如果 NGINX 无法继续提供出色的速度、弹性和灵活性（这些都是现代分布式应用的基本要求），那么这些高级功能就会都变得无关紧要。

《NGINX 完全指南》是最熟悉 NGINX 的一群专家编写的 NGINX 必备指南。无论是为小型项目运行 NGINX 开源版，还是为跨多个区域的企业部署运行 NGINX Plus，无论是在本地还是在云端运行，《NGINX 完全指南》均可帮助您充分利用 NGINX。本书提供了一百多条简单易学的实操指南，内容涉及 NGINX 的安装、针对几乎所有用例的配置、安全防护和扩展，以及常见故障排除。

2024 版不仅更新了许多章节介绍 NGINX 的新功能，而且还增添了几个新章节，涉及扩展的安全防护功能以及如何有效利用 HTTP/2、HTTP/3 (QUIC) 和 gRPC。技术和应用的发展日新月异，NGINX 也不断与时俱进，旨在助您取得成功。NGINX 最初的愿

景是提供高度可扩展、可靠、快速且安全的 Web 服务。如今，我们不忘初心，竭尽全力帮助我们的社区在任何环境中以任何规模自信从容地保护并部署所需应用。

敬请阅读最新版本，并随时分享您的感想。欢迎您提出宝贵意见。

—— *Peter Beardmore*

产品营销总监

F5 NGINX

序文

《NGINX 完全指南》旨在通过一些简单易懂的例子解析应用交付中真实存在的问题。本书内容丰富全面，探讨了 NGINX 的大部分主要特性，可帮助您学习如何使用这些功能。

本书将先解释 NGINX 和 NGINX Plus 的安装程序以及一些供 NGINX 新手借鉴的基础入门步骤。后面的内容将过渡到各种形式的负载均衡，并分章节提供了关于流量管理、缓存和自动化的配套内容。第 6 章“身份验证”的涉及面很广，但同时也很重要，因为 NGINX 通常是 Web 流量进入应用的第一个入口点，也是应用层抵御 Web 攻击和漏洞的第一道防线。书中还有许多章节介绍了 HTTP/3 (QUIC)、流媒体、云、SAML Auth 和容器环境等前沿主题，最后又以一些更传统的运维主题收尾，例如监控、调试、性能和运维技巧等。

我个人将 NGINX 用作一个多功能工具，相信本书也能帮您实现这一点。这是一款可靠、好用又值得信赖的软件。希望本书能给您带来些许启发，帮助您运用 NGINX 解决棘手的现实问题。

本书的使用惯例

本书使用了以下印刷惯例：

斜体

表示新术语、URL、电子邮件地址、文件名和文件扩展名。

等宽

在程序列表中以及在段落中引用程序元素时使用，这些元素包括变量或函数名称、数据库、数据类型、环境变量、语句和关键字等。



此元素表示一般注释。



此元素表示警告或注意事项。

O'Reilly 在线学习平台

O'REILLY® 40 多年来, *O'Reilly Media* 一直致力于提供技术和商业培训、知识以及洞察, 帮助企业取得成功。

我们拥有一个独一无二的专家和创新者网络, 他们通过图书、文章和我们的在线学习平台分享知识和专业技能。O'Reilly 在线学习平台允许您按需访问实时培训课程、深度学习路径、交互式编码环境以及来自 O'Reilly 和其他 200 多家出版商的大量精选文本和视频。如欲了解更多信息, 请访问: <http://oreilly.com>

如何联系我们

如对本书有任何意见和问题, 请发送给出版商:

O'Reilly Media, Inc.
1005 Gravenstein Highway North
Sebastopol, CA 95472
800-889-8969 (美国或加拿大)
707-827-7019 (国际或本地)
707-829-0104 (传真)
support@oreilly.com
<https://www.oreilly.com/about/contact.html>

我们为本书建立了一个网页, 上面列出了勘误表、示例和任何其他信息, 访问地址:
<https://oreil.ly/nginx-cookbook-3e>

有关我们的图书和课程的新闻和信息, 请访问: <https://oreilly.com>

请关注我们的 LinkedIn: <https://linkedin.com/company/oreilly-media>

请关注我们的 Twitter: <https://twitter.com/oreillymedia>

请关注我们的 YouTube 视频: <https://youtube.com/oreillymedia>

基础知识

1.0 简介

无论是 NGINX 开源版还是 NGINX Plus，您都要先在系统上安装它们并学习一些基础知识。在本章中，您将学习如何安装 NGINX、主要配置文件位于何处以及管理命令是什么，还将了解如何验证您的安装并向默认服务器发出请求。

本书中的一些实操指南将使用 NGINX Plus。请访问 <https://nginx-cn.net>，获取 NGINX Plus 免费试用版。

1.1 在 Debian/Ubuntu 上安装 NGINX

问题

您想要在 Debian 或 Ubuntu 机器上安装 NGINX 开源版。

解决方案

更新已配置源的软件包信息，并安装一些有助于配置官方 NGINX 软件包仓库的软件包：

```
$ apt update
$ apt install -y curl gnupg2 ca-certificates lsb-release \
    debian-archive-keyring
```

下载并保存 NGINX 签名密钥：

```
$ curl https://nginx.org/keys/nginx_signing.key | gpg --dearmor \
    | tee /usr/share/keyrings/nginx-archive-keyring.gpg >/dev/null
```

使用 `lsb_release` 设置定义操作系统和版本名称的变量，然后创建一个 `apt` 源文件：

```
$ OS=$(lsb_release -is | tr '[:upper:]' '[:lower:]')
$ RELEASE=$(lsb_release -cs)
$ echo "deb [signed-by=/usr/share/keyrings/nginx-archive-keyring.gpg] \
    http://nginx.org/packages/${OS} ${RELEASE} nginx" \
    | tee /etc/apt/sources.list.d/nginx.list
```

再次更新软件包信息，然后安装 NGINX：

```
$ apt update
$ apt install -y nginx
$ systemctl enable nginx
$ nginx
```

详解

本节提供的命令可指示高级打包工具（APT）软件包管理系统使用官方 NGINX 软件包仓库。NGINX GPG 软件包签名密钥被下载并保存至文件系统的某个位置，以供 APT 使用。为 APT 提供签名密钥有助于 APT 系统验证仓库中的软件包。`lsb_release` 命令用于自动确定操作系统和版本名称，用户可以在所有 Debian 或 Ubuntu 发布版本中使用这些指令。`apt update` 命令指示 APT 系统从其已知的仓库中刷新软件包列表，之后您便可以从官方 NGINX 仓库安装 NGINX 开源版。安装后，最后一个命令会启动 NGINX。

1.2 通过 YUM 软件包管理器安装 NGINX

问题

您想要在 Red Hat Enterprise Linux (RHEL)、Oracle Linux、AlmaLinux、Rocky Linux 或 CentOS 上安装 NGINX 开源版。

解决方案

创建一个名为 `/etc/yum.repos.d/nginx.repo` 的文件，其中包含以下内容：

```
[nginx]
name=nginx repo
baseurl=http://nginx.org/packages/centos/$releasever/$basearch/
gpgcheck=0
enabled=1
```

修改文件，将 URL 中间的 OS 替换为 **rhel** 或 **centos**，具体取决于您的发行版本。然后运行以下命令：

```
$ yum -y install nginx
$ systemctl enable nginx
$ systemctl start nginx
$ firewall-cmd --permanent --zone=public --add-port=80/tcp
$ firewall-cmd --reload
```

详解

您刚创建的文件将指示 YUM 软件包管理系统使用官方 NGINX 开源版软件包仓库。后面的命令将从官方仓库安装 NGINX 开源版，指示 systemd 在启动时启用 NGINX，并告知它将其立即启动。如有必要，防火墙命令为传输控制协议（TCP）打开端口 80，这是 HTTP 的默认端口。最后一个命令重新加载防火墙，以提交更改。

1.3 安装 NGINX Plus

问题

您想要安装 NGINX Plus。

解决方案

访问 [NGINX 文档](#)。选择您要将 NGINX Plus 安装到哪个操作系统，然后按照说明进行操作。其安装说明与开源解决方案类似，但是您需要获取证书和密钥，以便对 NGINX Plus 仓库进行身份验证。

详解

NGINX 会及时更新这个仓库安装指南和 NGINX Plus 安装说明。这些说明因您的操作系统和版本而略有不同，但都有一个共同点：您必须从 NGINX 门户获取证书和密钥并提供给您的系统，以便对 NGINX Plus 仓库进行身份验证。

1.4 验证安装

问题

您希望验证 NGINX 是否安装成功并检查版本。

解决方案

您可以使用以下命令验证 NGINX 是否安装成功并检查其版本：

```
$ nginx -v
nginx version: nginx/1.25.3
```

如本例所示，该响应显示了其版本。

您可以使用以下命令确认 NGINX 是否正在运行：

```
$ ps -ef | grep nginx
root      1738      1  0 19:54 ?    00:00:00 nginx: master process
nginx     1739  1738  0 19:54 ?    00:00:00 nginx: worker process
```

ps 命令列出了正在运行的进程。通过将该命令导入到 grep 中，您可以在输出中搜索特定词。此示例使用 grep 搜索 nginx。结果显示有两个正在运行的进程：master 和 worker。如果 NGINX 正在运行中，您将始终可以看到一个 master 以及一个或多个 worker 进程。请注意，master 进程以 root 身份运行，因为默认情况下，NGINX 只有使用最高权限才能正常运行。有关启动 NGINX 的说明，请参阅下一实操指南。要了解如何将 NGINX 作为守护进程启动，请使用 init.d 或 systemd 方法。

要验证 NGINX 能否正确返回请求，请使用浏览器向您的机器发出请求或使用 curl。发送请求时，请使用机器的 IP 地址或主机名。如果安装在本地，您可以使用 localhost，如下所示：

```
$ curl localhost
```

您将看到 NGINX 欢迎页面默认的 HTML 站点。

详解

nginx 命令允许您与 NGINX 二进制文件交互，以便检查版本、列出已安装的模块、测试配置以及向 master 进程发送信号。NGINX 必须在运行时才能服务请求。ps 命令是一种确定 NGINX 是否正在运行（作为守护进程或是前台进程）的可靠方法。NGINX 默认提供的配置在端口 80 上运行静态站点 HTTP 服务器。为了测试这一默认站点，您可以使用 localhost 向机器发送 HTTP 请求，并应使用主机的 IP 地址和主机名。

1.5 关键文件、目录和命令

问题

您希望了解重要的 NGINX 目录和命令。

解决方案

以下配置目录和文件位置可在编译 NGINX 时更改，因此可能因安装情况而异。

NGINX 文件和目录

/etc/nginx/

/etc/nginx/ 目录是 NGINX 服务器的默认配置根，您可以从中找到指示 NGINX 如何运行的配置文件。

/etc/nginx/nginx.conf

/etc/nginx/nginx.conf 文件是 NGINX 守护进程使用的默认配置入口点。此配置文件能够为 worker 进程、调优、日志记录、动态模块的加载以及对其他 NGINX 配置文件的引用设置全局设置。在默认配置中，*/etc/nginx/nginx.conf* 文件包括顶层 http 代码块，也就是上下文，它提供了下述目录中的所有配置文件。

/etc/nginx/conf.d/

/etc/nginx/conf.d/ 目录包含默认的 HTTP 服务器配置文件，其中以 *.conf* 结尾的文件都包含在 */etc/nginx/nginx.conf* 文件的顶层 http 代码块中。最佳实践是利用 `include` 语句并以这种方式组织配置，从而保持配置文件的简洁。在某些软件包仓库中，此文件夹被命名为 *sites-enabled*，配置文件链接到 *site-available* 文件夹；此惯例已不再使用。

/var/log/nginx/

/var/log/nginx/ 目录是 NGINX 的默认日志位置，您可以从中找到一个 *access.log* 文件和 *error.log* 文件。默认情况下，访问日志包含 NGINX 服务的每条请求的条目。如果启用了 debug 模块，则错误日志文件包含错误事件和调试信息。

NGINX 命令

`nginx -h`

显示 NGINX 帮助菜单。

`nginx -v`

显示 NGINX 版本。

`nginx -V`

显示 NGINX 版本、build 信息和配置参数，这些参数显示了 NGINX 二进制文件中内置的模块。

```
nginx -t
```

测试 NGINX 配置。

```
nginx -T
```

测试 NGINX 配置并将验证后的配置打印到屏幕上。此命令在寻求支持时很有用。

```
nginx -s signal
```

-s 标记向 NGINX master 进程发送信号。您可以发送 stop、quit、reload 和 reopen 等信号。stop 信号可立即停止 NGINX 进程。quit 信号会在完成当前正在处理的请求后停止 NGINX 进程。reload 信号可重新加载配置。reopen 信号指示 NGINX 重新打开日志文件。

详解

在了解这些关键文件、目录和命令后，您就可以准备开始使用 NGINX 了。您可以运用这些知识更改默认配置文件，并使用 `nginx -t` 命令测试您的更改。如果测试成功，您还将了解到如何使用 `nginx -s reload` 命令指示 NGINX 重新加载配置。

1.6 使用 include 简化配置

问题

您想要清理庞杂的配置文件，将配置按逻辑划分为模块化配置集。

解决方案

使用 `include` 指令来引用配置文件、目录或掩码：

```
http {  
    include conf.d/compression.conf;  
    include ssl_config/*.conf  
}
```

`include` 指令中只包含一个参数，它可以是文件的路径或是与许多文件匹配的掩码。该指令在任何上下文中都有效。

详解

通过使用 `include` 语句，您可以保持 NGINX 配置清晰简洁。您可以对配置进行逻辑分组，以避免配置文件达到数百行。您可以创建模块化配置文件，然后将这些文件添加到配置中的多个位置，无需复制配置。

以 NGINX 大多数软件安装包中都提供的 *fastcgi_param* 配置文件为例。如果您是在单个 NGINX 上管理多台 FastCGI 虚拟服务器，则可以将此配置文件添加到需要这些 FastCGI 参数的任何位置或上下文中，而无需复制此配置。另一个示例是安全套接字层（SSL）配置。如果您正在运行需要类似 SSL 配置的多台服务器，只需编写此配置一次，然后将其添加到所需位置即可。

通过对配置进行逻辑分组，您可确保配置简洁有序。只需编辑单个文件即可更改一组配置文件，无需在大型配置文件中的多个位置更改多组配置块。对于从业人员来说，最好将配置分组到不同文件中并使用 `include` 语句。

1.7 提供静态内容

问题

您想要使用 NGINX 提供静态内容。

解决方案

使用以下 NGINX 配置示例覆盖位于 */etc/nginx/conf.d/default.conf* 的默认 HTTP 服务器配置：

```
server {
    listen 80 default_server;
    server_name www.example.com;

    location / {
        root /usr/share/nginx/html;
        # alias /usr/share/nginx/html;
        index index.html index.htm;
    }
}
```

详解

此配置通过 HTTP 在端口 80 上从目录 */usr/share/nginx/html/* 提供静态文件。第一行配置定义了一个新的 `server` 代码块，这定义了一个新上下文，指定 NGINX 要侦听的内容。第二行指示 NGINX 侦听端口 80，`default_server` 参数指示 NGINX 使用此服务器作为端口 80 的默认上下文。`listen` 指令也可以使用一系列端口。`server_name` 指令定义了主机名或应定向到此服务器的请求的名称。如果配置没有将此上下文定义为 `default_server`，那么只有当 HTTP 主机请求头与提供给 `server_name` 指令的值相匹配时，

NGINX 才会将请求定向到这台服务器。如果您还没有要使用的域名，则可以通过设置 `default_server` 上下文省略 `server_name` 指令。

`location` 代码块根据 URL 中的路径定义配置。路径或域之后的部分 URL 被称为统一资源标识符 (URI)。NGINX 会将请求的 URI 关联到最匹配的 `location` 代码块。示例使用了 `/` 匹配所有请求。`root` 指令向 NGINX 显示了为给定上下文提供内容时应从何处查找静态文件。在查找请求的文件时，请求的 URI 会附加到 `root` 指令的值。如果我们为 `location` 指令提供了 URI 前缀，那么除非我们使用 `alias` 指令（而非 `root`），否则该前缀将包含在附加路径中。`location` 指令能够匹配一系列广泛的表达式。更多信息，请访问“其他参考资料”部分中的第一个链接。最后，`index` 指令为 NGINX 提供了一个默认文件或要检查的文件列表，以防 URI 中没有提供进一步的路径。

其他参考资料

[NGINX HTTP location 指令文档](#)

[NGINX 请求处理](#)

高性能负载均衡

2.0 简介

如今的互联网用户体验离不开出色的性能和正常运行时间。为此，企业需要运行多个相同的副本，并将负载分散在整个系统集群上。随着负载的增加，集群内会引入新的副本。这种架构技术称为“水平扩展”。基于软件的基础架构因其灵活性而愈发受欢迎，并创造了更多的可能性。无论是仅有两个系统副本组成的高可用性方案，还是全球范围内成千上万个系统构成的大型集群，它们都需要一款像基础架构一样动态的负载均衡解决方案。NGINX 能够以多种方式满足这一需求，例如 HTTP、传输控制协议（TCP）和用户数据报协议（UDP）负载均衡，本章将会对这些内容进行详细介绍。

在实施负载均衡时，必须要保证对客户体验产生完全正向的影响。许多现代 Web 架构采用无状态应用层，将状态存储在共享内存或数据库中。但是并非所有架构都是如此。会话状态不仅蕴含着重大价值，而且在交互式应用中得以广泛使用。出于多种原因——例如在需要处理大量数据的应用中，为了避免性能方面的高昂网络开销，这个状态可能就会存储到本地的应用服务器上。在这种情况下，后续的请求会被继续交付到同一台服务器，这对保障用户体验极其重要。与此同时，在会话完成之前，服务器不会得到释放。大规模使用有状态应用需要智能负载均衡器。NGINX 提供了多种跟踪 cookie 或路由的办法来解决这一问题。本章介绍了会话保持，因为它与 NGINX 的负载均衡有关。

确保 NGINX 所服务的应用的健康状态十分重要。出于多种原因，上游（upstream）请求可能会失败，例如由于网络连接不佳、服务器故障或应用故障等等。代理和负载均衡器必须足够智能，以检测上游服务器（负载均衡器或代理后面的服务器）的故障，并停止向它们传输流量；否则客户端只能等待，最终超时。

当服务器出现故障时，一种避免服务降级的方法是让代理检查上游服务器的健康状况。NGINX 提供了两种不同类型的健康检查方法：被动式（NGINX 开源版提供）和主动式（仅 NGINX Plus 提供）。定期执行的主动健康检查会与上游服务器建立连接或向其发送请求，并且验证响应是否正确。被动健康检查能够在客户端发出请求或建立连接时监控上游服务器的连接或响应。您可能希望使用被动健康检查来减少上游服务器的负载，并使用主动健康检查确定上游服务器的故障，以免引发客户端服务失败。本章最后探讨了如何监控正在实施负载均衡的上游应用服务器的健康状况。

2.1 HTTP 负载均衡

问题

您希望将负载分发到两台或多台 HTTP 服务器。

解决方案

在 NGINX 的 HTTP 模块内使用 upstream 代码块对 HTTP 服务器实施负载均衡：

```
upstream backend {
    server 10.10.12.45:80 weight=1;
    server app.example.com:80 weight=2;
    server spare.example.com:80 backup;
}
server {
    location / {
        proxy_pass http://backend;
    }
}
```

该配置对端口 80 的两台 HTTP 服务器实施负载均衡，然后再将另一台服务器定义为 backup，以便在两台主服务器不可用时发挥作用。可选的 weight 参数指示 NGINX 向第二台服务器传输两倍的请求。未使用时，它的默认值为 1。

详解

HTTP 的 `upstream` 模块控制着 HTTP 请求负载均衡。该模块定义了一个目标池——它可以是 Unix 套接字、IP 地址和服务器主机名的任意组合，也可以是它们的混合使用配置。`upstream` 模块还定义了如何将任一个请求分发给任何上游（`upstream`）服务器。

每个上游目标都通过 `server` 指令在上游池中进行定义。除了上游服务器地址以外，`server` 指令还接收可选参数。可选参数能够增强对请求路由的控制。这包括均衡算法中服务器的 `weight` 参数（无论服务器处于待机模式、可用还是不可用），以及确定服务器是否不可用的参数。NGINX Plus 还提供了许多其他好用的参数，例如对服务器的连接限制、高级 DNS 解析控制以及在服务器启动后缓慢增加与服务器的连接等等。

2.2 TCP 负载均衡

问题

您希望将负载分发到两台或多台 TCP 服务器。

解决方案

在 NGINX 的 `stream` 模块内使用 `upstream` 代码块对 TCP 服务器实施负载均衡：

```
stream {
    upstream mysql_read {
        server read1.example.com:3306 weight=5;
        server read2.example.com:3306;
        server 10.10.12.34:3306 backup;
    }

    server {
        listen 3306;
        proxy_pass mysql_read;
    }
}
```

此示例中的 `server` 代码块指示 NGINX 侦听 TCP 端口 3306，并在两个 MySQL 数据库读取副本之间实施负载均衡，同时将另一台服务器定义为 `backup`，以便在主服务器崩溃时接管流量。

此配置不会被添加到 *conf.d* 文件夹中，因为默认情况下，该文件夹包含在 http 代码块中；您应该另行创建名为 *stream.conf.d* 的文件夹，打开 *nginx.conf* 文件中的 stream 代码块，添加新文件夹以支持 stream 配置。示例见下。

在 */etc/nginx/nginx.conf* 配置文件中：

```
user nginx;
worker_processes auto;
pid /run/nginx.pid;

stream {
    include /etc/nginx/stream.conf.d/*.conf;
}
```

名为 */etc/nginx/stream.conf.d/mysql_reads.conf* 的文件可能包含以下配置：

```
upstream mysql_read {
    server read1.example.com:3306 weight=5;
    server read2.example.com:3306;
    server 10.10.12.34:3306 backup;
}

server {
    listen 3306;
    proxy_pass mysql_read;
}
```

详解

http 和 stream 上下文之间的主要区别在于它们在 OSI 模型的不同层上运行。http 上下文在应用层（七层）运行，stream 在传输层（四层）运行。这并不意味着 stream 上下文不能通过一些巧妙的脚本获得应用感知能力，而是说 http 上下文是专门为了完全理解 HTTP 协议而设计的，stream 上下文默认情况下只能对数据包进行路由和负载均衡。

TCP 负载均衡由 NGINX 的 stream 模块定义。stream 模块与 HTTP 模块一样，允许您定义上游（upstream）服务器池并配置侦听服务器。在配置服务器侦听给定端口时，您必须定义待侦听的端口或者地址加端口（可选）。然后您必须配置目标服务，无论这是连接另一个地址的直接反向代理还是上游资源池。

配置中有许多选项可以改变 TCP 连接反向代理的属性，包括 SSL/TLS 验证限制、超时和 keepalive 等。这些代理选项的一些值可以是（或者包含）变量，例如下载速率、验证 SSL/TLS 证书时使用的名称等。

TCP 与 HTTP 负载均衡中的 `upstream` 指令非常相似，它们均将上游资源定义为服务器，配置格式同样为 Unix 套接字、IP 或 FQDN，此外服务器 `weight` 参数、最大连接数、DNS 解析器、连接数缓增期以及判断服务器是激活状态、故障状态还是备用模式的参数也都相似。

NGINX Plus 甚至提供了更多 TCP 负载均衡特性，这些高级特性贯穿整份指南。本章稍后将介绍所有负载均衡的健康检查。

2.3 UDP 负载均衡

问题

您希望将负载分发到两台或多台 UDP 服务器。

解决方案

在 NGINX 的 `stream` 模块内使用 `upstream` 代码块（定义为 `udp`）对 UDP 服务器实施负载均衡：

```
stream {
    upstream ntp {
        server ntp1.example.com:123 weight=2;
        server ntp2.example.com:123;
    }

    server {
        listen 123 udp;
        proxy_pass ntp;
    }
}
```

这部分的配置对使用 UDP 协议的两台上游（`upstream`）网络时间协议（NTP）服务器实施了负载均衡。UDP 负载均衡的指定非常简单，只需使用 `listen` 指令中的 `udp` 参数便可。

如果进行负载均衡的服务需要在客户端和服务器之间来回发送多个数据包，则可以指定 `reuseport` 参数。例如，OpenVPN、互联网语音协议（VoIP）、虚拟桌面解决方案和数据报传输层安全（DTLS）都是这种类型的服务。下面举例说明了如何使用 NGINX 处理 OpenVPN 连接并将其代理到本地运行的 OpenVPN 服务：

```
stream {
    server {
        listen 1195 udp reuseport;
        proxy_pass 127.0.0.1:1194;
    }
}
```

详解

您可能会问：“我的 DNS A 或服务记录（SRV 记录）中已经有多个主机了，又何必再要负载均衡器呢？”原因是我们不仅有备选的负载均衡算法，我们还可以对 DNS 服务器本身实施负载均衡。UDP 服务构成了我们在网络系统中依赖的许多服务，例如 DNS、NTP、QUIC、HTTP/3 和 VoIP。UDP 负载均衡可能对某些企业来说不太常见，但在大型环境中十分有用。

与 TCP 类似，您可以在 stream 模块中找到 UDP 负载均衡，并以同样的方式完成大部分配置。两者的主要区别在于，UDP 负载均衡的 listen 指令指定打开的套接字用于处理 UDP 数据报（datagram）。此外，在处理数据报时，UDP 负载均衡还提供了 TCP 所没有的一些其他指令，例如 proxy_response 指令，它向 NGINX 指定了可以从上游服务器发送多少预期的响应。默认情况下，除非达到 proxy_timeout 限制，否则这一数量是无限的。proxy_timeout 指令设置了在连接关闭之前，客户端或代理服务器连接连续进行两次读取或写入操作之间的时间。

reuseport 参数指示 NGINX 为每个 worker 进程创建一个单独的侦听套接字。这允许内核在 worker 进程之间分发传入的连接，以处理在客户端和服务器之间发送的多个数据包。reuseport 功能仅适用于 Linux kernels 3.9 及更高版本、DragonFly BSD、FreeBSD 12 及更高版本。

2.4 负载均衡方式

问题

轮询负载均衡不适合您的用例，因为您有异构工作负载或服务器池。

解决方案

使用 NGINX 的负载均衡方法之一，例如最少连接、最短时间、通用哈希、随机算法或 IP 哈希。此示例将后端上游（upstream）池的负载均衡算法设为了选择连接数最少的服务器：

```
upstream backend {
    least_conn;
    server backend.example.com;
    server backend1.example.com;
}
```

除了通用哈希、随机算法和最短时间外，所有其他负载均衡算法都是独立的指令，如上例所示。下面的“详解”一节解释了这些指令的参数。

下一个示例使用通用哈希算法和 `$remote_addr` 变量。它采用了与 IP 哈希相同的路由算法；不过，通用哈希可在 `stream` 上下文中使用，而 IP 哈希仅在 `http` 上下文中可用。您可以替换所用的变量，或者添加更多变量来改变通用哈希算法分配负载的方式。在下面的示例中，`upstream` 代码块被配置为使用客户端 IP 地址和通用哈希算法：

```
upstream backend {
    hash $remote_addr;
    server backend.example.com;
    server backend1.example.com;
}
```

详解

并非所有请求或数据包都有相同的权重。有鉴于此，轮询甚至上例使用的加权轮询都将无法满足所有应用或流量的需求。NGINX 提供了多种负载均衡算法，您可以根据特定的用例进行选择，也可以对您选择的算法进行配置。以下负载均衡方法（IP 哈希除外）可用于 HTTP、TCP 和 UDP 上游池：

轮询

轮询是默认的负载均衡方法，它按照上游池中服务器列表的顺序分发请求。当上游服务器的容量变化时，您还可以考虑使用加权轮询。权重的整数值越高，服务器在轮询中的优势就越大。权重背后的算法只是加权平均值的统计概率。

最少连接

此方法通过将当前请求代理到打开连接数最少的上游服务器实现负载均衡。与轮询一样，在决定将连接发送到哪台服务器时，最少连接也会考虑权重。其指令名称是 `least_conn`。

最短时间

该算法仅在 NGINX Plus 中提供，与最少连接算法类似，它将请求代理到当前连接数最少的上游服务器，但首选平均响应时间最短的服务器。此方法是最复杂的负载均衡算法之一，能够满足高性能 Web 应用的需求。最短时间在最少连接的基础上进行了优化，因为少量连接并不一定意味着最快的响应。使用此算法时，切记要考虑服务请求时间的统计差异。有些请求可能本身就需要更多的处理，请求时间也就更长，因而拉宽了统计的范围。请求时间长并不一定意味着服务器性能欠佳或超负荷工作。但是，需要进行更多处理的请求可以考虑使用异步工作流。用户必须为此指令指定 `header` 或 `last_byte` 参数。当指定 `header` 时，使用接收响应头的时间；当指定 `last_byte` 时，使用接收完整响应的的时间。如果指定了 `inflight` 参数，那么不完整的请求也会被考虑在内。其指令名称是 `least_time`。

通用哈希

管理员使用请求或运行时给定的文本、变量或两者的组合定义哈希值。NGINX 能够为当前请求生成哈希值并将其放在上游服务器上，从而在这些服务器之间分发负载。当您希望更好地控制请求的发送位置或确定哪台上游服务器最有可能缓存数据时，此方法就会派上用场。请注意，当从池中添加或删除服务器时，将重新分发哈希请求。此算法有一个可选的参数：`consistent`，它能够将重新分发带来的影响最小化。其指令名称是 `hash`。

随机

该方法用于指示 NGINX 从组中随机选择一台服务器，同时考虑服务器的权重。可选的 `two [method]` 参数指示 NGINX 随机选择两台服务器，然后使用提供的负载均衡方法对两者均匀地分发请求。默认情况下，如果传输的参数只有 `two`，没有 `method`，则使用 `least_conn` 方法。随机负载均衡的指令名称是 `random`。

IP 哈希

此方法仅适用于 HTTP。IP 哈希算法使用客户端 IP 地址作为哈希。IP 哈希与通用哈希存在细微的不同，前者使用 IPv4 地址的前三个八进制位或整个 IPv6 地址，而后者使用的是远程变量。当会话状态十分重要，但又无法通过应用的共享内存进行处理时，此方法可确保客户端始终被代理到同一上游服务器（只要服务器可用）。此方法在分发哈希值时也考虑了 `weight` 参数。其指令名称是 `ip_hash`。

2.5 NGINX Plus 之 Sticky Cookie

问题

您想要使用 NGINX Plus 将下游（downstream）客户端绑定到上游（upstream）服务器。

解决方案

使用 `sticky cookie` 指令指示 NGINX Plus 创建和跟踪 cookie：

```
upstream backend {  
    server backend1.example.com;  
    server backend2.example.com;  
    sticky cookie  
        affinity  
        expires=1h  
        domain=.example.com  
        httponly  
        secure  
        path=/;  
}
```

此配置创建并跟踪了一个 cookie，该 cookie 可以将下游客户端绑定到上游服务器。在此示例中，cookie 被命名为 `affinity`，为 `example.com` 设置，有效期为一个小时，无法在客户端使用，只能通过 HTTPS 发送，并且对所有路径有效。

详解

在 `sticky` 指令中使用 `cookie` 参数，为第一个请求创建一个包含上游服务器信息的 cookie。NGINX Plus 将跟踪此 cookie，允许它继续将后续请求定向到同一台服务器。cookie 参数的第一个位置参数是待创建和跟踪的 cookie 的名称。其他参数提供了额外的控制，为浏览器提供了相应的使用信息——如有效期、域、路径，以及 cookie 是否可以在客户端使用，或者 cookie 是否可以通过不安全的协议传递。cookie 是 HTTP 协议的一部分，因此 `sticky cookie` 仅可在 http 上下文中使用。

2.6 NGINX Plus 之 Sticky Learn

问题

您想要使用 NGINX Plus 的现有 cookie 将下游 (downstream) 客户端绑定到上游 (upstream) 服务器。

解决方案

使用 `sticky learn` 指令发现和跟踪上游应用创建的 cookie:

```
upstream backend {  
    server backend1.example.com:8080;  
    server backend2.example.com:8081;  
  
    sticky learn  
        create=$upstream_cookie_cookie_name  
        lookup=$cookie_cookie_name  
        zone=client_sessions:1m;  
}
```

此示例指示 NGINX 通过在响应头中查找名为 `COOKIE_NAME` 的 cookie 来查找和跟踪会话，并通过在请求头中查找相同的 cookie 来查找现有会话。此会话的 affinity 被存储在一个 1 MB 的共享内存区中，后者可以跟踪大约 4,000 个会话。cookie 的名称始终是应用专有的。常用的 cookie 名称 (如 `jsessionid` 或 `phpsessionid`) 通常是应用或应用服务器配置中的默认设置。

详解

当应用创建自己的会话状态 cookie 时，NGINX Plus 可以在请求响应中发现它们并跟踪它们。当在 `sticky` 指令中使用 `learn` 参数时，就会执行这种类型的 cookie 跟踪。跟踪 cookie 的共享内存由 `zone` 参数指定，包括名称和大小。NGINX Plus 通过指定 `create` 参数来从上游服务器的响应中查找 cookie，并使用 `lookup` 参数搜索之前注册的服务器 affinity。这些参数的值是 HTTP 模块暴露的变量。

2.7 NGINX Plus 之 Sticky Routing

问题

您想要使用 NGINX Plus 对持久会话路由到上游（upstream）服务器的方式进行细粒度控制。

解决方案

通过带 route 参数的 sticky 指令使用描述请求的变量进行路由：

```
map $cookie_jsessionid $route_cookie {
    ~.+\\.?(?P<route>\\w+)$ $route;
}

map $request_uri $route_uri {
    ~jsessionId=.+\\.?(?P<route>\\w+)$ $route;
}

upstream backend {
    server backend1.example.com route=a;
    server backend2.example.com route=b;

    sticky route $route_cookie $route_uri;
}
```

此示例尝试提取 Java 会话 ID，首先通过将 Java 会话 ID cookie 映射到第一个 map 代码块的变量来提取，然后查找请求 URI 中名为 jsessionid 的参数，将值映射到使用第二个 map 代码块的变量。带有 route 参数的 sticky 指令可以传输任意数量的变量。第一个非零或非空值用于路由。如果使用 jsessionid cookie，请求将被路由到 backend1；如果使用 URI 参数，请求将被路由到 backend2。尽管此示例使用的是 Java 通用会话 ID，但这也同样适用于其他会话技术（如 phpsessionId）或者您的应用为会话 ID 生成的任何有保证的唯一标识符。

详解

有时，您可能希望利用更细粒度的控制，将流量定向到特定的服务器。sticky 指令的 route 参数就是为了实现这个目标而构建的。与通用哈希负载均衡算法相比，sticky route 能够为您提供更好的控制、实际跟踪能力和粘性。该算法会先根据指定的路线将客户端路由到上游服务器，随后的请求则会在 cookie 或 URI 中携带路由信息。sticky

route 采用了许多要计算的位置参数。第一个非空变量用于路由到服务器。map 代码块可以选择性地解析变量，并将它们保存为要在路由中使用的其他变量。本质上，sticky route 指令会在 NGINX Plus 共享内存区内创建一个会话，用于跟踪为上游服务器指定的任何客户端会话标识符，从而始终一致地将具有此会话标识符的请求传输到与原请求相同的上游服务器。

2.8 NGINX Plus 之连接清空

问题

出于维护或其他原因，您需要使用 NGINX Plus 优雅地移除服务器，同时仍然提供会话。

解决方案

借助 NGINX Plus API（详见第 5 章），使用 drain 参数指示 NGINX 停止发送未被跟踪的新连接：

```
$ curl -X POST -d '{"drain":true}' \
  'http://nginx.local/api/9/http/upstreams/backend/servers/0'

{
  "id":0,
  "server":"172.17.0.3:80",
  "weight":1,
  "max_conns":0,
  "max_fails":1,
  "fail_timeout":"10s",
  "slow_start":"0s",
  "route":"",
  "backup":false,
  "down":false,
  "drain":true
}
```

详解

当会话状态被存储到本地服务器时，必须要先清空连接和持久会话，然后再从池中删除服务器。连接清空是指在从上游（upstream）池中删除服务器之前，先让服务器会话在本地失效的过程。您可以通过将 drain 参数添加到 server 指令来配置特定服务器的清空。如果设置了 drain 参数，NGINX Plus 会停止向该服务器发送新会话，但允许在

当前会话长度内继续提供当前会话。您可以通过将 `drain` 参数添加到上游服务器指令来切换此配置，然后重新加载 NGINX Plus 配置。

2.9 被动健康检查

问题

您想要被动检查上游（upstream）服务器的健康状况，确保其成功提供代理流量。

解决方案

通过 NGINX 健康检查和负载均衡确保只使用健康的上游服务器：

```
upstream backend {  
    server backend1.example.com:1234 max_fails=3 fail_timeout=3s;  
    server backend2.example.com:1234 max_fails=3 fail_timeout=3s;  
}
```

此配置能够监控定向到上游服务器的客户端请求的响应，从而被动监控上游服务器的健康状况。该示例将 `max_fails` 指令设置为 3，将 `fail_timeout` 设置为 3 秒。这些指令参数在 stream 和 HTTP 服务器中的工作原理相同。

详解

NGINX 开源版提供了被动健康检查功能，并且使用了相同的 `server` 参数来实施 HTTP、TCP 和 UDP 负载均衡。当客户端发出请求时，被动监控功能可以监测通过 NGINX 的失效或超时连接。默认情况下启用被动健康检查；此处提到的参数允许您调整它们的行为。`max_fails` 的默认值为 1，`fail_timeout` 的默认值为 10s。健康监控在所有类型的负载均衡中都很重要，这不仅是为了保障用户体验，也是为了实现业务连续性。NGINX 能够被动监控上游 HTTP、TCP 和 UDP 服务器，确保它们健康、高效地运行。

其他参考资料

[HTTP 健康检查管理指南](#)

[TCP 健康检查管理指南](#)

[UDP 健康检查管理指南](#)

2.10 NGINX Plus 之主动健康检查

问题

您想要使用 NGINX Plus 主动检查上游（upstream）服务器的健康状况，确保其随时提供代理流量。

解决方案

对于 HTTP，请使用 location 代码块中的 health_check 指令：

```
http {
    server {
        # ...
        location / {
            proxy_pass http://backend;
            health_check interval=2s
                fails=2
                passes=5
                uri=/
                match=welcome;
        }
    }
    # 状态码是 200, 内容类型是 "text/html",
    # 正文包含 "Welcome to nginx!"
    match welcome {
        status 200;
        header Content-Type = text/html;
        body ~ "Welcome to nginx!";
    }
}
```

此处的 HTTP 服务器健康检查配置通过每 2 秒向 URI "/" 发送 HTTP GET 请求来检查上游服务器的健康状况。我们无法为健康检查定义 HTTP 方法，只能执行 GET 请求，因为其他方法可能会更改后端系统的状态。上游服务器只有连续通过五次健康检查才能被认为是健康的；如果连续两次未通过检查，则被认定为不健康并从池中移除。上游服务器的响应必须匹配定义的 match 代码块，后者将状态码定义为 200，将请求头的 Content-Type 值定义为 'text/html'，同时定义了响应正文中的字符串 "Welcome to nginx!"。HTTP match 代码块具有三个指令：status、header 和 body。这三个指令均带有比较标记。

TCP/UDP 服务的 stream 健康检查非常相似：

```
stream {
    # ...
    server {
```

```

listen 1234;
proxy_pass stream_backend;
health_check interval=10s
    passes=2
    fails=3;
health_check_timeout 5s;
}
# ...
}

```

在此示例中，TCP 服务器配置为侦听端口 1234，并代理到一组上游服务器，它将主动检查这些服务器的健康状况。除了 `uri` 之外，`stream health_check` 指令与 HTTP 中的其他参数都相同，并且 `stream` 版本有一个可以将检查协议切换到 `udp` 的参数。在此示例中，间隔时间设为 10 秒，规定通过两次被视为健康，失败三次被视为不健康。主动 `stream` 健康检查也能验证来自上游服务器的响应。但是 `stream` 服务器的 `match` 代码块只有两个指令：`send` 和 `expect`。`send` 指令是要发送的原始数据，`expect` 是确切的响应或要匹配的正则表达式。

详解

NGINX Plus 允许使用被动或主动健康检查来监控源服务器。这些健康检查可以测量的不仅仅是响应代码。在 NGINX Plus 中，主动 HTTP 健康检查根据一系列有关上游服务器响应的接受标准实施监控。您可以对主动健康检查的监控进行配置，包括上游服务器的检查频率、服务器必须通过多少次检查才能被视为健康、失败多少次会被视为不健康以及预期的结果应该是什么。对于更复杂的逻辑，`match` 代码块的 `require` 指令允许使用值不能为空或零的变量。`match` 参数指向了定义响应接受标准的 `match` 代码块。当在 TCP/UDP 的 `stream` 上下文中使用时，`match` 代码块还定义了要发送到上游服务器的数据。这些特性使得 NGINX 能够确保上游服务器始终处于健康状态。

其他参考资料

[HTTP 健康检查管理指南](#)

[TCP 健康检查管理指南](#)

[UDP 健康检查管理指南](#)

2.11 NGINX Plus 之慢启动

问题

您的应用需要缓慢增加流量，直至承担全部生产负载。

解决方案

当上游（upstream）负载均衡池重新引入服务器时，使用 `server` 指令中的 `slow_start` 参数控制在指定的时间内逐渐增加连接数：

```
upstream {  
    zone backend 64k;  
  
    server server1.example.com slow_start=20s;  
    server server2.example.com slow_start=15s;  
}
```

`server` 指令配置将在上游池重新引入服务器后缓慢增加流量。`server1` 和 `server2` 将分别在 20 秒和 15 秒内缓慢增加连接数量。

详解

“慢启动”是指在一段时间内缓慢增加代理到服务器的请求数量的概念。慢启动允许应用通过填充缓存和初始化数据库连接来“热身”，而不至于启动就被“汹涌而来”的连接压垮。当健康检查失败的服务器开始再次通过检查并重新进入负载均衡池时，该功能就会派上用场，并且仅在 NGINX Plus 中提供。慢启动不能与 `hash`、`IP hash` 或 `random` 负载均衡方法一起使用。

3.0 简介

NGINX 也属于 Web 流量控制器。您可以使用 NGINX 智能地路由流量，并根据多个属性控制流量。本章介绍了 NGINX 的多项功能，包括：按比例分配客户端请求；利用客户端的地理位置；通过速率、连接和带宽限制控制流量。阅读时请注意，您可以混合搭配这些功能来解锁更多可能。

3.1 A/B 测试

问题

您希望在文件或应用的两个或多个版本之间精分客户端流量，以测试接受度或参与度。

解决方案

使用 `split_clients` 模块将一定比例的客户端流量定向到一个不同的上游（upstream）池：

```
split_clients "${remote_addr}AAA" $variant {  
    20.0% "backendv2";  
    * "backendv1";  
}
```

`split_clients` 指令对作为第一个参数提供的字符串进行哈希处理，该哈希值将用第三个参数的键（key）所对应的值来分配并映射到作为第二个参数提供的变量的值。在第一个参数中添加 AAA 是为了证明这是一个可以包含多个变量的串联字符串，如通用哈

希负载均衡算法中所述。第三个参数是一个包含键值（key-value）对的对象，其中键是百分比权重，值是要分配的值。键可以是百分比或星号。星号表示取完所有百分比后的剩余部分。对于 `$variant` 变量的值，`backendv2` 为客户端 IP 地址流量的 20%，`backendv1` 为其余 80%。

在此示例中，`backendv1` 和 `backendv2` 表示上游服务器池，可以与 `proxy_pass` 指令一起使用，如下所示：

```
location / {  
    proxy_pass http://$variant  
}
```

使用变量 `$variant` 时，我们的流量将会分配给两个不同的应用服务器池。

为了理解 `split_clients` 的广泛用途，我们通过一个例子来看看两个静态站点版本之间的流量精分：

```
http {  
    split_clients "${remote_addr}" $site_root_folder {  
        33.3% "/var/www/sitev2/";  
        * "/var/www/sitev1/";  
    }  
    server {  
        listen 80 _;  
        root $site_root_folder;  
        location / {  
            index index.html;  
        }  
    }  
}
```

详解

在测试电子商务网站上不同类型的营销和前端功能的转化率时，这种类型的 A/B 测试非常有用。应用通常通过“灰度发布”进行部署。在这种类型的部署中，用户可以逐步增加路由到新版本的用户百分比，进而将流量慢慢切换到新版本。在推出新版本的代码时，在不同的应用版本之间精分客户端流量非常实用，这可以降低发生错误后带来的影响。更常见的部署模式是蓝绿部署，也就说在将用户切换到新版本的同时，仍然保持旧版本在验证部署期间的可用性。无论出于何种原因在两组不同的应用之间精分客户端流量，NGINX 的 `split_client` 模块都可以帮您轻松搞定。

其他参考资料

[split_clients 模块文档](#)

3.2 使用 GeoIP 模块和数据库

问题

您想要安装 GeoIP 数据库并启用它在 NGINX 中的嵌入式变量，以利用 NGINX 日志、被代理请求或请求路由中客户端的物理位置。

解决方案

安装 NGINX 时，官方 NGINX 开源版软件包仓库（配置方式见[第 2 章](#)）提供了一个名为 `nginx-module-geoip` 的软件包。当使用 NGINX Plus 软件包仓库时，该软件包被命名为 `nginx-plus-module-geoip`。不过，NGINX Plus 为 GeoIP2 提供了一个动态模块——这个更新的模块可与 NGINX stream 和 HTTP 配合使用。GeoIP2 模块将稍后在本节中进行介绍。以下示例展示了如何安装动态 NGINX GeoIP 模块软件包，以及如何下载 GeoIP 国家/地区和城市数据库。值得注意的是，原始 GeoIP 模块的数据库不再予以维护。

使用 YUM 软件包管理器的 NGINX 开源版：

```
$ yum install nginx-module-geoip
```

使用 APT 软件包管理器的 NGINX 开源版：

```
$ apt install nginx-module-geoip
```

使用 YUM 软件包管理器的 NGINX Plus：

```
$ yum install nginx-plus-module-geoip
```

使用 APT 软件包管理器的 NGINX Plus：

```
$ apt install nginx-plus-module-geoip
```

下载 GeoIP 国家/地区和城市数据库并解压：

```
$ mkdir /etc/nginx/geoip
$ cd /etc/nginx/geoip
$ wget "http://geolite.maxmind.com/download/geoip/database/GeoLiteCountry/GeoIP.dat.gz"
$ gunzip GeoIP.dat.gz
$ wget "http://geolite.maxmind.com/download/geoip/database/GeoLiteCity.dat.gz"
$ gunzip GeoLiteCity.dat.gz
```

这组命令在 `/etc/nginx` 目录下创建了一个 `geoip` 目录，请移动到这个新目录，下载并解压软件包。

将 GeoIP 国家/地区和城市数据库保存到本地磁盘后，您就可以指示 NGINX GeoIP 模块使用它们根据客户端 IP 地址公开嵌入式变量了：

```
load_module modules/nginx_http_geoip_module.so;

http {
    geoip_country /etc/nginx/geoip/GeoIP.dat;
    geoip_city /etc/nginx/geoip/GeoLiteCity.dat;
    # ...
}
```

`load_module` 指令从它在文件系统中的路径动态加载模块。`load_module` 指令仅在 `main` 上下文中有有效。`geoip_country` 指令获取了 `GeoIP.dat` 文件的路径，该文件包含将 IP 地址映射到国家/地区代码的数据库，并且仅在 `http` 上下文中有有效。

GeoIP2 模块可用作**动态模块**并在 NGINX 开源版构建之时进行编译。使用 GeoIP2 模块编译 NGINX 开源版不在本书的论述范围内。以下内容将说明如何安装 `nginx-plus-module-geoip2` 模块。

使用 APT 软件包管理器为 GeoIP2 安装 NGINX Plus 动态模块：

```
$ apt install nginx-plus-module-geoip2
```

使用 YUM 软件包管理器为 GeoIP2 安装 NGINX Plus 动态模块：

```
$ yum install nginx-plus-module-geoip2
```

加载 GeoIP2 的动态模块：

```
load_module modules/nginx_http_geoip2_module.so;
load_module modules/nginx_stream_geoip2_module.so;

http {
    # ...
}
```

下载**免费的 MaxMind GeoLite2 数据库（需要注册）**。然后配置 NGINX 以使用数据库，并通过变量公开 Geo 信息：

```
http {
    ...
    geoip2 /etc/maxmind-country.mmdb {
        auto_reload 5m;
        $geoip2_metadata_country_build metadata build_epoch;
        $geoip2_data_country_code default=US
        source=$variable_with_ip country iso_code;
        $geoip2_data_country_name country names en;
    }
}
```

```

}

geoip2 /etc/maxmind-city.mdb {
    $geoip2_data_city_name default=London city names en;
}

....

fastcgi_param COUNTRY_CODE $geoip2_data_country_code;
fastcgi_param COUNTRY_NAME $geoip2_data_country_name;
fastcgi_param CITY_NAME    $geoip2_data_city_name;
....
}

stream {
    ...
    geoip2 /etc/maxmind-country.mdb {
        $geoip2_data_country_code default=US
        source=$remote_addr country iso_code;
    }
    ...
}

```

详解

要使用此功能，您必须先安装 NGINX GeoIP 或 GeoIP2 模块以及本地 GeoIP 国家/地区和城市数据库，本节演示了相关安装和检索教程。

在原始 GeoIP 模块中，`geoip_country` 和 `geoip_city` 指令暴露了该模块中可用的多个嵌入式变量。`geoip_country` 指令允许使用一些变量来区分客户端的来源地/国。这些变量包括 `$geoip_country_code`、`$geoip_country_code3` 和 `$geoip_country_name`。`country code` 变量返回由两个字母组成的国家/地区代码，以 3 结尾的变量返回由三个字母组成的国家/地区代码。`country name` 变量返回国家/地区的全名。

`geoip_city` 指令启用了相当一部分变量。`geoip_city` 指令启用了与 `geoip_country` 指令相同的所有变量，只不过名称不一样，例如 `$geoip_city_country_code`、`$geoip_city_country_code3` 和 `$geoip_city_country_name`。其他变量包括 `$geoip_city`、`$geoip_latitude`、`$geoip_longitude`、`$geoip_city_continent_code` 和 `$geoip_postal_code`，所有这些都是它们返回的值的描述性变量。`$geoip_region` 和 `$geoip_region_name` 描述了地区、领地、州、省、联邦土地等。`region` 是由两个字母组成的代码，而 `region name` 是全名。`$geoip_area_code`（仅在美国有效）返回三位电话区号。

使用 GeoIP2 模块时, `geoip2` 指令暴露了相同的变量, 但变量的前缀是 `geoip2_data_`, 而非 `geoip_`。 `geoip2` 指令还配置默认值, 以及从 MaxMind 重新加载数据库的时间间隔。

您可以利用这些变量记录您的客户端信息。您可以选择性地将此信息作为请求头或变量传输给应用, 也可以使用 NGINX 以特定的方式路由流量。

其他参考资料

[geoip 模块文档](#)

[GeoIP Update GitHub](#)

[NGINX GeoIP2 动态模块文档](#)

[GitHub 上的 GeoIP2 源代码库及文档](#)

3.3 基于国家 / 地区的访问限制

问题

您想要根据合同或应用要求限制来自特定国家/地区的访问。

解决方案

将您要阻止或允许的国家代码映射到变量:

```
load_module modules/nginx_http_geoip2_module.so;
http {
    geoip2 /etc/maxmind-country.mmdb {
        auto_reload 5m;
        $geoip2_metadata_country_build metadata build_epoch;
        $geoip2_data_country_code default=US
            source=$variable_with_ip country iso_code;
        $geoip2_data_country_name country names en;
    }
    map $geoip2_data_country_code $country_access {
        "US" 0;
        default 1;
    }
    # ...
}
```

此映射会将新变量 `$country_access` 设置为 1 或 0。如果客户端 IP 地址来自美国, 该变量将设置为 0; 如果客户端 IP 地址来自其他国家/地区, 该变量将设置为 1。

现在，在我们的 server 代码块中，我们将使用 if 语句来拒绝来自美国以外的任何人的访问：

```
server {
    if ($country_access = '1') {
        return 403;
    }
    # ...
}
```

如果 \$country_access 变量设置为 1，则这个 if 语句的计算结果为 True。如果为 True，服务器将返回 403 Unauthorized，否则服务器就正常运行。因此，这个 if 代码块只是用来拒绝非美国地区的用户访问。

详解

这个例子简明扼要地说明了如何仅允许几个国家/地区的访问，您可以根据自己的需求在此基础上进行延伸。您可以借鉴这一做法，根据 geoip2 模块提供的任何嵌入式变量允许或阻止访问。

3.4 查找原始客户端

问题

NGINX 服务器前面设置了代理，您想要查找原始客户端 IP 地址。

解决方案

使用 geoip2_proxy 指令定义代理 IP 地址范围，并使用 geoip_proxy_recursive 指令查找原始 IP：

```
load_module "/usr/lib64/nginx/modules/nginx_http_geoip_module.so";

http {
    geoip2 /etc/maxmind-country.mmdb {
        auto_reload 5m;
        $geoip2_metadata_country_build metadata build_epoch;
        $geoip2_data_country_code default=US
            source=$variable_with_ip country iso_code;
        $geoip2_data_country_name country names en;
    }
    geoip2_proxy 10.0.16.0/26;
    geoip2_proxy_recursive on;
    # ...
}
```

geoip2_proxy 指令定义了一个**无类域间路由**（CIDR）范围，我们的代理服务器就在这个范围内，并指示 NGINX 利用 X-Forwarded-For 请求头查找客户端 IP 地址。geoip2_proxy_recursive 指令指示 NGINX 递归性地从 X-Forwarded-For 请求头中查找已知的最后一个客户端 IP。



Forwarded 请求头已经成为为被代理请求添加代理信息的标准请求头。NGINX GeoIP2 模块使用的请求头是 X-Forwarded-For，否则就无法在写入时进行配置。虽然 X-Forwarded-For 不是官方标准，但仍然是被大多数代理广为使用、接受和设置的请求头。

详解

您可能会发现，如果您在 NGINX 前面使用代理，NGINX 将获取代理（而非客户端）的 IP 地址。在这种情况下，当在给定范围内打开连接时，您可以使用 geoip2_proxy 指令指示 NGINX 使用 X-Forwarded-For 请求头。geoip2_proxy 指令使用一个地址或 CIDR 范围。当 NGINX 前面有多个传输流量的代理时，您可以使用 geoip2_proxy_recursive 指令递归性地从 X-Forwarded-For 地址中搜索和查找源客户端。当在 NGINX 前面使用 Amazon Web Services Elastic Load Balancing（AWS ELB）、Google Cloud Platform 负载均衡器或 Microsoft Azure 负载均衡器等负载均衡器时，这种方法比较有用。

3.5 限制连接数

问题

您想要根据预定义的键（例如客户端的 IP 地址）限制连接数。

解决方案

构造一个共享内存区来保存连接指标，并使用 limit_conn 限制打开的连接数：

```

http {
    limit_conn_zone $binary_remote_addr zone=limitbyaddr:10m;
    limit_conn_status 429;
    # ...
    server {
        # ...
        limit_conn limitbyaddr 40;
        # ...
    }
}

```

此配置创建了一个名为 `limitbyaddr` 的共享内存区。使用的预定义键是二进制形式的客户端 IP 地址。共享内存区的大小设置为 10 MB。`limit_conn` 指令使用两个参数：`limit_conn_zone` 名称和允许的连接数量。`limit_conn_status` 定义了连接状态被限制为 429 时的响应，此时表示响应过多。`limit_conn` 和 `limit_conn_status` 指令在 `http`、`server` 和 `location` 上下文中有效。

详解

通过使用键来限制连接数量，您不仅能够防止滥用，而且还能在所有客户端之间公平共享资源。请务必谨慎使用预定义键。正如我们在前面的示例中所示，如果许多用户位于源自同一 IP 地址的同一网络上，例如当在网络地址转换 (NAT) 后面时，使用 IP 地址是不合理的，这会使整个客户端组受到限制。`limit_conn_zone` 指令仅在 `http` 上下文中有效。您可以利用 NGINX 在 `http` 上下文中任意数量的可用变量来构建一个限制字符串。有一种更简洁的方法是，根据具体的用例利用一个变量（例如会话 cookie）识别应用层的用户。`limit_conn_status` 默认值为 503 时，代表服务不可用。由于服务可用，我们最好使用 429，而 500 级的响应码表示服务器错误，400 级的响应码表示客户端错误。

测试限制可能是个很棘手的问题，测试方案通常很难在替代环境中模拟实时流量。在这种情况下，您可以将 `limit_conn_dry_run` 指令设置为 `on`，然后使用访问日志中的变量 `$limit_conn_status`。`$limit_conn_status` 变量将计算为 `PASSED`、`DELAYED`、`REJECTED`、`DELAYED_DRY_RUN` 或 `REJECTED_DRY_RUN`。启用 `dry run` 后，您将能够分析实时流量日志，并根据需要调整限制以拒绝超限请求，从而确保您的限制配置正确。

3.6 限制速率

问题

您想要通过预定义的键（例如客户端的 IP 地址）来限制请求的速率。

解决方案

利用限速模块限制请求速率：

```
http {
    limit_req_zone $binary_remote_addr
        zone=limitbyaddr:10m rate=3r/s;
    limit_req_status 429;
    # ...
    server {
        # ...
        limit_req zone=limitbyaddr;
        # ...
    }
}
```

此示例配置创建了一个名为 `limitbyaddr` 的共享内存区。使用的预定义键是二进制形式的客户端 IP 地址。共享内存区的大小设置为 10 MB。该区域使用关键字参数设置速率。`limit_req` 指令使用了一个必不可少的关键字参数：`zone`。`zone` 指示了要使用哪个共享内存请求限制区的指令。根据 `limit_req_status` 指令的定义，超过明示速率的请求将返回 429 HTTP 代码。建议设置一个 400 级范围的状态码，因为默认值是 503，这代表服务器有问题，而实际问题是出在客户端方面。

使用 `limit_req` 指令的可选关键字参数来启用两级速率限制：

```
server {
    location / {
        limit_req zone=limitbyaddr burst=12 delay=9;
    }
}
```

在某些情况下，客户端需要同时发出许多请求，此后先在一段时间内降低速率，然后再发出更多请求。您可以使用关键字参数 `burst` 允许客户端超过其速率限制但不拒绝其请求。超出速率的请求将延迟处理，以将速率限制匹配到配置的值。有一组关键字参数可以改变这种行为，即 `delay` 和 `nodelay`。`nodelay` 参数不带值，只允许客户端一次性消

耗所有流量突发值；但是必须要先等待足够的时间，直到满足速率限制要求为止，否则所有请求都会被拒绝。在此示例中，如果我们使用 `nodeLAY`，客户端可以在第一秒消耗 12 个请求，但是必须要在初始请求之后等待 4 秒才能发出另一个请求。`delay` 关键字参数定义了在不限流的情况下可以预先发出多少请求。在这种情况下，客户端可以毫无延迟地预先发出 9 个请求，接下来的 3 个将受到限制，4 秒后的任何请求都将被拒绝。

详解

限速模块非常强大，可以防止滥用快速请求，同时仍然为每个人提供优质服务。限制请求速率的原因有很多，安全性就是其中之一。您可以通过严格限制登录页面的速率来防御暴力破解攻击。您可以对所有请求设置合理的限制，从而防止恶意用户试图对您的应用拒绝服务或浪费资源的不轨行为。

限速模块的配置很像[实操指南 3.5](#)中描述的连接限制模块，并且存在许多相同的问题。您可以按照每秒限速，也可以按照每分钟限速。当达到速率限制时，日志就会记录事件。此外，还有一条指令没有在示例中给出：`limit_req_log_level`，它的默认值为 `error`，您也可以把它设置为 `info`、`notice` 或 `warn`。在 NGINX Plus 中，速率限制目前具有集群感知能力（请参见[实操指南 12.5](#)中的“区域同步”示例）。

测试限制可能是个很棘手的问题，测试方案通常很难在替代环境中模拟实时流量。在这种情况下，您可以将 `limit_conn_dry_run` 指令设置为 `on`，然后使用访问日志中的变量 `$limit_conn_status`。`$limit_conn_status` 变量将计算为 `PASSED`、`REJECTED` 或 `REJECTED_DRY_RUN`。启用 `dry run` 后，您将能够分析实时流量日志，并根据需要调整限制以拒绝超限请求，从而确保您的限制配置正确。

3.7 限制带宽

问题

您想要按客户端限制资产的下载带宽。

解决方案

使用 NGINX 的 `limit_rate` 和 `limit_rate_after` 指令限制响应客户端的带宽：

```
location /download/ {  
    limit_rate_after 10m;  
    limit_rate 1m;  
}
```

这个 `location` 代码块的配置指定，对于前缀为 `download` 的 URI，向客户端提供响应的速率将在 10 MB 之后被限制为每秒 1 MB。带宽限制是针对每个连接的，因此您可能希望在适用的情况下配合使用连接限制和带宽限制。

详解

通过限制特定连接的带宽，NGINX 能够以您指定的方式在所有客户端上共享其上传带宽，并且只需 `limit_rate_after` 和 `limit_rate` 这两个指令就可以做到这一点。`limit_rate_after` 指令几乎可以在任何上下文中进行设置，包括 `http`、`server`、`location` 以及 `location` 代码块内的 `if`。`limit_rate` 指令的适用上下文与 `limit_rate_after` 相同，但是它还可以通过一个名为 `$limit_rate` 的变量进行设置。

`limit_rate_after` 指令规定，在传输指定数量的数据之前不得限制连接的速率。`limit_rate` 指令指定了给定上下文中的速率限制，默认单位是每秒字节数，但是您也可以设置为 `m`（兆字节）或 `g`（千兆字节）。这两条指令的默认值都是 0，表示不对下载速率进行任何限制。此模块允许您以编程方式更改客户端的速率限制。

大规模可扩展的内容缓存

4.0 简介

缓存能够存储响应结果，以供未来再次使用，进而加速内容的提供。通过从缓存中提供内容，NGINX 可卸载上游（upstream）服务器的高开销、重复性工作，从而减少其负载。缓存可以提高性能并减少负载，这意味着您可以用更少的资源更快地提供服务。缓存还能减少提供资源所需的时间和带宽。

缓存服务器在战略位置上的扩展和分布会对用户体验产生巨大影响。最好将内容托管在靠近消费者的地方，这有助于释放最高性能。您还可以在靠近用户的地方缓存内容，内容交付网络（CDN）就采用了这种模式。NGINX 允许您在放置 NGINX 服务器的任何地方缓存内容，从而有效创建自己的 CDN。借助 NGINX 缓存，您还可以在上游（upstream）发生故障时被动地缓存并提供缓存的响应。缓存功能仅在 http 上下文中可用。本章将介绍 NGINX 的缓存和内容交付功能。

4.1 缓存区

问题

您想要缓存内容并定义缓存的存储位置。

解决方案

使用 `proxy_cache_path` 指令定义共享内存缓存区和内容的位置：

```
proxy_cache_path /var/nginx/cache
                  keys_zone=main_content:60m
                  levels=1:2
                  inactive=3h
                  max_size=20g
                  min_free=500m;
proxy_cache CACHE;
```

上述缓存定义示例在文件系统 `/var/nginx/cache` 中为缓存响应创建了一个目录，以及一个名为 `main_content`、大小为 60M 的共享内存空间。此示例设置了目录结构级别，定义了如果缓存响应在 3 小时内未被请求就会被清除，同时定义了最大缓存大小为 20GB。`min_free` 参数告知 NGINX 在清除缓存资源之前应在 `max_size` 的基础上保留多少磁盘空间。`proxy_cache` 指令通知特定上下文使用缓存区。`proxy_cache_path` 在 `http` 上下文中有效，`proxy_cache` 指令在 `http`、`server` 和 `location` 上下文中有效。

详解

要在 NGINX 中配置缓存，就必须声明要使用的路径和区域。您可以使用 `proxy_cache_path` 指令创建 NGINX 中的缓存区。`proxy_cache_path` 指令指定了存储缓存信息的位置，以及存储活动键和响应元数据的共享内存空间。该指令的可选参数对维护和访问缓存的方式提供了更多控制。

`levels` 参数定义了目录结构的创建方式。该值是一个以冒号分隔的数表，定义了连续子目录名称的长度。更深层次的结构有助于避免单个目录中出现过多缓存文件。它将结果存储在提供的文件结构中，使用缓存键作为文件路径并根据 `levels` 值分解目录。

`inactive` 参数能够控制缓存项在最后一次使用后驻留的时间长度。您也可以使用 `max_size` 参数配置缓存的大小。其他参数与缓存加载进程有关，这个进程可以将缓存键从磁盘的缓存文件及许多其他选项加载到共享内存区。有关 `proxy_cache_path` 指令的更多信息，请访问下面“其他参考资料”部分中的文档链接。

其他参考资料

[proxy_cache_path 文档](#)

4.2 缓存哈希键

问题

您想要控制内容的缓存和检索方式。

解决方案

使用 `proxy_cache_key` 指令和变量定义缓存的命中或未命中：

```
proxy_cache_key "$host$request_uri $cookie_user";
```

上述缓存哈希键将根据被请求的主机和 URI、定义用户的 cookie 指示 NGINX 缓存页面。这样您就可以缓存动态页面，而不用提供为其他用户生成的内容。

详解

`proxy_cache_key` 的默认配置是 `"$scheme$proxy_host$request_uri"`，该配置适用于大多数用例。使用的变量包括 `scheme` (`http` 或 `https`)、`proxy_host` (发送请求) 以及请求 URI。总之，这反映了 NGINX 将请求代理到哪个 URL。您可能会发现，还有许多其他因素定义了每个应用的唯一请求，比如请求参数、请求头、会话标识符等等，您希望为这些因素创建自己的哈希键。¹

选择一个好的哈希键非常重要，其中离不开对应用的了解。为静态内容选择缓存键通常非常简单，只需使用主机名和 URI 便可。但如果是为相当动态的内容（例如仪表盘应用的页面）选择缓存键，就要求对用户与应用的交互方式以及用户体验之间的差异程度有着更深刻的认识。缓存动态内容时，使用 cookie 或 JWT token 等会话标识符特别有用。出于安全考虑，您可能不希望在没有完全了解上下文的情况下就将用户的缓存数据展示给其他用户。`proxy_cache_key` 指令将字符串配置为缓存键的哈希值，该指令可以在 `http`、`server` 和 `location` 代码块中进行设置，从而灵活控制请求的缓存方式。

¹ 暴露给 NGINX 的文本或变量的任何组合都可以用来形成一个缓存键。NGINX 提供了这些变量的列表。

4.3 缓存锁定

问题

您希望控制 NGINX 如何处理对正在更新缓存的资源的并发请求。

解决方案

使用 `proxy_cache_lock` 指令确保一次只能将一个请求写入缓存，随后的请求将等待响应被写入缓存并由此提供内容：

```
proxy_cache_lock on;  
proxy_cache_lock_age 10s;  
proxy_cache_lock_timeout 3s;
```

详解

我们不想在代理请求时，之前对相同内容的请求仍在传输中，并且正在写入缓存。`proxy_cache_lock` 指令指示 NGINX 保存当前正在填充的最终将成为缓存资源的请求。根据 `proxy_cache_lock_age` 指令的定义，代理请求填充缓存的时间是有限制的，默认为 5 秒，此后才允许另一个请求填充资源。NGINX 还允许将已经等待指定时间 `proxy_cache_lock_timeout`（同样默认为 5 秒）的请求传递到代理服务器，该请求不会尝试来填充缓存。您可以将 `proxy_cache_lock_age` 理解成“你花的时间太长了，我来为你填充缓存”，将 `proxy_cache_lock_timeout` 理解成“你让我等的的时间太长了，你先慢慢填充，我去干点别的”。

4.4 使用陈旧缓存

问题

您想要在上游（upstream）服务器不可用时发送过期缓存条目。

解决方案

使用 `proxy_cache_use_stale` 指令和参数值来定义 NGINX 应使用陈旧缓存的场景：

```
proxy_cache_use_stale error timeout invalid_header updating  
http_500 http_502 http_503 http_504  
http_403 http_404 http_429;
```

详解

NGINX 能够在应用无法正确返回时提供陈旧缓存资源，这确实可以让问题迎刃而解。但这一功能会给最终用户造成 Web 服务正在运行的假象，而实际上后端可能已经无法访问。当出现问题时，从陈旧缓存提供内容还能减轻后端服务器的压力，从而让工程团队有些许时间调试解决问题。

当上游请求超时、返回 `invalid_header` 错误或返回 400 和 500 级响应码时，该配置会指示 NGINX 使用陈旧缓存资源。`error` 和 `updating` 参数有点特别。当无法选择上游服务器时，`error` 参数允许使用陈旧缓存。`updating` 参数会指示 NGINX 在缓存更新内容时使用陈旧缓存资源。

其他参考资料

[NGINX 使用陈旧缓存指令文档](#)

4.5 绕过缓存

问题

有时您需要绕过缓存。

解决方案

使用具有非空或非零值的 `proxy_cache_bypass` 指令。动态执行此操作的一种方法是，在您不希望进行缓存的 `location` 代码块内，使用一组字符串不能为空或为 0 的变量：

```
proxy_cache_bypass $http_cache_bypass;
```

该配置指示 NGINX，如果名为 `cache_bypass` 的 HTTP 请求头的值不是 0，则绕过缓存。本例使用请求头作为变量来确定是否应该绕过缓存 —— 客户端需要专门为其请求设置此请求头。

详解

我们有很多使用场景都不要缓存请求。为此，NGINX 提供了 `proxy_cache_bypass` 指令。这样，当值设置为非空或非零时，请求将被发送到上游（upstream）服务器，而不是从缓存中拉取。绕过缓存的不同需求和场景将由您的应用的用例来决定。绕过缓存可以简单到只需使用请求头或响应头就能搞定，也会复杂到协同配合多个 `map` 代码块来解决。

绕过缓存的原因有很多，比如为了故障排除或调试。如果您总是拉取缓存的页面，或者您的缓存键只是针对一个用户标识符的，那么复盘问题可能会很难。因此，具备绕过缓存的能力十分重要。绕过缓存的方法有很多，包括但不限于设置特定的 cookie、请求头或请求参数时等。您还可以通过设置 `proxy_cache off;`，彻底关闭给定上下文（例如 `location` 代码块）的缓存功能。

4.6 NGINX Plus 之缓存清除

问题

您想要使缓存中的对象变得无效。

解决方案

使用 `proxy_cache_purge` 指令（NGINX Plus 的清除功能）以及非空或零值变量：

```
map $request_method $purge_method {
    PURGE 1;
    default 0;
}
server {
    # ...
    location / {
        # ...
        proxy_cache_purge $purge_method;
    }
}
```

在此示例中，如果使用 `PURGE` 方法请求特定对象，那么它的缓存将被清除。以下是使用 `curl` 清除名为 `main.js` 的文件缓存的示例：

```
$ curl -X PURGE http://www.example.com/main.js
```

详解

处理静态文件的一种常见方法是将文件的哈希放在文件名中。当您推出新的代码和内容时，这可以确保您的 CDN 将其识别为新文件（因为 URI 已更改）。但是，对设置了不适合此模型的缓存键的动态内容来说，这种方法并不好用。在每种缓存场景中，您都必须有一种对应的清除缓存的方法。

NGINX Plus 的处理方法很简单，只要将 `proxy_cache_purge` 指令的值设置为零或非空，与请求匹配的缓存项就会被清除掉。此外还有一种简便方法，那就是为 PURGE 映射请求方法。但是您可能需要配合使用 `geoip` 模块或简单的身份验证方法，以确保不是任何人都能清除您宝贵的缓存项。NGINX 还允许使用 `*` 来清除与常见 URI 前缀相匹配的缓存项。要使用通配符，您需要使用 `purger=on` 参数配置 `proxy_cache_path` 指令。

其他参考资料

NGINX 缓存清除示例

4.7 缓存切片

问题

您想要通过将资源分段来提高缓存效率。

解决方案

使用 NGINX `slice` 指令及其嵌入式变量将缓存结果分段：

```
proxy_cache_path /tmp/mycache keys_zone=mycache:10m;
server {
    # ...
    proxy_cache mycache;
    slice 1m;
    proxy_cache_key $host$uri$is_args$args$slice_range;
    proxy_set_header Range $slice_range;
    proxy_http_version 1.1;
    proxy_cache_valid 200 206 1h;

    location / {
        proxy_pass http://origin:80;
    }
}
```

详解

此配置为服务器定义并启用了缓存区。slice 指令指示 NGINX 将响应分成 1MB 的文件段。缓存资源根据 proxy_cache_key 指令进行存储。请注意，该指令使用了一个名为 slice_range 的嵌入式变量。向源端发送请求时也使用了同一变量作为请求头，并且该请求的 HTTP 版本已升级到 HTTP/1.1，因为 1.0 不支持字节范围的请求。200 或 206 响应码的缓存有效期设为 1 个小时，然后定义位置和源端。

Cache Slice 模块是为了交付 HTML5 视频开发的，它使用字节范围的请求将伪流内容传输到浏览器。默认情况下，NGINX 能够从缓存中提供字节范围的请求。如果对未缓存的内容发出了一个字节范围的请求，NGINX 将向源端请求整个文件。当您使用 Cache Slice 模块时，NGINX 仅向源端请求必要的文件段。如果请求范围大于切片大小（包括整个文件），就会触发对每个所需文件段的子请求，并对这些文件段进行缓存处理。当所有文件段都被缓存后，系统将组合响应并将它们发送给客户端，这使得 NGINX 能够更有效地缓存并提供请求的范围内的内容。

Cache Slice 模块只能用于不会进行更改的大型资源。NGINX 每接收一个来自源端的文件段，就验证一次 ETag。如果源端的 ETag 发生了变化，那么由于缓存不再有效，NGINX 就会中止这个段的缓存填充。如果内容确实发生了变化，文件也变小了，或者如果源端可以处理缓存填充过程中的负载峰值，那么最好使用[实操指南 4.3](#)中所述的 cache_lock 指令。请参阅“其他参考资料”部分中列出的博文，了解有关缓存切片技术的更多信息。

其他参考资料

[“借助 NGINX 和 NGINX Plus 实现智能高效的字节范围缓存”](#)

可编程性和自动化

5.0 简介

可编程性是指通过编程进行交互的能力。NGINX Plus 的 API 可以帮助实现这一点，即通过 HTTP 接口与 NGINX Plus 的配置和行为进行交互。该 API 支持通过 HTTP 请求添加或删除上游（upstream）服务器，从而实现对 NGINX Plus 的重新配置。NGINX Plus 中的键值（key-value）存储功能支持其他级别的动态配置——您可以利用 HTTP 调用注入 NGINX Plus 动态路由或控制流量所需的信息。本章将探讨 NGINX Plus API 以及通过该 API 暴露的键值存储模块。

配置管理工具能够实现服务器的自动化安装和配置，这在云时代是非常宝贵的实用程序。通过使用这些工具，大型 Web 应用的工程师不必再手动配置服务器，只需一次性写好配置和代码，就能够以可重复、可测试和模块化的方式构建出许多具有相同配置的服务器。本章介绍了当前市面上最受欢迎的一些配置管理工具，并讨论了如何使用它们安装 NGINX 以及如何将基本配置模板化。其中有一些很基础的例子，但却很好地演示了如何在每个平台上启用 NGINX 服务器。

5.1 NGINX Plus API

问题

您有一个动态的环境，想要随时重新配置 NGINX Plus。

解决方案

配置 NGINX Plus API，以便通过 API 调用启用和删除服务器：

```
upstream backend {
    zone http_backend 64k;
}
server {
    # ...
    # 启用 /api/ location 并进行适当的访问控制
    # 以便充分利用 NGINX Plus API
    location /api {
        # 建议将 write=off 设置为只读模式
        api write=on;
        # 限制访问 API 的指令
        # 见第 7 章
    }

    # 启用 NGINX Plus 仪表盘；需要启用 /api/ location
    # 并对远程访问进行适当的访问控制
    location = /dashboard.html {
        root /usr/share/nginx/html;
    }
}
```

NGINX Plus 的这一配置创建了一个带有共享内存区的上游（upstream）服务器，允许启用 /api location 代码块中的 API，并为 NGINX Plus 仪表盘提供了一个 location。

您可以使用 API 添加上线的服务器：

```
$ curl -X POST -d '{"server":"172.17.0.3"}' \
'http://nginx.local/api/9/http/upstreams/backend/servers/'
{
  "id":0,
  "server":"172.17.0.3:80",
  "weight":1,
  "max_conns":0,
  "max_fails":1,
  "fail_timeout":"10s",
  "slow_start":"0s",
  "route":"",
  "backup":false,
  "down":false
}
```

此示例中的 curl 调用请求 NGINX Plus 将新服务器添加到后端上游配置中。HTTP 采用了 POST 方法，将 JSON 对象作为正文传输，并返回 JSON 响应。JSON 响应显示了服务器对象配置——请注意，它生成了一个新的 id，其他配置均设置为默认值。

NGINX Plus API 是 RESTful，因此请求 URI 中带有参数，URI 的格式如下：

```
/api/{version}/http/upstreams/{httpUpstreamName}/servers/
```

您可以利用 NGINX Plus API 列出上游池中的服务器：

```
$ curl 'http://nginx.local/api/9/http/upstreams/backend/servers/'
[
  {
    "id":0,
    "server":"172.17.0.3:80",
    "weight":1,
    "max_conns":0,
    "max_fails":1,
    "fail_timeout":"10s",
    "slow_start":"0s",
    "route":"",
    "backup":false,
    "down":false
  }
]
```

此示例中的 curl 调用请求 NGINX Plus 列出 backend 上游池中的所有服务器。目前我们只有一台在以前的 API curl 调用中添加的服务器。该请求将返回一个上游服务器对象，它包含服务器所有可配置的选项。

使用 NGINX Plus API 清空上游服务器的连接，为将它从上游池中删除做好准备。有关连接清空的信息，请参见[实操指南 2.8](#)；清空连接的 API 请求如下所示：

```
$ curl -X PATCH -d '{"drain":true}' \
'http://nginx.local/api/9/http/upstreams/backend/servers/0'
{
  "id":0,
  "server":"172.17.0.3:80",
  "weight":1,
  "max_conns":0,
  "max_fails":1,
  "fail_timeout":"10s",
  "slow_start":"0s",
  "route":"",
  "backup":false,
  "down":false,
  "drain":true
}
```

这个 curl 调用指定了请求方法为 PATCH，传输 JSON 正文以指示清空服务器的连接，并指定将服务器 ID 附加到 URI。通过列出上一个 curl 命令指示添加的上游服务器，我们找到了服务器 ID。

NGINX Plus 将开始执行清空连接的进程，它用时比较长，相当于应用的会话长度。要检查正在实施连接清空的服务器正在提供多少个活动连接，请使用以下调用并查找相关服务器的 active 属性：

```
$ curl 'http://nginx.local/api/9/http/upstreams/backend'
{
  "zone" : "http_backend",
  "keepalive" : 0,
  "peers" : [
    {
      "backup" : false,
      "id" : 0,
      "unavail" : 0,
      "name" : "172.17.0.3",
      "requests" : 0,
      "received" : 0,
      "state" : "draining",
      "server" : "172.17.0.3:80",
      "active" : 0,
      "weight" : 1,
      "fails" : 0,
      "sent" : 0,
      "responses" : {
        "4xx" : 0,
        "total" : 0,
        "3xx" : 0,
        "5xx" : 0,
        "2xx" : 0,
        "1xx" : 0
      },
      "health_checks" : {
        "checks" : 0,
        "unhealthy" : 0,
        "fails" : 0
      },
      "downtime" : 0
    }
  ],
  "zombies" : 0
}
```

等所有连接都清空后，使用 NGINX Plus API 将服务器从上游池中彻底删除：

```
$ curl -X DELETE \
  'http://nginx.local/api/9/http/upstreams/backend/servers/0'
[]
```

curl 命令使用 DELETE 方法向更新服务器状态时使用的同一 URI 发出了请求。DELETE 方法指示 NGINX 删除服务器。此 API 调用返回仍留在池中的所有服务器及其 ID。由于

一开始是一个空池，而后只通过 API 添加了一台服务器，清空连接后又将其删除，现在我们还是只剩下一个空池。

详解

NGINX Plus 独有的 API 允许动态应用服务器随时自行添加到 NGINX 配置中或者从 NGINX 配置中删除。服务器上线后就会被注册到池中，NGINX 也将开始向新添加的服务器发送负载。当需要删除服务器时，服务器可以请求 NGINX Plus 清空其连接，然后在被关闭之前自行从上游池中删除。这样基础架构无需人工干预，便可以在一定程度上自动化地调整规模。

其他参考资料

[NGINX Plus REST API 文档](#)

5.2 使用 NGINX Plus 的键值 (Key-Value) 存储功能

问题

您想要 NGINX Plus 根据应用的输入做出动态的流量管理决策。

解决方案

本节将以动态拦截列表为例演示流量管理决策。

设置集群感知型键值存储和 API，然后添加键和值：

```
keyval_zone zone=blocklist:1M;
keyval $remote_addr $blocked zone=blocklist;

server {
    # ...
    location / {
        if ($blocked) {
            return 403 'Forbidden';
        }
        return 200 'OK';
    }
}
server {
```

```

# ...
# 限制访问 API 的指令
# 见第 6 章
location /api {
    api write=on;
}
}

```

NGINX Plus 的这一配置使用 `keyval_zone` 指令构建了一个名为 `blocklist` 的键值存储共享内存区，并将内存限制设置为 1 MB。然后，`keyval` 指令映射键的值，从而将第一个参数 `$remote_addr` 匹配到区中的新变量 `$blocked`。这一新变量用来确定 NGINX Plus 是应该服务请求还是返回 403 Forbidden 代码。

使用此配置启动 NGINX Plus 服务器后，您可以向本地计算机发送 `curl` 命令，服务器可能会返回 200 OK 响应：

```

$ curl 'http://127.0.0.1/'
OK

```

现在将本地机器的 IP 地址添加到值为 1 的键值存储中：

```

$ curl -X POST -d '{"127.0.0.1":"1"}' \
'http://127.0.0.1/api/9/http/keyvals/blocklist'

```

这个 `curl` 命令提交了带 JSON 对象的 HTTP POST 请求，其中包含一个待提交到 `blocklist` 共享内存区的键值对象。键值存储 API URI 的格式如下：

```

/api/{version}/http/keyvals/{httpKeyvalZoneName}

```

本地机器的 IP 地址现已添加到值为 1 的 `blocklist` 键值区中。在接下来的请求中，NGINX Plus 会在键值区中查询 `$remote_addr`，查找条目并将值映射到变量 `$blocked`。该变量将在 `if` 语句中进行计算。变量赋值后，`if` 计算结果为 `True`，NGINX Plus 返回 403 Forbidden 状态码：

```

$ curl 'http://127.0.0.1/'
Forbidden

```

您可以通过使用 `PATCH` 方法发送请求来更新或删除键：

```

$ curl -X PATCH -d '{"127.0.0.1":null}' \
'http://127.0.0.1/api/9/http/keyvals/blocklist'

```

如果值为 `null`，NGINX Plus 将删除键，并且请求将再次返回 200 OK。

详解

键值存储是 NGINX Plus 独有的功能，允许应用将信息注入到 NGINX Plus 中。上述示例中的 `$remote_addr` 变量用于创建动态拦截列表。您可以使用 NGINX Plus 可能用作变量的任何键（例如会话 cookie）填充键值存储，并为 NGINX Plus 提供一个外部值。在 NGINX Plus R16 中，键值存储具备集群感知能力，这意味着您只需为一台 NGINX Plus 服务器提供键值更新便可，这样所有服务器都会收到信息。

在 NGINX Plus R19 中，键值存储启用了 `type` 参数，支持为特定类型的键添加索引。默认情况下，键的类型值为 `string`，您也可以选择使用 `ip` 和 `prefix`。`string` 类型不构建索引，并且所有键的请求都必须是确切的匹配，而 `prefix` 允许部分键匹配（只要键的前缀匹配即可）。`ip` 类型允许使用 CIDR notation。在我们的示例中，如果已经指定 `type=ip` 为键值区的参数，那么我们也可以提供要拦截的整个 CIDR 范围，例如使用 `192.168.0.0/16` 拦截 RFC 1918 私有网络地址范围，或者使用 `127.0.0.1/32` 拦截本地主机，这种方法的效果与示例中呈现的一样。

其他参考资料

[“使用 NGINX Plus 键值存储实现动态带宽限制”](#)

5.3 在 NGINX 中使用 njs 模块暴露 JavaScript 功能

问题

您想要 NGINX 执行对请求或响应的自定义逻辑。

解决方案

通过为 NGINX 安装 NGINX JavaScript (njs) 模块启用 JavaScript。以下软件包安装步骤假设您已经为您的 Linux 发行版添加了官方 NGINX 仓库（如第 1 章所示）。

使用 APT 软件包管理器的 NGINX 开源版：

```
$ apt install nginx-module-njs
```

使用 APT 软件包管理器的 NGINX Plus：

```
$ apt install nginx-plus-module-njs
```

使用 YUM 软件包管理器的 NGINX 开源版：

```
$ yum install nginx-module-njs
```

使用 YUM 软件包管理器的 NGINX Plus：

```
$ yum install nginx-plus-module-njs
```

如果您的 NGINX 配置中没有 JavaScript 文件目录，请创建一个：

```
$ mkdir -p /etc/nginx/njs
```

创建一个名为 `/etc/nginx/njs/jwt.js` 的 JavaScript 文件，且该文件应包含以下内容：

```
function jwt(data) {
  var parts = data.split('.').slice(0,2)
    .map(v=>Buffer.from(v, 'base64url').toString())
    .map(JSON.parse);
  return { headers:parts[0], payload: parts[1] };
}
function jwt_payload_subject(r) {
  return jwt(r.headersIn.Authorization.slice(7)).payload.sub;
}
function jwt_payload_issuer(r) {
  return jwt(r.headersIn.Authorization.slice(7)).payload.iss;
}
export default {jwt_payload_subject, jwt_payload_issuer}
```

提供的 JavaScript 示例定义了一个解码 JSON Web Token (JWT) 的函数。此外它还定义了其他两个函数，它们使用 JWT 解码器返回 JWT 中的特定键。导出这些函数，以供 NGINX 使用。这些函数将返回 JWT 中的常见键：subject 和 issuer。代码中的 `.slice(7)` 部分负责删除 Authorization 请求头值的前 7 个字符。JWT 出现以后，类型值变成 Bearer。Bearer 是 6 个字符，我们还需要删除空格分隔符，这是我们分割前 7 个字符的原因。有一些身份验证服务未提供类型（比如 AWS Cognito），因为此类服务会更改分割数或完全删除它，以便 jwt 函数仅接收令牌值。

在 NGINX 核心配置中加载 njs 模块。在 http 代码块中导入并使用 JavaScript：

```
load_module /etc/nginx/modules/nginx_http_js_module.so;

http {
    js_path "/etc/nginx/njs/";
    js_import main from jwt.js;
    js_set $jwt_payload_subject main.jwt_payload_subject;
    js_set $jwt_payload_issuer main.jwt_payload_issuer;
    ...
}
```

提供的 NGINX 配置动态加载了 njs 模块，并导入了我们先前定义的 JavaScript 文件。使用 NGINX 指令将 NGINX 变量设置为 JavaScript 函数的返回值。

定义返回 JavaScript 设置的变量的 server：

```
server {
    listen 80 default_server;
    listen [::]:80 default_server;
    server_name _;
    location / {
        return 200 "$jwt_payload_subject $jwt_payload_issuer";
    }
}
```

提供的配置将返回客户端通过 Authorization 请求头提供的 subject 和 issuer 值。这些值由定义的 JavaScript 代码解码。

为了验证代码的有效性，应使用给定的 JWT 向服务器发送请求。以下是 JWT 的 JSON 格式，您可以用它来验证代码的有效性：

```
{
  "iss": "nginx",
  "sub": "alice",
  "foo": 123,
  "bar": "qq",
  "zyx": false
}
```

用给定的 JWT 向服务器发送请求，以验证 JavaScript 代码是否运行正常并返回了正确的值：

```
$ curl 'http://localhost/' -H \
  "Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1\
  NiIsImV4cCI6MTU0NDcyMzA4NX0.eyJpc3MiOiJuZ2lueCIsInN1YiI6Im\
  FsaWNIiwiZm9vIjoxMjMsImJhcnI6InF1Iiwie\
  nI4IjpmYXxzZX0.Kftl23Rvv9dIso1RuZ8uHaJ83BkMmMtTwch09rJtwgk"

alice nginx
```

详解

NGINX 的 njs 模块可以在处理请求和响应的过程中暴露 JavaScript 标准功能。该模块允许您将业务逻辑嵌入到代理层中。我们之所以选择 JavaScript，是因为它的应用非常广泛。

njs 模块支持在请求进入 NGINX 以及 NGINX 返回响应时注入逻辑。您可以通过解码 JWT 来验证和操作穿过代理的请求（如本节所示）。njs 模块还能够通过 JavaScript 逻辑传输响应数据流，进而操作来自上游（upstream）服务的响应。此外，njs 还可以赋予 stream 服务应用层感知能力；有关此模块的更多信息，请参阅“其他参考资料”部分。

其他参考资料

[njs 脚本语言文档](#)

[NGINX Plus njs 模块的安装](#)

5.4 使用通用编程语言扩展 NGINX

问题

您想要 NGINX 使用通用编程语言执行一些自定义扩展。

解决方案

在用 C 语言编写自定义 NGINX 模块之前，请先评估您的用例是否适合使用其他编程语言模块。C 编程语言是一种非常强大且高效的语言，但是也有许多其他语言模块能够支持所需的自定义。NGINX 引入了 NGINX JavaScript (njs)，只需启用一个模块就可以将 JavaScript 的强大功能集成到 NGINX 配置中。您也可以使用 Lua 和 Perl 模块。使用这些语言模块时，您要么需要导入一个包含代码的文件，要么直接在配置中定义代码块。

要使用 Lua，请安装 Lua 模块和以下 NGINX 配置，以定义一个内联的 Lua 脚本：

```
load_module modules/ndk_http_module.so;
load_module modules/ngx_http_lua_module.so;

events {}
```

```

http {
    server {
        listen 8080;
        location / {
            default_type text/html;
            content_by_lua_block {
                ngx.say("hello, world")
            }
        }
    }
}

```

Lua 模块通过 ngx 模块定义的对象提供自己的 NGINX API。与 njs 中的 request 对象一样，ngx 对象也有描述请求和操作响应的属性和方法。

安装了 Perl 模块后，此示例将使用 Perl 从运行时环境设置 NGINX 变量：

```

load_module modules/ngx_http_perl_module.so;

events {}

http {
    perl_set $app_endpoint 'sub { return $ENV{"APP_DNS_ENDPOINT"}; }';
    server {
        listen 8080;
        location / {
            proxy_pass http://$app_endpoint
        }
    }
}

```

上述示例表明，这些语言模块不仅仅是返回响应，而且还会暴露更多功能。perl_set 指令将一个 NGINX 变量设置为从 Perl 脚本返回的数据。这个示例比较有限，只是返回了用作请求代理端点的系统环境变量。

详解

NGINX 具备出色的可扩展性，这为启用更多功能开创了更多可能。NGINX 可通过使用 C 语言模块自定义代码来扩展，您可以从一开始构建时就把这些模块编译到 NGINX 中，也可以在配置中动态加载。目前还有一些模块可以暴露 JavaScript (njs)、Lua 和 Perl 的功能与语法。在多数情况下这些预先存在的模块是够用的——除非将 NGINX 的自定义功能分配出去。现在开源社区中有很多为这些模块构建的脚本。

该解决方案演示了 NGINX 中可用的 Lua 和 Perl 脚本语言的基本用法。无论是寻求响应、设置变量、发送子请求还是定义复杂的重写，这些 NGINX 模块都能提供相应的支持。

其他参考资料

[NGINX Plus Lua 模块安装](#)

[NGINX Plus Perl 模块安装](#)

[NGINX Lua 模块文档](#)

[NGINX Perl 模块文档](#)

5.5 使用 Ansible 安装

问题

您想要使用 Ansible 安装和配置 NGINX，以将 NGINX 配置作为代码管理，并与其的 Ansible 配置保持一致。

解决方案

从 Ansible Galaxy 安装 Ansible NGINX collection:

```
ansible-galaxy collection install nginxinc.nginx_core
```

创建一个 playbook，使用 collection 和 nginx role 安装 NGINX:

```
---
- hosts: all
  collections:
    - nginxinc.nginx_core
  tasks:
    - name: Install NGINX
      include_role:
        name: nginx
```

将一个 task 添加到 playbook，并使用内置 nginx_config role，同时根据自己的用例覆盖默认模板的变量:

```
- name: Configure NGINX
  ansible.builtin.include_role:
    name: nginx_config
  vars:
    nginx_config_http_template_enable: true
    nginx_config_http_template:
      - template_file: http/default.conf.j2
        deployment_location: /etc/nginx/conf.d/default.conf
    config:
      servers:
```

```

- core:
  listen:
    - port: 80
    server_name: localhost
  log:
    access:
      - path: /var/log/nginx/access.log
        format: main
  sub_filter:
    sub_filters:
      - string: server_hostname
        replacement: $hostname
    once: false
  locations:
    - location: /
      core:
        root: /usr/share/nginx/html
        index: index.html

nginx_config_html_demo_template_enable: true
nginx_config_html_demo_template:
  - template_file: www/index.html.j2
    deployment_location: /usr/share/nginx/html/index.html
    web_server_name: Ansible NGINX collection

```

详解

Ansible 使用 Python 编写而成，是一款应用广泛、功能强大的配置管理工具。task 的配置位于 YAML 中，您可以使用 Jinja2 模板语言将文件模板化。Ansible 提供了订阅模式的服务器：Ansible 自动化平台。但它通常是在本地机器中使用，以直接将服务器构建到客户端中，也可以独立使用。Ansible 会将 Secure Shell (SSH) 批量导入到服务器中并运行配置。与其他配置管理工具一样，Ansible 也有一个大型社区 —— Ansible Galaxy，里面有很多公开可用的 role，其中不乏一些复杂巧妙的 role，您可以将它们在自己的 playbook 中。

该解决方案使用了由 F5 公司维护的公共 role 集合来安装 NGINX 并生成样本配置。此示例中使用的配置生成了一个 NGINX demo HTML 文件模板，并把它放在了 `/usr/share/nginx/html/index.html` 中。此外它还将 NGINX 配置文件模板化，以便生成侦听 `localhost:80` 的 server 代码块，并配置一个 location 代码块来提供演示文件。示例中提供的 NGINX 配置模板非常全面，但是 `nginx_config` role 允许您提供自己的模板，以便在进行全面控制的同时利用预先构建和维护的 Ansible 配置。F5 公司还维护了一个 NGINX App Protect role，可用于安装并配置 NGINX App Protect WAF 和 DoS 模块。

其他参考资料

NGINX 提供的 Ansible Collection

Ansible 文档

5.6 使用 Chef 安装

问题

您想要使用 Chef 安装和配置 NGINX，以将 NGINX 配置作为代码管理，并与其余的 Chef 配置保持一致。

解决方案

从 Chef Supermarket 安装由 Sous Chefs 维护的 NGINX cookbook：

```
$ knife supermarket install nginx
```

此 cookbook 以资源为基础，这意味着它为您提供了可以供您在自己的 cookbook 中使用的 Chef 资源。为您的 NGINX 用例创建 cookbook。该 cookbook 将包含从 Supermarket 安装的 NGINX cookbook 依赖包。依赖包安装成功后，您可以使用提供的资源。为安装 NGINX 创建 recipe：

```
nginx_install 'nginx' do
  source 'repo'
end
```

将 source 设置为 repo 后，NGINX 将从 F5 公司维护的仓库进行安装，该仓库提供了最新的版本。

使用 recipe 中的 nginx_config 资源覆盖核心 NGINX 配置：

```
nginx_config 'nginx' do
  default_site_enabled true
  keepalive_timeout 65
  worker_processes 'auto'
  action :create
  notifies :reload, 'nginx_service[nginx]', :delayed
end
```

使用 recipe 中的 nginx_site 资源配置 NGINX server 代码块：

```
nginx_site 'test_site' do

  variables(
    'server' => {
      'listen' => [ '*:80' ],
```

```

      'server_name' => [ 'test.example.com' ],
      'access_log' => '/var/log/nginx/test_site.access.log',
      'locations' => {
        '/' => {
          'root' => '/var/www/nginx-default',
          'index' => 'index.html index.htm',
        },
      },
    }
  }
)

action :create
notifies :reload, 'nginx_service[nginx]', :delayed
end

```

详解

Chef 是一款使用 Ruby 编写的配置管理工具，它可以在客户端/服务器关系或独立配置中运行。Chef 拥有一个非常庞大的社区（称为 Supermarket），其中有许多公开可用的 cookbook。用户可以使用命令行实用程序 Knife 从 Supermarket 安装和维护公共 cookbook。Chef 的功能十分强悍，本 recipe 所展示的只是冰山一角。

Supermarket 中的公共 NGINX cookbook 非常灵活，提供了从软件包管理器或源代码轻松安装 NGINX 的选项，能够编译和安装多种不同的模块并将基础配置模板化。本节演示了如何从 Sous Chefs 维护的仓库安装 NGINX，并简单举例说明了如何配置 server 代码块以托管 HTML 文件。您可以在 nginx_site 资源中添加自己的模板，以进一步控制您的 NGINX 配置。

其他参考资料

[Chef 文档](#)

[Chef Supermarket 之 NGINX cookbook](#)

5.7 使用 Consul 模板自动进行配置

问题

您想要使用 Consul 自动化进行 NGINX 配置，以响应环境的变化。

解决方案

使用 `consul-template` 守护进程和模板文件将您选择的 NGINX 配置文件模板化：

```
upstream backend { {{range service "app.backend"}}  
    server {{.Address}};{{end}}  
}
```

此示例是一个 Consul 模板文件，能够帮助生成上游（`upstream`）配置代码块的模板。该模板将在 Consul 中标记为 `app.backend` 的节点循环，并为 Consul 中的每个节点生成一个带有其 IP 地址的服务器指令。

`consul-template` 守护进程通过命令行运行，它可以在每次修改配置文件的模板时重载 NGINX：

```
$ consul-template -consul-addr consul.example.internal -template \  
./upstream.template:/etc/nginx/conf.d/upstream.conf:"nginx -s reload"
```

此命令指示 `consul-template` 守护进程连接到 `consul.example.internal` 的 Consul 集群，使用当前工作目录中名为 `upstream.template` 的文件将文件模板化，同时将生成的内容输出到 `/etc/nginx/conf.d/upstream.conf`，然后在每次修改模板化文件时重载 NGINX。
`-template` 标记使用由模板文件、输出位置以及执行模板化流程后运行的命令组成的字符串，这三个变量用冒号隔开。如果正在运行的命令有空格，请用双引号把它括起来。
`-consul` 标记负责告知守护进程 Consul 集群要连接到什么。

详解

Consul 是一款功能强大的服务发现和配置存储工具，它通过一个类似目录的结构存储节点和键值（`key-value`）对信息，并支持 RESTful API 交互。Consul 还在每个客户端上提供了 DNS 接口，支持查找连接到集群的节点的域名。`consul-template` 守护进程也会用到 Consul 集群，该工具通过将文件模板化来响应 Consul 节点、服务或键值对的变化。这使得 Consul 成为一个非常强有力的 NGINX 自动化选择。`consul-template` 还可以指示守护进程在修改模板后运行命令。这样您就可以重新加载 NGINX 配置，并让您的 NGINX 配置与环境一起激活。借助 Consul 和 `consul-template`，您能够随着环境的变化动态调整 NGINX 配置。基础架构、配置和应用信息会进行集中存储，`consul-template` 能够在必要时根据事件进行订阅和重新模板化。有了这项技术，NGINX 就可以实现动态重新配置，从而应对服务器、服务和应用版本等项目的添加和删除。

其他参考资料

[使用 NGINX Plus 的服务发现集成功能实现负载均衡](#)

[使用 NGINX 和 Consul 模板实现负载均衡](#)

[Consul](#)

[使用 Consul 模板进行服务配置](#)

[consul-template GitHub](#)

6.0 简介

NGINX 能够对客户端进行身份验证。使用 NGINX 对客户端请求进行身份验证可以减轻服务器的工作负载，并能够阻止未经身份验证的请求到达应用服务器。NGINX 开源版模块包括基本身份验证和身份验证子请求。NGINX Plus 专有的 JSON Web Token (JWT) 验证模块可与使用身份验证标准 OpenID Connect 的第三方身份验证提供商集成。本章介绍了结合使用 NGINX 与身份验证来保护资源的多种方法。

6.1 HTTP 基本身份验证

问题

您想要通过 HTTP 基本身份验证保护应用或内容。

解决方案

生成以下格式的文件，其中的密码使用某个受支持的格式进行了加密或哈希处理：

```
# comment
name1:password1
name2:password2:comment
name3:password3
```

第一个字段是用户名，第二个字段是密码，冒号是分隔符。第三个字段为可选项，您可以使用该字段对每个用户进行评论。NGINX 能理解几种不同的密码格式，其中一个是

使用 C 函数 `crypt()` 加密的密码。该函数通过 `openssl passwd` 命令暴露在命令行中。安装 `openssl` 后，您可以使用以下命令创建加密的密码字符串：

```
$ openssl passwd MyPassword1234
```

输出结果将是一个字符串，可供 NGINX 在密码文件中使用。

在 NGINX 配置中使用 `auth_basic` 和 `auth_basic_user_file` 指令，实现基本身份验证：

```
location / {  
    auth_basic "Private site";  
    auth_basic_user_file conf.d/passwd;  
}
```

您可以在 `http`、`server` 或 `location` 上下文中使用 `auth_basic` 指令。`auth_basic` 指令带一个字符串参数，如有未经授权的用户访问，该参数将显示在基本身份验证弹窗中。`auth_basic_user_file` 指定了用户文件的路径。

如要测试配置，您可以使用带 `-u` 或 `--user` 标志的 `curl` 来构建请求的 `Authorization` 请求头：

```
$ curl --user myuser:MyPassword1234 https://localhost
```

详解

您可以通过几种方式生成具有不同格式和安全等级的基本身份验证密码。Apache 的 `htpasswd` 命令也可以生成密码。`openssl` 和 `htpasswd` 都可以使用 `apr1` 算法生成密码，并且 NGINX 也能理解这种格式的密码。该密码也可以是轻型目录访问协议 (LDAP) 和 Dovecot 使用的 Salted SHA-1 格式。NGINX 支持其他更多格式和哈希算法；然而，许多算法容易遭到暴力破解攻击，因此人们认为这些算法不安全。

您可以使用基本身份验证来保护整个 NGINX 主机的上下文、特定的虚拟服务器甚至只是特定的 `location` 代码块。基本身份验证不会取代 Web 应用的用户身份验证，但可以保护私人信息的安全。实际上，基本身份验证是由服务器返回 401 Unauthorized HTTP 代码和响应头 `WWW-Authenticate` 完成的。该响应头的值为 `Basic realm="your string"`。收到此响应后，浏览器将提示输入用户名和密码。用户名和密码用冒号连接和分隔，然后用 `base64` 编码，最后在名为 `Authorization` 的请求头中发送。`Authorization` 请求头将指定一个 `Basic` 和 `user:password` 编码字符串。服务器对请求头进行解码，并根据提供的 `auth_basic_user_file` 进行验证。由于用户名和密码字符串

仅通过 base64 编码，我们建议在基本身份验证中使用 HTTPS，否则用户凭证将以明文形式通过互联网发送。

6.2 身份验证子请求

问题

您希望通过第三方身份验证系对请求进行身份验证。

解决方案

使用 `http_auth_request_module` 请求访问身份验证服务，在响应请求之前验证身份：

```
location /private/ {
    auth_request /auth;
    auth_request_set $auth_status $upstream_status;
}

location = /auth {
    internal;
    proxy_pass http://auth-server;
    proxy_pass_request_body off;
    proxy_set_header Content-Length "";
    proxy_set_header X-Original-URI $request_uri;
}
```

`auth_request` 指令带一个 URI 参数，该参数必须是本地内部位置。`auth_request_set` 指令允许您设置身份验证子请求的变量。

详解

`http_auth_request_module` 可验证 NGINX 服务器处理的每个请求的身份。该模块将使用子请求来确定是否授予请求访问权。子请求是指 NGINX 将请求传递给内部替代位置，并在将请求路由到目的地之前观察它的响应。`/auth` location 代码块将原始请求（包括正文和请求头）传递到身份验证服务器。子请求的 HTTP 状态码决定是否授予请求访问权。如果子请求返回 HTTP 200 状态码，则表示身份验证成功，请求已完成。如果子请求返回 HTTP 401 或 403，则原始请求也将返回 HTTP 401 或 403。

如果您的身份验证服务未请求请求正文，则可以使用 `proxy_pass_request_body` 指令删除请求正文，如上所示。这种做法可减少请求的大小和时间。由于响应正文被丢弃，`Content-Length` 请求头必须设置为空字符串。如果身份验证服务需要知道请求访问的 URI，则需要将该值放入身份验证服务检查和验证的自定义请求头中。如果您确实希望将子请求中的某些内容保留给身份验证服务（例如响应头或其他信息），您可以使用 `auth_request_set` 指令从响应数据中创建新的变量。

6.3 使用 NGINX Plus 验证 JWT

问题

您想要在使用 NGINX Plus 处理请求之前验证 JWT。

解决方案

使用 NGINX Plus 的 HTTP JWT 身份验证模块来验证令牌签名，并将 JWT 声明和请求头作为 NGINX 变量嵌入：

```
location /api/ {  
    auth_jwt "api";  
    auth_jwt_key_file conf/keys.json;  
}
```

此配置可以验证该位置的 JWT。auth_jwt 指令传递一个字符串，该字符串被用作身份验证领域。auth_jwt 配置带一个保存 JWT 的变量的可选令牌参数。默认情况下，根据 JWT 标准使用 Authentication 请求头。auth_jwt 指令还可用于从继承的配置中消除所需的 JWT 身份验证的影响。要关闭 (off) 身份验证，使用 auth_jwt 指令即可，无需传递任何参数。要取消继承的身份验证要求，只需将 off 关键字传递到 auth_jwt 指令。auth_jwt_key_file 带一个参数。该参数是采用标准 JSON Web Key (JWK) 格式的密钥文件的路径。

在 NGINX Plus R29 中添加了一个模块，当 JWS 与速率限制搭配使用时，可有效防范未签名 JWT 拒绝服务攻击。该模块名为 `nginx_http_internal_redirect_module`，可将速率限制检查重排在 JWT 检查之前，以加强对攻击者防御。请查看下面的示例及 `internal_redirect` 指令的用法：

```

limit_req_zone $jwt_claim_sub zone=jwt_sub:10m rate=1r/s;

server {
    location / {
        auth_jwt "realm";
        auth_jwt_key_file key.jwk;

        internal_redirect @rate_limited;
    }

    location @rate_limited {
        internal;

        limit_req zone=jwt_sub burst=10;
        proxy_pass http://backend;
    }
}

```

详解

NGINX Plus 支持验证 JSON Web 签名类型的令牌，而不是将整个令牌进行加密的 JSON Web 加密类型。NGINX Plus 支持验证使用 HS256、RS256 和 ES256 算法签名的签名。使用 NGINX Plus 验证令牌可以节省向身份验证服务发出子请求所需的时间和资源。NGINX Plus 可破解 JWT 请求头和有效载荷，并将标准请求头和声明捕获到嵌入式变量中，供您使用。auth_jwt 指令可在 http、server、location 及 limit_except 上下文中使用。本节还演示了如何使用 NGINX Plus 的 internal_redirect 模块来防范未签名 JWT 拒绝服务攻击；有关该模块的更多信息，请参阅“其他参考资料”部分。

其他参考资料

[RFC JSON Web Signature 标准文档](#)

[RFC JSON Web 算法标准文档](#)

[RFC JSON Web Token 标准文档](#)

[NGINX Plus JWT 身份验证](#)

[“借助 JWT 和 NGINX Plus 验证 API 客户端身份”](#)

[NGINX internal_redirect 模块](#)

6.4 创建 JSON Web Key

问题

您想要使用 JSON Web Key (JWK) 才能使用 NGINX Plus。

解决方案

NGINX Plus 使用 RFC 标准中指定的 JWK 格式。该标准允许在 JWK 文件中包含一个关键对象数组。

以下是该密钥文件的示例：

```
{  
  "keys":  
  [  
    {  
      "kty": "oct",  
      "kid": "0001",  
      "k": "OctetSequenceKeyValue"  
    },  
    {  
      "kty": "EC",  
      "kid": "0002",  
      "crv": "P-256",  
      "x": "XCoordinateValue",  
      "y": "YCoordinateValue",  
      "d": "PrivateExponent",  
      "use": "sig"  
    },  
    {  
      "kty": "RSA",  
      "kid": "0003",  
      "n": "Modulus",  
      "e": "Exponent",  
      "d": "PrivateExponent"  
    }  
  ]  
}
```

所示的 JWK 文件演示了 RFC 标准中指出的三类初始密钥。这些密钥的格式也是 RFC 标准的一部分。kty 属性是密钥类型。该文件显示了三种密钥类型：Octet Sequence (oct)、Elliptic Curve (EC) 和 RSA 类型。kid 属性是密钥 ID。这些密钥的标准中指定了其他属性。更多信息请查看这些标准的 RFC 文档。

详解

有许多提供不同语言的库可以生成 JWK。建议创建一个密钥服务，作为 JWK 的中央权限，以定期创建和轮换您的 JWK。为了增强安全性，建议让 JWK 与 SSL/TLS 证书一样安全。使用适当的用户和组权限保护密钥文件。最好将它们保存在主机的内存中。您可以通过创建类似 ramfs 的内存文件系统来实现。定期轮换密钥也很重要；您可以选择创建一个密钥服务来创建公钥和私钥，并通过 API 将它们提供给应用和 NGINX。

其他参考资料

[RFC JSON Web Key 标准文档](#)

6.5 使用 NGINX Plus 通过现有的 OpenID Connect SSO 验证用户身份

问题

您希望将 NGINX Plus 与 OpenID Connect (OIDC) 身份验证供应商集成在一起。

解决方案

该解决方案由许多配置要素和一些 NGINX JavaScript (njs) 代码组成。身份验证供应商 (IdP) 必须支持 OpenID Connect 1.0。NGINX Plus 将在授权代码流中充当 OIDC 的中继方。

F5 公司维护了一个 GitHub 公共仓库，该仓库包含作为 OIDC 与 NGINX Plus 集成参考实现的配置和代码。“其他参考资料”部分中的仓库链接提供了关于如何使用您自己的 IdP 设置参考实现的最新说明。

详解

该解决方案只与参考实现有关，可确保各位读者拥有最新的解决方案。提供的参考将 NGINX Plus 配置为 OpenID Connect 1.0 授权代码流的中继方。在此配置中，如果将对受保护资源未经授权的请求发送到 NGINX Plus，NGINX Plus 首先会将该请求重定向到 IdP。IdP 会让客户端完成自己的登录流程，然后向 NGINX Plus 返回一个身份验证代码。然后，NGINX Plus 直接与 IdP 通信，用身份验证代码交换一组 ID 令牌。这些令牌使用 JWT 进行验证，并存储在 NGINX Plus 的键值 (key-value) 存储中。通过

使用键值存储，采用高可用性（HA）配置的所有 NGINX Plus 节点都能获得这组令牌。在这个过程中，NGINX Plus 为客户端生成了一个会话 cookie，该 cookie 被用于在键值存储中查找令牌的密钥。然后，客户端的 cookie 被重定向到初始请求的资源。随后的请求将通过使用 cookie 查找 NGINX Plus 键值存储中的 ID 令牌来进行验证。

该功能支持与大多数主要身份验证供应商进行集成，包括 CA 单点登录（以前称为 SiteMinder）、ForgeRock OpenAM、Keycloak、Okta、OneLogin 和 Ping Identity。作为一项标准，OIDC 与身份验证密切相关——前面提到的身份验证供应商只是可能集成的一个子集。

其他参考资料

[“借助 OpenID Connect 和 NGINX Plus 对应用的现存用户进行身份验证”](#)

[OpenID Connect](#)

[NGINX OpenID Connect GitHub](#)

[为 OIDC Connect 提供访问令牌支持](#)

6.6 使用 NGINX Plus 验证 JSON Web Token (JWT)

问题

您想要使用 NGINX Plus 验证 JSON Web Token。

解决方案

使用 NGINX Plus 自带的 JWT 模块保护位置或服务器，并指示 auth_jwt 指令使用 \$cookie_auth_token 作为要验证的令牌：

```
location /private/ {  
    auth_jwt "Google OAuth" token=$cookie_auth_token;  
    auth_jwt_key_file /etc/nginx/google_certs.jwk;  
}
```

此配置指示 NGINX Plus 通过 JWT 验证保护 /private/ URI 路径。Google OAuth 2.0 OpenID Connect 使用 cookie auth_token 而不是默认的 bearer 令牌。因此，您必须指示 NGINX 在此 cookie 中查找令牌，而不是在 NGINX Plus 的默认位置中查找。我们将在[实操指南 6.7](#)中介绍如何将 auth_jwt_key_file 位置设置为任意路径。

详解

此配置说明了如何使用 NGINX Plus 验证 Google OAuth 2.0 OpenID Connect JWT。NGINX Plus 的 HTTP JWT 身份验证模块支持验证符合 RFC JSON Web Signature 规范的 JWT，允许任何使用 JWT 的 SSO 授权立即在 NGINX Plus 层中进行验证。OpenID 1.0 协议层位于 OAuth 的上面。OpenID 2.0 身份验证协议添加了身份，支持使用 JWT 来证明发送请求的用户身份。NGINX Plus 可通过令牌的签名验证令牌自签名以来是否经过修改。这允许 Google 使用一种异步签名方法，可在保护其私人 JWK 的同时分发公共 JWK。

其他参考资料

“借助 JWT 和 NGINX Plus 验证 API 客户端身份”

6.7 使用 NGINX Plus 自动获取和缓存 JSON Web Key Set

问题

希望 NGINX Plus 自动请求提供商的 JSON Web Key Set (JWKS) 并进行缓存。

解决方案

利用缓存区和 `auth_jwt_key_request` 指令自动更新密钥：

```
proxy_cache_path /data/nginx/cache levels=1 keys_zone=foo:10m;

server {
    # ...

    location / {
        auth_jwt "closed site";
        auth_jwt_key_request /jwks_uri;
    }

    location = /jwks_uri {
        internal;
        proxy_cache foo;
        proxy_pass https://idp.example.com/keys;
    }
}
```

在此示例中，`auth_jwt_key_request` 指令指示 NGINX Plus 从内部子请求中检索 JWKS。该子请求被定向到 `/jwks_uri`，后者将请求代理给身份验证供应商。默认请求缓存时间为 10 分钟，以限制开销。

详解

NGINX Plus R17 中引入了 `auth_jwt_key_request` 指令。此功能支持 NGINX Plus 服务器在发出请求时动态更新其 JWKS。使用子请求方法来获取 JWKS，这意味着指令指向的位置必须是 NGINX Plus 服务器本地的位置。在上面的示例中，子请求的位置被锁定，以确保仅响应内部的 NGINX Plus 请求。还可使用缓存来确保仅在必要时发出 JWKS 检索请求，并且不会导致身份验证供应商过载。`auth_jwt_key_request` 指令在 `http`、`server`、`location` 和 `limit_except` 上下文中有效。

其他参考资料

[“借助 JWT 和 NGINX Plus 验证 API 客户端身份”](#)

[NGINX Plus “借助 JSON Web Key Set 缓存加快 JWT 验证”](#)

6.8 将 NGINX Plus 配置为 SAML 身份验证的服务提供商

问题

您希望将 NGINX Plus 与 SAML 身份验证供应商（IdP）相集成，以保护资源。

解决方案

此解决方案利用了 NGINX Plus 键值存储，因此仅适用于 NGINX Plus。此外，它还使用了 NGINX JavaScript (njs) 模块。首先安装 njs。

使用 APT 软件包管理器的 NGINX Plus：

```
$ apt install nginx-plus-module-njs
```

使用 YUM 软件包管理器的 NGINX Plus：

```
$ yum install nginx-plus-module-njs
```

上面的操作将 njs 安装为动态模块。您需要将以下内容添加到 *nginx.conf* 配置，从而指示 NGINX Plus 加载该模块：

```
load_module modules/nginx_http_js_module.so;
```

下载、解压并移动 SAML JavaScript 和 NGINX Plus 配置文件，以激活此功能：

```
$ wget https://github.com/nginxinc/nginx-saml/archive/refs/heads/main.zip \
    -O nginx-saml-main.zip
$ unzip nginx-saml-main.zip
$ mv nginx-saml-main/* /etc/nginx/conf.d/
```

NGINX Plus SAML 解决方案尚无法解析标准 SAML XML 配置文件。您需要更新在前面步骤中下载的配置文件，以便与您的系统相集成。以下是每个文件的描述以及所需更新内容的说明：

saml_sp_configuration.conf

在 `map{}` 代码块中包含一个或多个服务提供商（SP）和 IdP 的主要配置。可使用映射功能根据 `$host` 变量配置多个 SP 或 IdP。

- 修改所有提供带 `$saml_sp_` 前缀的变量的 `map` 代码块，以匹配 SP 配置。
- 修改所有提供带 `$saml_idp_` 前缀的变量的 `map` 代码块，以匹配 IdP 配置。
- 修改提供 `$saml_log_out_redirect` 变量的 `map` 代码块中定义的 URI，以指定在请求 `/logout` location 后显示的未受保护资源。
- 如果 NGINX Plus 部署在另一个代理或负载均衡器后面，请修改 `$redirect_base` 和 `$proto` 变量的映射，以定义获取原始协议和端口号的方式。

frontend.conf

使用 SAML 解决方案通过导入 *saml_sp.server_conf* 文件来保护资源的反向代理配置示例。

- 在服务器配置中复制 `include conf.d/saml_sp.server_conf;`。
- 当请求未经授权时，使用 `error_page` 指令启动 SAML SP 流。在需要保护的 `location` 代码块中添加以下内容：

```

error_page 401 = @do_samlsp_flow;
if ($saml_access_granted != "1") {
    return 401;
}

```

- 使用经过身份验证的用户名设置上游请求的请求头：

```
proxy_set_header username: $saml_name_id;
```

- 可选择通过更新传递给 `error_log` 指令的 `level` 参数来更新日志级别。

saml_sp.server_conf

用于处理 IdP 响应的 NGINX 配置。

- 该文件通常无需更改。
- 为了进行优化，修改 `client_body_buffer_size` 指令，以匹配最大 IdP 响应（post 正文）大小。

saml_sp.js

用于执行 SAML 身份验证的 JavaScript 代码。

- 无需更改。

详解

此解决方案结合使用了 NGINX JavaScript 模块与 NGINX Plus 键值存储模块，支持 NGINX 作为 SAML 服务提供商（SP）通过单点登录保护资源。F5 公司的 NGINX 团队在 GitHub 上以公共仓库的形式提供了 JavaScript 代码（需要将其安装到配置中并启用）。端点和密钥的配置及其他 SAML 配置由一系列 map 代码块提供给 JavaScript 代码，以便根据所处理请求的主机名配置多个 SP 或 IdP。

配置 SP 和 IdP 后，您可将 *saml_sp.server_conf* 添加到服务器配置中，并在 NGINX Plus 未为用户保存会话时使用 `error_page` 指令启动与 IdP 的 SP 流。SP 流会将用户定向到 IdP 以使用现有会话或登录，成功后，用户将被重定向到 NGINX Plus，并收到经过 NGINX Plus 验证并存储在键值存储中的 SAML 响应。SAML 响应的密钥将以 cookie 的形式返回给客户端，供后续请求使用，最后，客户端将被定向回最初请求的资源。

该解决方案还支持 SAML 单点注销 (SLO)，允许用户通过单个操作注销所有 SP 和 IdP。这一操作既可由 SP 发起，也可由 IdP 发起。当 SP 发起注销时，NGINX Plus 会向 IdP 发送 LogoutRequest 消息。然后，IdP 终止用户会话，并向 NGINX Plus 返回 LogoutResponse 消息，NGINX Plus 会删除存储在该用户会话键值存储中的值。

当 IdP 发起注销时，它负责以 SP 的身份联系 NGINX Plus，并通过向已注册的 Logout URL 发送 LogoutRequest 来启动注销流程。在此过程中，NGINX Plus 会从与用户会话相关的键值存储中删除值，并向 IdP 返回 LogoutResponse 消息。

如果 IdP 不支持 SLO 功能，或者您不希望 NGINX Plus SP 允许使用该功能，则可将其禁用。要禁用 SLO，请将 `$saml_idp_slo_url` 的变量映射配置设置为空字符串。

其他参考资料

NGINX Plus 支持 SAML SSO

7.0 简介

安全性必须要分层实施，因此您的安全模型必须要有多个层面才能真正得到强化。本章介绍了通过 NGINX 保护 Web 应用安全的许多不同方法。您可以将这些安全防护方法结合使用来增强安全性。

您可能会注意到，本章没有涉及将 NGINX 用作 Web 应用防火墙（WAF）的 ModSecurity 3.0 NGINX 模块。如欲了解 WAF 功能的更多信息，请参阅《[为 NGINX 开源版编译和安装 ModSecurity](#)》。

请注意，NGINX Plus 的 NGINX ModSecurity WAF 将于 2024 年 3 月 31 日起停用。更多详情，请参阅博文《[F5 NGINX ModSecurity WAF 即将转入生命周期终止阶段](#)》。

7.1 基于 IP 地址的访问

问题

您想要根据客户端的 IP 地址控制访问。

解决方案

使用 HTTP 或 stream 访问模块控制对受保护资源的访问：

```
location /admin/ {  
    deny 10.0.0.1;  
    allow 10.0.0.0/20;  
    allow 2001:0db8::/32;
```

```
    deny all;
}
```

给定的 location 代码块允许来自 10.0.0.0/20 中的任何 IPv4 地址访问（10.0.0.1 除外），允许来自 2001:0db8::/32 子网中的 IPv6 地址访问，并在收到来自其他任何地址的请求后返回 403。allow 和 deny 指令在 http、server 和 location 上下文以及 TCP/UDP 的 stream、server 上下文中有效。按顺序检查规则，直到找到与远程地址的匹配的规则为止。

详解

互联网上的宝贵资源和服务必须要进行多层保护，NGINX 就是其中一层的安全卫士。deny 指令可阻止对给定上下文的访问，而 allow 指令可用来允许访问被阻止地址的子集。您可以使用 IP 地址、IPv4 或 IPv6、无类别域间路由（CIDR）块范围，关键字 all 和 Unix 套接字。在保护资源时，通常会允许一个内部 IP 地址块，并拒绝所有访问请求。

7.2 允许跨域资源共享（CORS）

问题

您提供了来自另一个域的资源，需要允许跨域资源共享（CORS），以便浏览器能够利用这些资源。

解决方案

根据 request 方法更改请求头，启用 CORS：

```
map $request_method $cors_method {
    OPTIONS 11;
    GET 1;
    POST 1;
    default 0;
}
server {
    # ...
    location / {
        if ($cors_method ~ '1') {
            add_header 'Access-Control-Allow-Methods'
                'GET,POST,OPTIONS';
            add_header 'Access-Control-Allow-Origin'
                '*.example.com';
            add_header 'Access-Control-Allow-Headers'
                'DNT,
```

```

        Keep-Alive,
        User-Agent,
        X-Requested-With,
        If-Modified-Since,
        Cache-Control,
        Content-Type';
    }
    if ($cors_method = '11') {
        add_header 'Access-Control-Max-Age' 1728000;
        add_header 'Content-Type' 'text/plain; charset=UTF-8';
        add_header 'Content-Length' 0;
        return 204;
    }
}
}
}

```

这个示例中的内容很多，但通过使用 `map` 将 GET 和 POST 方法进行分组，简化了示例中的内容。OPTIONS 请求方法向客户端返回有关该服务器 CORS 规则的 *preflight* 请求。在 CORS 下允许使用 OPTIONS、GET 和 POST 方法。设置 `Access-Control-Allow-Origin` 请求头，允许该服务器提供的内容也可以在与此请求头匹配的原始页面上使用。preflight 请求可以在客户端缓存 1,728,000 秒，相当于 20 天。

详解

当 JavaScript 等资源所请求的资源属于其他域时，就会使用 CORS。当请求被视为跨域的时候，浏览器就要遵守 CORS 规则。如果浏览器没有专门允许其使用资源的请求头，那么它将不会使用该资源。为了允许其他子域使用我们的资源，我们必须设置 CORS 请求头，这可通过 `add_header` 指令完成。如果请求方法是具有标准内容类型的 GET、HEAD 或 POST，并且没有特殊的请求头，则浏览器将发出请求并且只检查来源。其他请求方法将导致浏览器发出 preflight 请求，以检查它将遵守的关于该资源的服务器条款。如果您没有正确设置这些请求头，那么浏览器会在尝试利用这些资源时出错。

7.3 客户端加密

问题

您希望在 NGINX 服务器和客户端之间加密流量。

解决方案

使用一个 SSL 模块加密流量，例如 `ngx_http_ssl_module` 或 `ngx_stream_ssl_module`：

```
http { # 下面使用的所有指令在 stream 中也有效
    server {
        listen 8443 ssl;
        ssl_certificate /etc/nginx/ssl/example.crt;
        ssl_certificate_key /etc/nginx/ssl/example.key;
    }
}
```

此配置将服务器设置为侦听使用 SSL/TLS 加密的端口 8443。指令 `ssl_certificate` 定义了提供给客户端的证书和可选的证书链。`ssl_certificate_key` 指令定义了 NGINX 用来解密请求和加密响应的密钥。许多 SSL/TLS 协商配置默认为 NGINX 版本的缺省配置。

详解

安全传输层是最常见的加密传输中信息的方法。截至本文撰写之时，TLS 协议比 SSL 协议更受欢迎。这是因为现在人们认为 SSL 版本 1 至 3 不安全。在 NGINX 1.23.4 版本及更高版本中，默认 SSL 协议为 TLSv1、TLSv1.1、TLSv1.2 及 TLSv1.3（如果 OpenSSL 库支持）。尽管协议名称可能不同，但 TLS 仍然建立了安全套接字层。NGINX 能够让您的服务保护您和客户端之间交换的信息，进而保护客户端和您的业务。

在使用 CA 签名的证书时，您需要将证书与证书颁发机构链连接起来。在连接证书和证书颁发机构链时，证书应位于证书颁发机构链文件之上。如果证书颁发机构提供了多个文件作为证书颁发机构链的中间证书，则它们会按顺序进行分层。有关排序问题，请参阅证书提供商的文档。

其他参考资料

[Mozilla 安全/服务器端 TLS](#)

[Mozilla SSL 配置生成器](#)

[SSL Labs 的 SSL 服务器测试](#)

7.4 高级客户端加密

问题

您具有高级客户端/服务器加密配置需求。

解决方案

NGINX 的 http 和 stream 模块可完全控制接受的 SSL/TLS 握手。证书和密钥可通过文件路径或可变值的方式提供给 NGINX。NGINX 根据其配置为客户端提供可接受的协议、密码和密钥类型列表。客户端与 NGINX 服务器之间的最高标准是经过协商的。NGINX 可以将客户端/服务器 SSL/TLS 的协商结果缓存一段时间。

下面的示例同时展示了许多选项，意在说明客户端/服务器协商的复杂性：

```
http { # 下面使用的所有指令在 stream 中也有效
    server {
        listen 8443 ssl;
        # 设置接受的协议和密码
        ssl_protocols TLSv1.2 TLSv1.3;
        ssl_ciphers HIGH:!aNULL:!MD5;

        # 从文件中加载 RSA 证书链
        ssl_certificate /etc/nginx/ssl/example.crt;
        # 从文件中加载 RSA 加密密钥
        ssl_certificate_key /etc/nginx/ssl/example.pem;

        # 从变量值生成椭圆曲线证书
        ssl_certificate $ecdsa_cert;
        # 作为文件路径变量的椭圆曲线密钥
        ssl_certificate_key data:$ecdsa_key_path;

        # 客户端-服务器协商缓存
        ssl_session_cache shared:SSL:10m;
        ssl_session_timeout 10m;
    }
}
```

服务器接受 SSL 协议版本 TLSv1.2 和 TLSv1.3。接受的密码设置为 HIGH，这是最高标准的宏；对于 aNULL 和 MD5，显式拒绝是通过 ! 符号来表示的。

使用了两组证书密钥对。传递给 NGINX 指令的值展示了提供 NGINX 证书密钥值的不同方式。变量被解释为文件的途径。当带有前缀 `data:` 时，变量值被解释为直接值。可提供多种证书密钥格式，以实现与客户端的反向兼容性。客户端支持的、服务器接受的最强标准将是协商的结果。



如果 SSL/TLS 密钥作为直接值变量暴露，则有可能被配置记录或暴露。如果将密钥值暴露为变量，请确保您拥有严格的变更和访问控制措施。

SSL 会话缓存和超时允许 NGINX worker 进程在给定的时间内缓存和存储会话参数。作为单个实例化中的进程，NGINX worker 进程会相互共享此缓存（而非在机器之间共享的缓存）。还有许多其他会话缓存选项可以帮助提高所有类型的用例的性能或安全性。您可以结合使用不同的会话缓存选项。但是，指定没有默认值的会话缓存后，默认的内置会话缓存将会关闭。

详解

在上面的高级配置示例中，NGINX 为客户端提供了备受推重的密码算法——TLSv1.2 或 1.3 的 SSL/TLS 选项，以及使用 RSA 或椭圆曲线密码（ECC）格式密钥的能力。客户端所能实现的最强协议、密码和关键格式是协商的结果。该配置指示 NGINX 将协商结果缓存 10 分钟，可用内存分配为 10MB。

测试发现，ECC 证书的速度比同等强度的 RSA 证书快。密钥占用空间较小，可以提供更多的 SSL/TLS 连接及更快的握手速度。NGINX 允许您配置多个证书和密钥，然后为客户浏览器提供最佳证书。您可以利用更新的技术，但仍然服务旧客户端。



在此示例中，NGINX 正在加密它与客户端之间的流量。但是，与上游（upstream）服务器的连接也可能进行了加密。[实操指南 7.5](#) 中演示了 NGINX 与上游服务器之间的协商。

其他参考资料

Mozilla 安全/服务器端 TLS

Mozilla SSL 配置生成器

SSL Labs 的 SSL 服务器测试

7.5 Upstream 加密

问题

您想要加密 NGINX 和上游服务之间的流量，并为合规性法规设置特定的协商规则；或者上游（upstream）服务位于有安全防护措施的网络之外。

解决方案

使用 HTTP 代理模块的 SSL 指令来指定 SSL 规则：

```
location / {  
    proxy_pass https://upstream.example.com;  
    proxy_ssl_verify on;  
    proxy_ssl_verify_depth 2;  
    proxy_ssl_protocols TLSv1.3;  
}
```

这些代理指令为 NGINX 设定了需要遵守的特定 SSL 规则。配置的指令可确保 NGINX 验证上游服务上的证书和证书链是否有效，验证深度最多为两个证书。proxy_ssl_protocols 指令指定 NGINX 只使用 TLS 版本 1.3。默认情况下，NGINX 不会验证上游证书并接受所有 TLS 版本。

详解

HTTP 代理模块的配置指令非常庞大，如果您需要加密上游流量，那么至少应该启用验证。您只需更改传递给 proxy_pass 指令的值对应的协议，即可通过 HTTPS 进行代理。但是，这不会验证上游证书。其他指令（如 proxy_ssl_certificate 和 proxy_ssl_certificate_key）允许您锁定上游加密，以增强安全性。您还可以指定 proxy_ssl_crl 或证书吊销列表，该列表列出了被认为无效的证书。这些 SSL 代理指令有助于增强系统在自己的网络或跨公共互联网的通信通道。

7.6 保护 location

问题

您希望使用 secret 保护 location 代码块。

解决方案

通过 secure link 模块和 secure_link_secret 指令，仅允许拥有安全链接的用户访问资源：

```
location /resources {
    secure_link_secret mySecret;
    if ($secure_link = "") { return 403; }

    rewrite ^ /secured/$secure_link;
}

location /secured/ {
    internal;
    root /var/www;
}
```

这种配置创建了一个内部和公共 location 代码块。请求 URI 需包含 md5 哈希字符串（可通过提供给 secure_link_secret 指令的 secret 进行验证），否则公共 location 代码块 /resources 将返回 403 Forbidden。URI 中的哈希值需进行验证，否则 \$secure_link 变量将是一个空字符串。

详解

使用 secret 保护资源是一种有效的文件保护方法。结合使用 secret 与 URI，然后对该字符串进行 md5 哈希计算，并在 URI 中使用这个 md5 哈希算法的十六进制摘要。将哈希值放置在链接中，并通过 NGINX 进行评估。NGINX 知道要请求的文件的路径，因为它位于哈希算法之后的 URI 中。NGINX 还知道您的 secret，因为它通过 secure_link_secret 指令提供。NGINX 能够快速验证 md5 哈希算法，并将 URI 存储在 \$secure_link 变量中。如果无法验证哈希值，则将变量设置为空字符串。需要注意的是，传递给 secure_link_secret 的参数必须是静态字符串，不能是变量。

7.7 使用 secret 生成安全链接

问题

您想要使用 secret 从应用中生成安全链接。

解决方案

NGINX 中的 `secure link` 模块接受 md5 哈希字符串的十六进制摘要，其中字符串由 URI 路径和 `secret` 组成。在上一节[实操指南 7.6](#) 的基础上，我们将创建一个安全链接，与前面的配置示例搭配使用，因为 `/var/www/secured/index.html` 上有一个文件。要生成 md5 哈希算法的十六进制摘要，我们可以使用 Unix `openssl` 命令：

```
$ echo -n 'index.htmlmySecret' | openssl md5 -hex
(stdin)= a53bee08a4bf0bbea978ddf736363a12
```

此处展示了我们要保护的 URI `index.html`，它与我们的 `secret`，即 `mySecret` 拼接在一起。该字符串被传递到 `openssl` 命令，输出 md5 十六进制摘要。

下面是使用 Python Standard Library 中包含的 `hashlib` 库在 Python 中生成相同哈希摘要的示例：

```
import hashlib
hashlib.md5(b'index.htmlmySecret').hexdigest()
'a53bee08a4bf0bbea978ddf736363a12'
```

现在我们得到了这个哈希摘要，并可以在 URL 中使用它了。我们的示例是 `www.example.com` 通过我们的 `/resources` 位置请求 `/var/www/secured/index.html` 上的文件。我们的完整 URL 将是：

```
www.example.com/resources/a53bee08a4bf0bbea978ddf736363a12/\
index.html
```

详解

我们可以通过多种方式，用多种语言生成哈希摘要。请记住：URI 路径在 `secret` 之前；字符串中没有回车；使用 md5 哈希算法的十六进制摘要。

7.8 使用过期日期保护 location

问题

您想要保护一个位置，该位置的链接将在未来某个时间过期，并且特定于某个客户端。

解决方案

利用 `secure link` 模块中的其他指令设置过期时间，并在 `secure link` 中使用变量：

```
location /resources {
    root /var/www;
    secure_link $arg_md5,$arg_expires;
    secure_link_md5 "$secure_link_expires$uri$remote_addrmySecret";
    if ($secure_link = "") { return 403; }
    if ($secure_link = "0") { return 410; }
}
```

secure_link 指令带两个参数，这两个参数用两个逗号隔开。第一个参数是保存 md5 哈希值的变量。本例中使用了 md5 的 HTTP 参数。第二个参数是一个变量，该变量保存了以 Unix 时间戳格式显示的链接过期时间。secure_link_md5 指令带一个参数，该参数声明了用于生成 md5 哈希值的字符串的格式。与其他配置一样，如果哈希值未经验证，则 \$secure_link 变量将设置为空字符串。但是，使用这种用法，如果哈希值匹配，但时间已过期，则 \$secure_link 变量将设置为 0。

详解

这种保护链接的方法比[实操指南 7.6](#)中所示的 secure_link_secret 更灵活、更简洁。通过这些指令，您可以在哈希字符串中使用 NGINX 可用的任意数量的变量。在哈希字符串中使用特定于用户的变量可增强安全性，因为用户无法交换到受保护资源的链接。建议使用类似于 \$remote_addr 或 \$http_x_forwarded_for 的变量，或者由应用生成的会话 cookie 请求头。secure_link 的参数可以来自您喜欢的任何变量，并且能够以最合适的方式命名。问题是：您拥有访问权限吗？您是否在链接有效期内访问？如果您没有访问权限，将会收到消息 Forbidden。如果您拥有访问权限，但链接却过期了，将会收到消息 Gone。HTTP 410 Gone 非常适合过期的链接，因为该条件应被视为永久性的。

7.9 生成过期链接

问题

您想要生成一个会过期的链接。

解决方案

为过期的时间生成一个 Unix 时间戳格式的时间戳。在 Unix 系统上，您可以使用如下所示的日期进行测试：

```
$ date -d "2030-12-31 00:00" +%s --utc
1924905600
```

接下来，您需要连接哈希字符串，以匹配使用 `secure_link_md5` 指令配置的字符串。在本例中，我们要使用的字符串将是 `1924905600/resources/index.html127.0.0.1mySecret`。md5 哈希值与十六进制摘要稍有不同。它是二进制格式的 md5 哈希值，base64 编码，加号 (+) 转换成了连字符 (-)，斜杠 (/) 转换成了下划线 (_)，删除了等号 (=)。以下是在 Unix 系统上操作的一个示例：

```
$ echo -n '1924905600/resources/index.html127.0.0.1 mySecret' \
| openssl md5 -binary \
| openssl base64 \
| tr +/ -_ \
| tr -d =
sqys0w5kMvQBL3j90DCyoQ
```

现在我们得到了哈希值，我们可以将它和过期日期用作参数：

```
/resources/index.html?md5=sqys0w5kMvQBL3j90DCyoQ&expires=1924905600
```

下面是 Python 中使用相对过期时间的更实际的示例，它将链接设置为生成后一小时过期。在撰写本文之时，此示例适用于 Python 2.7 和使用 Python 标准库的 3.x：

```
from datetime import datetime, timedelta
from base64 import b64encode
import hashlib

# 设置环境变量
resource = b'/resources/index.html'
remote_addr = b'127.0.0.1'
host = b'www.example.com'
mysecret = b'mySecret'

# 生成过期时间戳
now = datetime.utcnow()
expire_dt = now + timedelta(hours=1)
expire_epoch = str.encode(expire_dt.strftime('%s'))

# 对字符串进行 md5 哈希处理
uncoded = expire_epoch + resource + remote_addr + mysecret
md5hashed = hashlib.md5(uncoded).digest()

# 对字符串进行 Base64 编码和转换
b64 = b64encode(md5hashed)
unpadded_b64url = b64.replace(b'+', b'-')\
    .replace(b'/', b'_')\
    .replace(b'=', b'')

# 格式化并生成链接
linkFormat = "{}{}?md5={}?expires={}"
securelink = linkFormat.format(
    host.decode(),
```

```
resource.decode(),
unpadded_b64url.decode(),
expire_epoch.decode()
)
print(securelink)
```

详解

有了这种模式，我们能够生成具有特殊格式并能在 URL 中使用的安全链接。secret 通过使用从未发送给客户端的变量来提供安全性。您可以使用尽可能多的其他变量来保护位置的安全。md5 哈希计算和 base64 编码是通用的、轻量级的，并且支持几乎任何语言。

7.10 HTTPS 重定向

问题

您希望将未加密的请求重定向到 HTTPS。

解决方案

使用 rewrite 指令将所有 HTTP 流量发送到 HTTPS：

```
server {
    listen 80 default_server;
    listen [::]:80 default_server;
    server_name _;
    return 301 https://$host$request_uri;
}
server {
    listen 443 ssl;
    listen [::]:443 ssl;
    ssl_certificate /etc/nginx/ssl/example.crt;
    ssl_certificate_key /etc/nginx/ssl/example.key;
    ...
}
```

此配置用于侦听 IPv4 和 IPv6 以及任何主机名的默认服务器的 80 端口。return 语句向使用同一主机和请求 URI 的 HTTPS 服务器返回 301 永久重定向，该重定向由为 SSL/TLS 配置的 server 代码块进行处理。

详解

在适当的情况下始终重定向到 HTTPS，这是非常重要的。您可能会发现，不需要重定

向所有请求，而只需要重定向在客户端和服务器之间传递敏感信息的请求。在这种情况下，您可能希望只将 `return` 语句放在特定的位置，例如 `/login`。

7.11 在 NGINX 之前卸载 SSL/TLS 后重定向到 HTTPS

问题

您希望重定向到 HTTPS，然而，您已在 NGINX 之前的某个层卸载了 SSL/TLS。

解决方案

使用常见的 `X-Forwarded-Proto` 请求头确定是否需要重定向：

```
server {
    listen 80 default_server;
    listen [::]:80 default_server;
    server_name _;
    if ($http_x_forwarded_proto = 'http') {
        return 301 https://$host$request_uri;
    }
}
```

这种配置与 HTTPS 重定向十分相似。然而，在这个配置中，我们只在请求头 `X-Forwarded-Proto` 为 HTTP 时进行重定向。

详解

在 NGINX 前面的某个层卸载 SSL/TLS 是一种常见的做法，为的是节省计算成本。但是，您需要确保每个请求都是 HTTPS 请求，而卸载 SSL/TLS 的层面不具有重定向能力。但它可以设置代理请求头。这种配置适用于 Amazon Web Services Elastic Load Balancer (AWS ELB) 等层面，这些层可免费卸载 SSL/TLS。这是一个小技巧，可保护 HTTP 流量的安全。

7.12 HTTP 严格传输安全协议

问题

您希望指示浏览器永远不通过 HTTP 发送请求。

解决方案

设置 Strict-Transport-Security 请求头，使用 HTTP 严格传输安全协议（HSTS）增强功能：

```
add_header Strict-Transport-Security max-age=31536000;
```

此配置将 Strict-Transport-Security 请求头的使用时间设置为最长一年。如 HTTP 请求尝试访问该域，这将指示浏览器始终进行内部重定向，这样所有请求都将通过 HTTPS 发出。

详解

对于某些应用来说，一个 HTTP 请求遭到中间人攻击就可能会摧毁整个公司。如果一个包含敏感信息的表单通过 HTTP 发送，势必会造成损害，就算是通过 NGINX 进行 HTTPS 重定向也于事无补。这个可选的安全增强功能可指示浏览器，永远不要发出 HTTP 请求，因此请求始终都在加密后发出。

其他参考资料

[RFC HTTP 严格传输安全协议标准文档](#)

[OWASP HTTP 严格传输安全协议备忘单](#)

7.13 基于国家 / 地区的访问限制

问题

您想要根据合同或应用要求限制来自特定国家/地区的访问。

解决方案

从官方 NGINX 仓库为您的发行版安装 NGINX GeoIP 模块。

NGINX Plus：

```
$ apt install nginx-plus-module-geoip
```

NGINX 开源版：

```
$ apt install nginx-module-geoip
```

将您要阻止或允许的国家代码映射到变量：

```
load_module
    "/etc/nginx/modules/ngx_http_geoip_module.so";

http {
    map $geoip_country_code $country_access {
        "US" 0;
        "CA" 0;
        default 1;
    }
    # ...
}
```

此映射会将新变量 `$country_access` 设置为 1 或 0。如果客户端 IP 地址来自美国或加拿大，该变量将设置为 0；如果客户端 IP 地址来自其他国家/地区，该变量将设置为 1。

现在，在我们的 server 代码块中，我们将使用 `if` 语句来拒绝来自美国或加拿大以外的任何地址的访问：

```
server {
    if ($country_access = '1') {
        return 403;
    }
    # ...
}
```

如果 `$country_access` 变量设置为 1，则这个 `if` 语句的计算结果为 `True`。如果为 `True`，服务器将返回 403 Unauthorized，否则服务器就正常运行。因此，这个 `if` 代码块只是用来拒绝非美国或加拿大地区的地址。

详解

这个例子简明扼要地说明了如何仅允许几个国家/地区的访问，您可以根据自己的需求在此基础上进行延伸。您可以借鉴这一做法，根据 GeoIP 模块提供的任何嵌入式变量允许或阻止访问。

其他参考资料

[NGINX GeoIP 模块文档](#)

7.14 提供多种安全方法

问题

您希望提供多种方法将安全性传递到封闭的站点。

解决方案

使用 `satisfy` 指令，指示 NGINX 您想要满足所用的任何或所有安全方法：

```
location / {  
    satisfy any;  
  
    allow 192.168.1.0/24;  
    deny all;  
  
    auth_basic "closed site";  
    auth_basic_user_file conf/htpasswd;  
}
```

此配置告知 NGINX 请求 `location /` 的用户需要满足其中一种安全方法：请求需要来自 `192.168.1.0/24` CIDR 代码块，或者必须能够提供可以在 `conf/htpasswd` 文件中找到的用户名和密码。`satisfy` 指令带两个选项：`any` 或 `all`。

详解

`satisfy` 指令是一个不错的方法，它提供了多种验证 Web 应用的方法。在 `satisfy` 指令中指定 `any`，代表用户必须满足其中一个安全要求。在 `satisfy` 指令中指定 `all`，代表用户必须满足所有安全要求。这个指令可以与[实操指南 7.1](#)中详述的 `http_access_module`、[实操指南 6.1](#)中详述的 `http_auth_basic_module`、[实操指南 6.2](#)中详述的 `http_auth_request_module` 及[实操指南 6.3](#)中详述的 `http_auth_jwt_module` 一起使用。只有通过多层才能真正实现安全性。`satisfy` 指令可帮助您为需要深度安全规则的位置和服务器实现这个目标。

7.15 NGINX Plus 动态应用层 DDoS 防护

问题

您想要动态分布式拒绝服务（DDoS）防护解决方案。

解决方案

使用 NGINX App Protect DoS 模块或 NGINX Plus 功能构建集群感知型速率限制和自动拦截列表。

```
limit_req_zone $remote_addr zone=per_ip:1M rate=100r/s sync;
# 集群感知型速率限制

limit_req_status 429;

keyval_zone zone=sinbin:1M timeout=600 sync;
# 具有 10 分钟 TTL 的
# 集群感知型 "sin bin"

keyval $remote_addr $in_sinbin zone=sinbin;
# 使用匹配的客户端 IP 地址
# 填充 $in_sinbin

server {
    listen 80;
    location / {
        if ($in_sinbin) {
            set $limit_rate 50; # 限制恶意客户端的带宽
        }

        limit_req zone=per_ip;
        # 此处应用速率限制

        error_page 429 = @send_to_sinbin;
        # 过多的客户端被移动到
        # 此位置

        proxy_pass http://my_backend;
    }

    location @send_to_sinbin {
        rewrite ^ /api/9/http/keyvals/sinbin break;
        # 设置 "sin bin" 键值的
        # URI

        proxy_method POST;
        proxy_set_body '{"$remote_addr":"1"}';
        proxy_pass http://127.0.0.1:80;
    }

    location /api/ {
        api write=on;
        # 用于控制对 API 访问的指令
    }
}
```

详解

该解决方案通过使用同步键值（key-value）存储来使用同步速率限制，以动态响应 DDoS 攻击并减轻其影响。或者，如果您使用 NGINX App Protect，DoS 模块也提供了这一功能。提供给 `limit_req_zone` 和 `keyval_zone` 指令的 `sync` 参数将共享内存区与 NGINX Plus active-active 集群中的其他机器同步。上面的示例标识了每秒发送超过 100 个请求的客户端——无论哪个 NGINX Plus 节点接收请求。当客户端超过速率限制时，将通过调用 NGINX Plus API 将其 IP 地址添加到“sin bin”键值存储中。集群内的 sin bin 是同步的。无论哪个 NGINX Plus 节点接收请求，来自 sin bin 中的客户端的其他请求都会面临非常低的带宽限制。限制带宽比直接拒绝请求更可取，因为它不会向客户端清楚地表明 DDoS 防护已生效。10 分钟后，客户端将自动从 sin bin 中移除。

其他参考资料

F5 NGINX App Protect DoS

7.16 安装和配置 NGINX Plus 的 NGINX App Protect WAF 模块

问题

您想要安装和配置 NGINX App Protect WAF 模块。

解决方案

请参考《[NGINX App Protect WAF 管理指南](#)》。请不要跳过从单独的仓库中安装 NGINX App Protect WAF 签名的相关部分。

确保 NGINX App Protect WAF 模块由 NGINX Plus 在 main 上下文中使用 `load_module` 指令动态加载，并使用 `app_protect_*` 指令启用：

```
user nginx;
worker_processes auto;

load_module modules/ngx_http_app_protect_module.so;

# ...其他主要上下文指令

http {
    app_protect_enable on;
```

```

app_protect_policy_file "/etc/nginx/AppProtectTransparentPolicy.json";
app_protect_security_log_enable on;
app_protect_security_log "/etc/nginx/log-default.json"
    syslog:server=127.0.0.1:515;

# ...其他 http 上下文指令
}

```

在该示例中，`app_protect_enable` 指令设置为 `on`，启用了当前上下文的模块。该指令以及下面所有指令在 `http` 上下文，以及 `HTTP` 的 `server` 和 `location` 上下文中都是有效的。`app_protect_policy_file` 指令指向一个 NGINX App Protect WAF 策略文件，接下来我们将对其进行定义；如果没有定义，则使用默认策略。接下来配置了安全日志，并需要远程日志服务器。例如，我们将其配置为本地系统日志（Syslog）侦听器。`app_protect_security_log` 指令带两个参数：第一个参数是一个定义了日志设置的 JSON 文件，第二个参数是日志数据流目的地。日志设置文件将稍后在本节展示。

构建一个 NGINX App Protect WAF 策略文件，将其命名为 `/etc/nginx/AppProtectTransparentPolicy.json`：

```

{
  "policy": {
    "name": "transparent_policy",
    "template": { "name": "POLICY_TEMPLATE_NGINX_BASE" },
    "applicationLanguage": "utf-8",
    "enforcementMode": "transparent"
  }
}

```

该策略文件通过使用模板来配置默认的 NGINX App Protect WAF 策略，将策略名称设置为 `transparent_policy`，将 `enforcementMode` 设置为 `transparent`，这表示 NGINX Plus 将记录日志但不会拦截日志。透明（transparent）模式非常适合在新策略实施之前对其进行测试。

将 `enforcementMode` 改为 `blocking`，启用拦截模式。该策略文件可以命名为 `/etc/nginx/AppProtectBlockingPolicy.json`。要在两个文件之间切换，请在 NGINX Plus 配置中更新 `app_protect_policy_file` 指令：

```

{
  "policy": {
    "name": "blocking_policy",
    "template": { "name": "POLICY_TEMPLATE_NGINX_BASE" },
    "applicationLanguage": "utf-8",
    "enforcementMode": "blocking"
  }
}

```

要启用 NGINX App Protect WAF 的某些保护特性，需添加一些违规行为：

```
{
  "policy": {
    "name": "blocking_policy",
    "template": { "name": "POLICY_TEMPLATE_NGINX_BASE" },
    "applicationLanguage": "utf-8",
    "enforcementMode": "blocking",
    "blocking-settings": {
      "violations": [
        {
          "name": "VIOL_JSON_FORMAT",
          "alarm": true,
          "block": true
        },
        {
          "name": "VIOL_PARAMETER_VALUE_METACHAR",
          "alarm": true,
          "block": false
        }
      ]
    }
  }
}
```

在上面的示例中，向策略添加了两违规行为。注意，VIOL_PARAMETER_VALUE_METACHAR 未设置为 block，而只是设置为 alarm；而 VIOL_JSON_FORMAT 设置为 block 和 alarm。当设置为 blocking 时，该功能允许重写默认的 enforcementMode。当 enforcementMode 设置为 transparent 时，将优先使用默认的强制设置。

构建一个 NGINX Plus 日志文件，将其命名为 */etc/nginx/log-default.json*：

```
{
  "filter": {
    "request_type": "all"
  },
  "content": {
    "format": "default",
    "max_request_size": "any",
    "max_message_size": "5k"
  }
}
```

该文件由 app_protect_security_log 指令在 NGINX Plus 配置中定义，并且是 NGINX App Protect WAF 日志的必要文件。

详解

该解决方案演示了使用 NGINX App Protect WAF 模块配置 NGINX Plus 的基础知识。NGINX App Protect WAF 模块支持完整的 Web 应用防火墙 (WAF) 定义。这些定义源自 F5 公司提供的高级应用安全防护功能。这套全面的 WAF 攻击签名已得到广泛的实践测试和证明。通过在 NGINX Plus 安装过程中添加这些签名, 可同时获得强大的 F5 应用安全防护功能以及 NGINX 平台的敏捷性。

安装并启用该模块后, 大多数配置都在策略文件中完成。本节中的策略文件展示了如何启用主动拦截、被动监控和透明模式, 并解释了如何在出现违规时重写该功能。违规行为只是所提供的其中一种防护措施。其他防护措施包括 HTTP 合规、规避技术、攻击特征、服务器技术、数据保护等等。要追踪 NGINX App Protect WAF 日志, 需要使用 NGINX Plus 日志格式, 并将日志发送到远程侦听服务、文件或 `/dev/stderr`。

其他参考资料

[NGINX App Protect WAF 管理指南](#)

[NGINX App Protect WAF 配置指南](#)

[NGINX App Protect DoS 部署指南](#)

HTTP/2 和 HTTP/3 (QUIC)

8.0 简介

HTTP/2 和 HTTP/3 是 HTTP 协议的主要修订版。这两个修订版旨在解决被称为“队头阻塞”的性能问题，该问题会导致网络数据包因等待排在其前面的数据包完成处理而受阻。HTTP/2 通过在单个 TCP 连接上启用完整的请求和响应多路复用，缓解了应用层中初始 TCP 握手的队头阻塞问题。但 TCP 协议本身仍存在这个问题，因为其丢包恢复机制要求按顺序传输数据包，丢失的数据包必须重传后才能继续进行，故而会造成阻塞。HTTP/3 将 TCP 更换为了基于用户数据报协议 (UDP) 而构建的 QUIC (与 “quick” 同音) 协议。UDP 不需要连接握手、传输排序或实施丢包恢复，因此 QUIC 能够更高效地解决这些传输问题。

HTTP/2 还引入了 HTTP 请求头字段压缩技术，并添加了请求优先级支持。本章详细介绍了在 NGINX 中启用 HTTP/2 和 HTTP/3 的基本配置，以及如何配置 Google open source Remote Procedure Call (gRPC) 。

8.1 启用 HTTP/2

问题

您想要利用 HTTP/2。

解决方案

在 NGINX 服务器上启用 HTTP/2:

```
server {  
    listen 443 ssl default_server;  
    http2 on;  
  
    ssl_certificate server.crt;  
    ssl_certificate_key server.key;  
    # ...  
}
```

详解

要启用 HTTP/2, 您只需要将 http2 指令设置为 on 即可。然而, 尽管该协议不需要将连接封装在 SSL/TLS 中, 但 HTTP/2 客户端的某些实现仅支持加密连接上的 HTTP/2。另一个问题是, HTTP/2 规范阻止了许多 TLSv1.2 密码套件, 因此会导致握手失败。NGINX 默认使用的密码套件不在规范阻止列表上。TLS 的应用层协议协商 (Application-Layer Protocol Negotiation) 允许应用层协商在安全连接上使用哪个协议, 以避免额外往返。为了测试设置是否正确, 您可以在 Chrome 和 Firefox 浏览器中安装一个插件, 显示站点何时使用 HTTP/2, 或者在命令行上执行 nghttp 实用程序。

其他参考资料

[HTTP/2 RFC 密码套件黑名单](#)

[Chrome HTTP/2 和 SPDY 指示器插件](#)

[Firefox HTTP 版本指示器插件](#)

8.2 启用 HTTP/3

问题

您想要利用基于 QUIC 协议的 HTTP/3。

解决方案

在 NGINX 服务器上启用 QUIC, 并通过返回 Alt-Svc 请求头告知客户端该协议可用:

```

server {
    # 为了提高兼容性, 我们建议
    # 对 QUIC 和 TCP 使用相同的端口号
    listen 443 quic reuseport; # QUIC
    listen 443 ssl; # TCP

    ssl_certificate certs/example.com.crt;
    ssl_certificate_key certs/example.com.key;
    ssl_protocols TLSv1.3;

    location / {
        # 告知 QUIC 在配置端口上可用
        add_header Alt-Svc 'h3=":$server_port"; ma=86400';
        # ...
    }
}

```

详解

要启用 HTTP/3, 您只需要将 quic 参数添加到 listen 指令即可。您还可以而且应该配置另一个 listen 指令, 指示 NGINX 在同一端口上侦听 TCP 连接。QUIC 协议要求使用 TLSv1.3, 因此我们将 ssl_protocol 指令配置为只与 TLSv1.3 进行协商。

为了告知客户端所请求的 HTTP 服务可通过 HTTP/3 获取, 返回一个名为 Alt-Svc 的 HTTP 请求头。客户端首次向服务发出请求时, 将使用 HTTP/1.1, 并接收带有此请求头的响应。其值会告知客户端哪些服务可用 (h3 表示 HTTP/3) 以及服务对应的端口号。在示例中, 我们在同一端口上侦听 TCP 和 UDP, 因此可以使用 \$server_port 变量来减少硬编码值。切记, 返回的端口号应为客户端应连接的端口, 如果您正在后台进行任何端口映射, 该端口号可能不同于 NGINX 正在侦听的端口。此外, 它还定义了该服务的预期可用时长。因此, 客户端在指定的时间内 (在本例中为 86400 秒, 即 24 小时) 不断对该服务发出 UDP 请求。

从 NGINX 开源版 1.25.2 和 NGINX Plus R30 版本开始, --with-http_v3_module 成为了默认模块。在这些版本之前, 要使用 HTTP/3, 必须使用源代码编译 NGINX, 并添加备用 SSL 库和 --with-http_v3_module 配置标志。在撰写本文时, OpenSSL 尚不支持 QUIC 的 TLS 接口。F5 NGINX 团队开发了一个 OpenSSL 兼容层, 这样就无需构建和交付第三方 TLS 库, 例如 quictls、BoringSSL 和 LibreSSL。

其他参考资料

“隆重推出 NGINX Plus R30”

“入门指导：NGINX 中的 QUIC 网络连接和加密”

“NGINX QUIC+HTTP/3 预览版的二进制包现已发布”

NGINX HTTP V3 模块文档

8.3 gRPC

问题

您希望终止、检查、路由或负载均衡 gRPC 方法调用。

解决方案

使用 NGINX 代理 gRPC 连接：

```
server {  
    listen 80;  
  
    http2 on;  
    location / {  
        grpc_pass grpc://backend.local:50051;  
    }  
}
```

在这个配置中，NGINX 侦听端口 80 上未加密的 HTTP/2 流量，并将这些流量代理到端口 50051 上名为 backend.local 的机器。grpc_pass 指令指示 NGINX 将该通信视为 gRPC 调用。后端服务器前面的 grpc:// 不是必需的，但它可以直接显示后端通信是未加密的。

要在客户端和 NGINX 之间利用 TLS 加密，并在将调用传递给应用服务器之前终止加密，需像在第一节中那样启用 SSL 和 HTTP/2：

```
server {  
    listen 443 ssl default_server;  
  
    http2 on;  
    ssl_certificate server.crt;  
    ssl_certificate_key server.key;  
    location / {  
        grpc_pass grpc://backend.local:50051;  
    }  
}
```

该配置在 NGINX 上终止了 TLS，并通过未加密的 HTTP/2 将 gRPC 通信传递到应用。

要配置 NGINX 加密与应用服务器的 gRPC 通信，提供端到端加密流量，只需修改 `grpc_pass` 指令，在服务器信息之前指定 `grpc://`（注意增加的 `s` 表示安全通信）：

```
grpc_pass grpc://backend.local:50051;
```

您还可以根据 gRPC URI（包括软件包、服务和方法）通过 NGINX 去路由调用到不同的后端服务。方法是使用 `location` 指令：

```
location /mypackage.service1 {
    grpc_pass grpc://$grpc_service1;
}
location /mypackage.service2 {
    grpc_pass grpc://$grpc_service2;
}
location / {
    root /usr/share/nginx/html;
    index index.html index.htm;
}
```

该配置示例使用 `location` 指令在两个单独的 gRPC 服务之间路由传入的 HTTP/2 流量，并使用一个 `location` 提供静态内容。对 `mypackage.service1` 服务的方法调用被定向到变量 `grpc_service1` 的值，可能包含主机名或 IP 和可选端口。对 `mypackage.service2` 的调用被定向到变量 `grpc_service2` 的值。`location /` 捕获任何其他 HTTP 请求并提供静态内容。这展示了 NGINX 如何在相同的 HTTP/2 端点和相应的路由下支持 gRPC 和非 gRPC。

对 gRPC 调用的负载均衡也类似于非 gRPC HTTP 流量：

```
upstream grpcservers {
    server backend1.local:50051;
    server backend2.local:50051;
}
server {
    listen 443 ssl http2 default_server;

    ssl_certificate server.crt;
    ssl_certificate_key server.key;
    location / {
        grpc_pass grpc://grpcservers;
    }
}
```

upstream 代码块处理 gRPC 的方式与处理其他 HTTP 流量的方式完全相同。唯一的区别是 upstream 由 grpc_pass 引用。

详解

NGINX 能够接收、代理、负载均衡、路由和终止加密的 gRPC 调用。借助 gRPC 模块，NGINX 可设置、更改或删除 gRPC 调用请求头，设置请求超时，及设置上游（upstream）SSL/TLS 规范。当 gRPC 通过 HTTP/2 协议进行通信时，您可以配置 NGINX 在同一端点上接受 gRPC 和非 gRPC Web 流量。

复杂的媒体串流

9.0 简介

本章涵盖了 MPEG-4 (MP4) 或 Flash Video (FLV) 格式的 NGINX 的流媒体。NGINX 被广泛用于向大众分发和串流内容。NGINX 支持行业标准格式和串流技术，本章将对此展开介绍。NGINX Plus 可通过 HTTP Live Streaming (HLS) 模块动态分割内容，及提供已分段媒体的 HTTP Dynamic Streaming (HDS)。NGINX 具有原生的带宽限制功能，并且 NGINX Plus 的高级功能提供了比特率限制，能够以最有效的方式交付内容，同时以最少的服务器资源触及最多用户。

9.1 传输 MP4 和 FLV 格式的文件

问题

您希望流式传输 MP4 或 FLV 格式的数字媒体。

解决方案

指定一个 HTTP location 代码块，提供 *.mp4* 或 *.flv* 格式的视频。NGINX 将使用渐进式下载或 HTTP 伪流技术流式传输媒体并支持搜索：

```
http {  
    server {  
        # ...  
  
        location /videos/ {
```

```

        mp4;
    }
    location ~ /\.flv$ {
        flv;
    }
}
}

```

示例中的第一个 location 代码块告知 NGINX，*videos* 目录中的文件是 MP4 格式的，可以通过渐进式下载进行传输。第二个 location 代码块告知 NGINX，所有以 *.flv* 结尾的文件都是 FLV 格式的，可以利用 HTTP 伪流技术进行传输。

详解

NGINX 中的流视频或音频文件处理就像单个指令一样简单。渐进式下载使客户能够在文件下载完成之前播放媒体文件。NGINX 为两种格式的媒体提供未下载部分搜索支持。

9.2 使用 NGINX Plus 的 HLS 模块进行流式传输

问题

您想要 HTTP Live Streaming (HLS) 处理 MP4 文件中封装的 H.264/AAC 编码内容。

解决方案

利用 NGINX Plus 的 HLS 模块进行实时分段、分包和多路复用，及控制分段缓冲等等，例如转发 HLS 参数：

```

location /hls/ {
    hls; # 使用 HLS 处理程序来管理请求

    # 从以下位置提供内容
    alias /var/www/video;

    # HLS 参数
    hls_fragment 4s;
    hls_buffers 10 10m;
    hls_mp4_buffer_size 1m;
    hls_mp4_max_buffer_size 5m;
}

```

该 location 代码块指示 NGINX 流式传输来自 */var/www/video* 目录的 HLS 媒体，并将该媒体分割成 4 秒长度的片段。HLS 缓冲区的数量设置为 10，大小为 10MB。初始 MP4 缓冲区大小设置为 1MB，最大为 5MB。

详解

NGINX Plus 的 HLS 模块支持动态复用转换 MP4 媒体文件。许多指令都支持控制媒体的分段和缓冲方式。location 代码块必须配置为使用 HLS 处理程序将媒体作为 HLS 流进行传输。HLS 分段时长设置为几秒，指示 NGINX 按时间长度对媒体进行分段。缓冲的数据量通过 hls_buffers 指令进行设置，该指令指定了缓冲区的数量和大小。允许客户端在缓冲数据达到一定数量（由 hls_mp4_buffer_size 指定）后开始播放媒体文件。但这可能需要较大的缓冲区，因为视频相关的元数据可能会超过初始缓冲区的大小。缓冲区的上限通过 hls_mp4_max_buffer_size 设置。NGINX 可通过这些缓冲变量优化最终用户体验。为这些指令选择正确的值需要了解目标受众和媒体。举例来说，如果您的大部分媒体是大型视频文件，并且目标受众拥有很高的带宽，那么您可以选择更大的最大缓冲区和更长时间的分段。这样，与内容相关的元数据在初始下载时不会出错，并且用户能够接收到更大的片段。

9.3 使用 NGINX Plus 的 HDS 模块进行流式传输问题

需要支持已经分段且与元数据分开的 Adobe HTTP Dynamic Streaming (HDS) 文件。

解决方案

NGINX Plus 通过 f4f 模块支持分段的 FLV 文件，使用该功能向用户提供 Adobe Adaptive Streaming (Adobe 自适应流媒体)：

```
location /video/ {  
    alias /var/www/transformed_video;  
    f4f;  
    f4f_buffer_size 512k;  
}
```

该示例指示 NGINX Plus 使用 NGINX Plus f4f 模块向客户端提供来自磁盘某个位置的此前已分段的媒体。索引文件 (.f4x) 的缓冲区大小设置为 512KB。

详解

NGINX Plus f4f 模块支持 NGINX 向最终用户提供此前已分段的媒体。这样的配置就像在 HTTP location 代码块内部使用 f4f 处理程序一样简单。f4f_buffer_size 指令为此类媒体的索引文件配置缓冲区大小。

9.4 使用 NGINX Plus 限制带宽

问题

您希望限制下游（downstream）媒体串流客户端的带宽，且不会影响观看体验。

解决方案

利用 NGINX Plus 的比特率限制功能处理 MP4 媒体文件：

```
location /video/ {  
    mp4;  
    mp4_limit_rate_after 15s;  
    mp4_limit_rate 1.2;  
}
```

该配置允许下游客户端在应用比特率限制之前下载 15 秒。15 秒钟过后，客户端能够以 120% 的比特率速率下载媒体，因此客户端的下载速度始终快于媒体播放速度。

详解

NGINX Plus 的比特率限制能够让流媒体服务器根据传输的媒体动态限制带宽，客户端可以下载尽可能多的内容，从而确保无缝的用户体验。[实操指南 9.1](#) 中描述的 MP4 处理程序将这个 location 代码块指定为流式传输 MP4 媒体格式。速率限制指令（例如 mp4_limit_rate_after）告知 NGINX 仅在指定的时间（几秒钟）之后限制流量的速度。MP4 速率限制方面的另一个指令是 mp4_limit_rate，该指令指定了允许客户端下载的比特率（相对于媒体的比特率）。赋予 mp4_limit_rate 指令的值 1 指定 NGINX 将带宽限制为媒体的比特率（1 比 1）。如果为 mp4_limit_rate 指令提供的值超过 1，用户将能够以超过观看的速度下载，进而缓冲媒体，并在下载时无缝观看媒体内容。

10.0 简介

云提供商的出现改变了 Web 应用托管的现状。过去，配置新机器等流程通常需要花费数小时到几个月的时间，如今，只需点击或调用 API 就能创建一个流程。这些云提供商通过按使用付费（即用即付）的模式租赁虚拟机（称为基础架构即服务（IaaS））或托管软件解决方案。工程师可以构建整个测试环境，并在不需要时将其拆除。这些云提供商还支持应用根据性能需求即时实现水平扩展。本章介绍了 NGINX 在几个主要云提供商平台上的基本部署。

10.1 自动配置

问题

您希望在 Amazon Web Services (AWS) 上自动配置 NGINX 服务器，以实现机器的自动配置。

解决方案

本节将以 AWS 为例；不过，自动配置核心理念也适用于 Azure、Google Cloud Platform (GCP) 和 DigitalOcean 等其他云提供商。

利用 Amazon 弹性计算云 (EC2) UserData 以及预配置的 Amazon 机器镜像 (AMI) 。使用 NGINX 以及安装的任何受支持的软件包创建 AMI。利用 EC2 UserData 在运行时配置任何环境特定的配置。

详解

在 AWS 上配置 NGINX 服务器时，有三种思维方式：

启动时配置

首先使用通用的 Linux 镜像启动，然后在启动时运行配置管理或 shell 脚本，以配置服务器。这种方式的启动速度很慢，并且容易出错。

完全配置 AMI

完全配置服务器，然后刻录 AMI。这种方式的启动速度非常快，并且非常准确。但在配置周围环境时不够灵活，而且维护许多镜像可能会很复杂。

部分配置 AMI

它集成了前面两种方式的优点。部分配置是将软件要求安装并刻录到 AMI，并且环境配置在启动时完成。这种方式比完全配置灵活，比启动时配置快速。

无论选择部分配置还是完全配置 AMI，都需要自动化配置过程。要创建 AMI 构建流水线，建议使用几个工具：

配置管理

配置管理工具在代码中定义了服务器的状态，例如运行哪个版本的 NGINX、以什么用户的身份运行、要使用的 DNS 解析器以及由谁代理上游 (upstream) 。该配置管理代码可以像软件项目一样进行源代码控制和版本控制。[第 5 章](#)描述了一些热门的配置管理工具，例如 Chef 和 Ansible。

来自 HashiCorp 的 Packer

Packer 可用于自动运行几乎任何虚拟化或云平台的配置管理，并在运行成功后刻录机器镜像。Packer 首先在您选择的平台上构建了一个虚拟机 (VM)，然后使用 SSH 连接虚拟机，运行您指定的任何配置，并刻录镜像。您可以利用 Packer 运行配置管理工具，并根据您的规格要求安全可靠地刻录机器镜像。

如要在启动时进行环境配置，您可以利用 Amazon EC2 UserData 在实例第一次启动时运行命令。如果您使用的是部分配置方法，则可以在启动时利用它来配置基于环境的项目。基于环境的配置可能包括侦听的服务器名称、要使用的解析器、要代理的域名或开始使用的上游（upstream）服务器池。UserData 是一个 base64 编码的字符串，在第一次启动和运行时下载。UserData 可以像 AMI 中其他启动进程访问的环境文件一样简单，也可以是用 AMI 中存在的任何语言编写的脚本。UserData 通常是一个 bash 脚本，用于指定变量或下载变量，并将其传递给配置管理工具。配置管理工具可确保系统配置无误，模板配置文件基于环境变量及重新加载服务。UserData 运行之后，NGINX 机器应已安全可靠地进行了完全配置。

10.2 将 NGINX 虚拟机部署到云端

问题

您想要云提供商平台中创建 NGINX 服务器，对其余资源进行负载均衡或代理。

解决方案

本节将以 Azure 为例；不过，部署 NGINX 虚拟机的核心理念也适用于 AWS、GCP 和 DigitalOcean 等其他云提供商。

在控制台的“Azure Virtual Machine (Azure 虚拟机)”部分中新建一台虚拟机。系统会提示您选择“Azure Subscription and Resource Group (Azure 订阅和资源组)”。如果没有任何资源组，则使用该选项创建一个。提供虚拟机名称以及“Region (区域)”、“Availability Options (可用性选项)”、“Security Type (安全类型)”和“Base Image (基础镜像)”。选择您最熟悉的 Linux 发行版。您还必须提供所需的“Size of the VM (虚拟机大小)”以及需要 Azure 如何设置管理员账户的信息。在“Inbound Ports (入站端口)”部分，确保端口 22 已打开，可通过 SSH 访问。点击“Next (下一步)”按钮，进入“Disk Configuration (磁盘配置)”部分。

输入最适合您工作负载的磁盘配置选项。要使用 NGINX 提供静态资产或缓存资源，则需要考虑磁盘大小和速度，然后进入“Networking (网络)”部分。

在“网络”部分，为虚拟机选择或新建虚拟网络和子网。为该虚拟机分配一个公网 IP，以便从公共互联网对其进行访问，并同样确保允许端口 22 用于 SSH 访问。为了方便清理，您可能需要选择“Delete the public IP and NIC when the VM is deleted（删除虚拟机时删除公共 IP 和网卡）”选项。

除非有特殊情况，否则其余部分可以保持默认设置。选择“Review and Create（查看和创建）”按钮。查看配置并点击“Create（创建）”按钮，然后等待 Azure 控制台进入部署进度界面。

Azure 将创建和配置所有必要的资源，同时向您展示进度。创建完所有资源后，“Go to resource（转到资源）”按钮将变为蓝色。此时，您可以点击该按钮查看新建虚拟机的相关信息。

在“Properties（属性）”选项卡的“网络”部分，找到虚拟机的公网 IP。使用此 IP 和配置过程中提供的管理员凭证，对虚拟机进行 SSH 访问。通过指定操作系统类型的软件包管理器安装 NGINX 或 NGINX Plus。根据您的需求配置 NGINX，然后重新加载。另外，您可通过自定义数据启动脚本安装和配置 NGINX，这是创建虚拟机时“Advanced（高级）”部分中的一个配置选项。

详解

Azure 可即刻提供高度可配置的虚拟机，并在虚拟化云环境中提供网络和计算资源。虚拟机启动不费吹灰之力，即可为您创造各种可能性。借助 Azure 虚拟机，无论何时何地您都可以使用 NGINX 服务器的全部功能。

10.3 创建 NGINX 机器镜像

问题

您想要创建 NGINX 机器镜像，以快速实例化虚拟机或为实例组创建实例模板。

解决方案

本节将以 GCP 为例；不过，创建机器映像的核心理念也适用于 AWS、Azure 和 DigitalOcean 等其他云提供商。

在虚拟机实例上安装和配置 NGINX 后，将启动盘的自动删除状态设为 `false`。要设置磁盘的自动删除状态，请编辑虚拟机。在磁盘配置下的“Edit（编辑）”页面上有一个标记为“Delete boot disk when instance is deleted（删除实例时删除启动盘）”的复选框。取消勾选此复选框并保存虚拟机配置。实例的自动删除状态设置为 `false` 后，删除实例。出现提示后，请勿选择提供删除启动盘的复选框。通过执行这些任务，您将会得到一个未连接的 NGINX 启动盘。

删除实例并得到未连接的启动盘后，即可创建 Google Compute Image。在 Google Compute Engine 控制台的“Image（镜像）”部分，选择“Create Image（创建镜像）”。系统将提示您输入镜像名称、系列、描述、加密类型和源。您需要使用的源类型是磁盘；为源磁盘选择未连接的 NGINX 启动盘。选择 Create（创建），Google Compute Cloud 将从磁盘创建镜像。

详解

您可以利用 Google Cloud Images 创建其引导盘与刚刚创建的服务器相同的虚拟机。创建镜像的价值在于可确保此镜像的每个实例都是相同的。在动态环境中的启动过程中安装软件包时，除非使用私有仓库的版本锁定，否则您将面临软件包版本和更新在生产环境中运行之前未被验证的风险。借助机器镜像，您可以验证在该机器上运行的每个软件包都完全与测试相同，从而增强了服务提供时的可靠性。

其他参考资料

Google Cloud 镜像：[创建自定义镜像文档](#)

10.4 无需云原生负载均衡器将流量路由到 NGINX 节点

问题

您想要将流量路由到多个 active-active NGINX 节点或创建一个 active-passive 故障转移集，从而无需在 NGINX 前面放置负载均衡器便可以实现高可用性。

解决方案

本节将以 AWS 为例；不过，DNS A 记录的核心理念也适用于 Azure、GCP 和 DigitalOcean 等其他云提供商。

使用 Amazon Route 53 DNS 服务将流量路由到多个 active-active NGINX 节点，或者在一组 active-passive NGINX 节点之间配置健康检查和故障转移。

详解

DNS 一直都对不同的服务器实施负载均衡，即使是迁移到云也不会改变这一点。Amazon 的 Route 53 服务提供了 DNS 服务，它具有许多高级特性，并且都可通过一个 API 获得。Route 53 服务具有所有典型的 DNS 技巧，例如单个 A 记录上的多个 IP 地址和加权 A 记录。在运行 active-active NGINX 节点时，您需要使用其中一个 A 记录特性在所有节点上分配负载。当一个 A 记录中列出多个 IP 地址时，使用轮询算法。加权分布可通过定义 A 记录中每个服务器 IP 地址的权重来实现负载的不均匀分布。

Route 53 另外一个更有趣的功能是健康检查功能。您可以通过建立 TCP 连接或发送 HTTP 或 HTTPS 请求来配置 Route 53 监控端点的健康。健康检查具有高度可配置的选项，包括 IP、主机名、端口、URI 路径、间隔速率、监控和地理位置。通过这些健康检查，Route 53 可在 IP 地址失效时终止其循环。您还可以配置 Route 53 在发生故障时将其转移到次级记录，这将是一个高度可用的 active-passive 设置。

Route 53 具有基于地理位置的路由功能，支持以最少的延迟将客户端路由到距其最近的 NGINX 节点。按照地理位置进行路由时，您的客户端将被定向到最近的运行健康的物理位置。当运行采用 active-active 配置的多组基础架构时，可通过使用健康检查自动故障转移到另一个地理位置。

当使用 Route 53 DNS 将流量路由到 Auto Scaling 组的 NGINX 节点时，需要自动创建和删除 DNS 记录。如要在 NGINX 节点扩展时自动添加和删除 Route 53 上的 NGINX 机器，您可以使用 Amazon 的 Auto Scaling 生命周期挂钩触发 NGINX 机器中的脚本或在 Amazon Lambda 上独立运行的脚本。这些脚本将使用 Amazon 命令行接口（CLI）或软件开发套件（SDK）与 Amazon Route 53 API 进行连接，在启动时或终止前添加或删除 NGINX 机器 IP 和之前配置的健康检查。

其他参考资料

使用 Amazon Route 53 和 NGINX Plus 实现全局服务器负载均衡

使用 Lambda 服务为 AWS EC2 实例设置动态 DNS

10.5 负载均衡器 Sandwich

问题

您想要自动扩展 NGINX 层，并在应用服务器之间均匀、轻松地分配负载。

解决方案

本节将以 AWS 为例；不过，在云原生负载均衡器之间实现 NGINX sandwich 的核心理念也适用于 Azure、GCP 和 DigitalOcean 等其他云提供商。NGINX Plus 提供了 NLB 的功能集，因此该模式仅适用于 NGINX 开源版。

创建一个网络负载均衡器（NLB）。通过控制台创建 NLB 时，系统会提示您创建一个新的目标组。如果不通过控制台创建 NLB，则需要创建此类资源并将其附加到 NLB 上的侦听器。通过启动配置创建一个 Auto Scaling 组，并且该启动配置中配置安装了 NGINX 的 EC2 实例。Auto Scaling 组有一个链接到目标组的配置，该配置自动将 Auto Scaling 组中的任何实例注册到第一次启动时配置的目标组。目标组由 NLB 上的侦听器引用。将上游（upstream）应用放在另一个网络负载均衡器和目标组之后，然后将 NGINX 配置为代理应用 NLB。

详解

这种模式十分常见，被称为“负载均衡器 sandwich”（参见图 10-1），它将 NGINX 放在 NLB 后面的一个 Auto Scaling 组中，同时将应用 Auto Scaling 组放在另一个 NLB 的后面。在每一层之间使用 NLB 的原因是 NLB 能够与 Auto Scaling 组完美配合使用；它们能够自动注册新的节点并删除被终止的节点，还能运行健康检查，并只将流量传递给健康的节点。

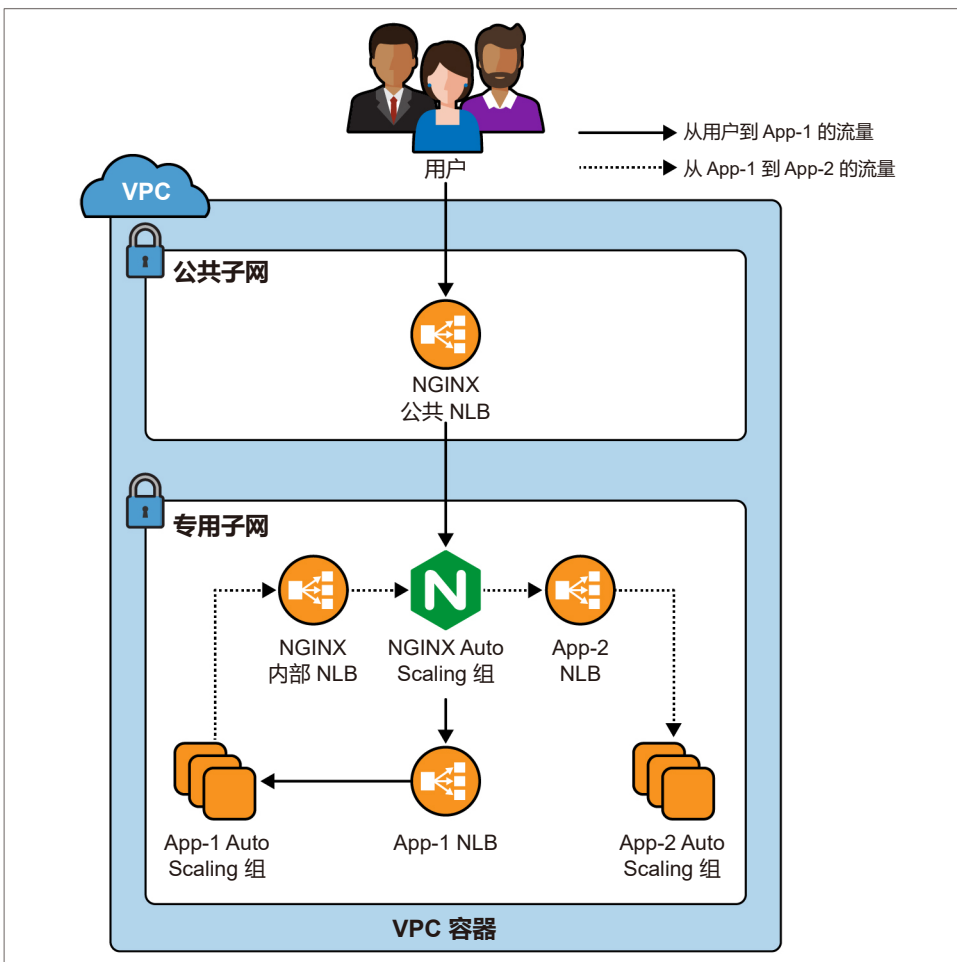


图 10-1: 该图描述了 NGINX 的负载均衡器 Sandwich 模式，它具有一个供内部应用使用的内部 NLB。用户向 App-1 发出请求，App-1 通过 NGINX 向 APP-2 发出请求，从而完成用户的请求。

您可能需要为 NGINX（开源版）层构建第二个内部 NLB，以便应用中的服务通过 NGINX Auto Scaling 组调用其他服务，而无需离开网络并通过公共 NLB 重新进入。这使得 NGINX 位于应用中所有网络流量的中间位置，从而成为应用流量路由的核心。该模式最初称为“elastic load balancer (ELB) sandwich”，然而当使用 NGINX 时，NLB 更受欢迎，因为 NLB 是四层负载均衡器，而 ELB 和 ALB 是七层负载均衡器。七层负载均衡器通过 PROXY 协议转换请求，无需使用 NGINX。

您也可以在其他云提供商的产品中使用此模式。如果您使用 Azure 负载均衡器和规模集 (scale set) , 或 GCP 负载均衡器和 Auto Scaling 组, 则此概念同样适用。此模式的核心价值是使用云原生服务自动注册和负载均衡扩展的应用服务器, 以及使用 NGINX 执行代理逻辑。

10.6 通过动态扩展 NGINX 服务器实现负载均衡问题

您希望在云原生负载均衡器后面扩展 NGINX 节点, 以获得高可用性并使用动态资源。

解决方案

本节将以 Azure 为例; 不过, 扩展 NGINX 层以及如何将流量路由到该层的核心理念也适用于 AWS、GCP 和 DigitalOcean 等其他云提供商。

创建一个面向公众或内部的 Azure 负载均衡器。将 NGINX 虚拟机镜像或 Marketplace 中的 NGINX Plus 镜像部署到 Azure 虚拟机扩展集 (VMSS) 中。负载均衡器和 VMSS 部署完成后, 在负载均衡器上为 VMSS 配置一个后端池。为您想要接收流量的端口和协议设置负载均衡规则, 并将其定向到后端池。

详解

通过扩展 NGINX 来实现高可用性或处理峰值负载而不过度提供资源十分常见。而在 Azure 中, 可通过 VMSS 来实现这一点。使用 Azure 负载均衡器可在扩展时轻松添加和删除资源池中的 NGINX 节点。借助 Azure 负载均衡器, 您可以检查后端池的健康状况, 并且只能将流量传递到健康的节点。您可以在 NGINX 前面运行内部 Azure 负载均衡器, 这样就可以只通过内部网络访问 NGINX。您可以使用 NGINX 代理 VMSS 内某个应用前的内部负载均衡器, 然后使用这个负载均衡器轻松地在池中进行注册和注销。

10.7 创建 Google App Engine 代理

问题

您想要为 Google App Engine 创建代理，以便在应用之间进行上下文切换或在自定义域中提供 HTTPS 服务。

解决方案

在 Google Compute Cloud 中使用 NGINX。在 Google Compute Engine 中创建虚拟机，或创建一个安装了 NGINX 的虚拟机镜像，并将其作为引导盘创建实例模板。实例模板创建完成后，创建一个使用该模板的实例组。

将 NGINX 配置为代理 Google App Engine 端点。Google App Engine 是公共的，所以确保代理使用 HTTPS，同时还要确保不会在 NGINX 实例中终止 HTTPS，从而允许信息在 NGINX 和 Google App Engine 之间进行不安全的传递。因为 App Engine 仅提供了一个 DNS 端点，所以当使用 NGINX 开源版时，您将需要使用 `proxy_pass` 指令，而不是 `upstream` 代码块，因为仅 NGINX Plus 提供 `upstream` DNS 解析。在代理到 Google App Engine 时，确保将端点设置为 NGINX 中的变量，然后在 `proxy_pass` 中使用该变量，从而确保 NGINX 对每个请求都进行 DNS 解析。为了让 NGINX 执行任何 DNS 解析，您还需要使用 `resolver` 指令，定向到您喜欢的 DNS 解析器。谷歌为大众提供了 IP 地址 8.8.8.8。如果您正在使用 NGINX Plus，您将能够在代理到 Google App Engine 时，使用 `upstream` 代码块中 `server` 指令上的 `resolve` 标志、`keepalive` 连接并获得 `upstream` 模块的其他好处。

您可以选择将 NGINX 配置文件存储在 Google Cloud Storage 中，然后使用实例的启动脚本在启动时下拉配置。无需刻录新镜像即可更改配置。但是，它会增加 NGINX 服务器的启动时间。

详解

如果您使用自己的域，并且希望通过 HTTPS 提供应用，则需要在 Google App Engine 前运行 NGINX。目前，Google App Engine 不支持上传自己的 SSL 证书。因此，如果您想在 Google 提供的 `appspot.com` 以外的其他域为应用提供加密服务，则需使用 NGINX 创建一个代理来侦听您的自定义域。NGINX 将对自己与客户端之间，以及自己与 Google App Engine 之间的通信进行加密。

您想在 Google App Engine 前运行 NGINX 的另一个原因是，在同一个域下托管多个 App Engine 应用，并使用 NGINX 进行基于 URI 的上下文切换。微服务是一种流行的架构，使用 NGINX 等代理路由流量十分常见。Google App Engine 简化了应用的部署，与 NGINX 相结合可提供功能完整的应用交付平台。

11.0 简介

容器在应用层提供了一层抽象，将软件包和依赖项的安装从部署流程转移到了构建流程。这点很重要，因为工程师现在发布的代码单元无论在何种环境中都以统一方式运行和部署。作为可运行单元提供的容器可降低环境之间发生依赖项和配置混乱的风险。考虑到这一点，组织都迫切希望将应用部署到容器平台上。在容器平台上运行应用时，通常会尽可能地将堆栈（包括代理或负载均衡器）容器化。NGINX 可以很容易的容器化。它还包含许多有助于容器化应用交付的特性。本章将重点介绍 NGINX 容器镜像的构建，有助于在容器化环境中运行的特性，以及 Kubernetes 和 OpenShift 之上的镜像部署。

在进行容器化时，通常的做法是将服务分解到多个较小的应用中，然后再通过 API 网关将分解的服务连接起来。本章介绍了使用 NGINX 作为 API 网关，对请求进行保护、校验、身份验证并将请求路由到适当服务的常见用例场景。

在容器中运行 NGINX 时，应留意几个架构注意事项。当将服务容器化，使用 Docker 日志驱动程序时，必须将日志输出到 `/dev/stdout` 并将错误日志定向到 `/dev/stderr`。通过这样做，您可以将日志传输到 Docker 日志驱动程序，Docker 驱动程序则能够将日志路由到本地合并日志服务器。

在容器化环境中使用 NGINX Plus 时，还需要考虑负载均衡算法。least_time 负载均衡算法专为容器化网络叠加层而设计。NGINX Plus 偏爱较短的响应时间，会将传入请

求传递给平均响应时间最短的上游服务器。当所有服务器经过充分的负载均衡，在等量负载下运行后，NGINX Plus 可通过优选最近网络上的服务器，降低网络延迟。

11.1 使用 NGINX 作为 API 网关

问题

您想要使用 API 网关对用例的传入请求进行校验、身份验证、操纵和路由。

解决方案

使用 NGINX 作为 API 网关。API 网关为接入一个或多个 API 提供了入口点。NGINX 非常适合这一角色。本节将重点介绍一些核心概念，并列出本书中可提供更多相关详情的章节。值得一提的是，NGINX 已就该主题发布了一本完整的电子书，即由 Liam Crilly 编写的《将 NGINX 部署为 API 网关》。

首先在 API 网关配置文件中为 API 网关定义 server 代码块。使用诸如 */etc/nginx/api_gateway.conf* 之类的名称命名：

```
server {
    listen 443 ssl;
    server_name api.company.com;
    # SSL 设置, 请见第 7 章

    default_type application/json;
}
```

在服务器定义中添加一些基本的错误处理响应：

```
proxy_intercept_errors on;

error_page 400 = @400;
location @400 { return 400 '{"status":400,"message":"Bad request"}\n'; }

error_page 401 = @401;
location @401 { return 401 '{"status":401,"message":"Unauthorized"}\n'; }

error_page 403 = @403;
location @403 { return 403 '{"status":403,"message":"Forbidden"}\n'; }

error_page 404 = @404;
location @404 { return 404 '{"status":404,"message":"Resource not found"}\n'; }
```

可直接将上一节中的 NGINX 配置添加到 `/etc/nginx/api_gateway.conf` 内的 `server` 代码块中或单独文件中，并通过 `include` 指令导入。有关 `include` 指令的介绍请参阅[实操指南 1.6](#)。

使用 `include` 指令将该服务器配置导入到 `http` 上下文中的主配置文件 `nginx.conf` 中：

```
include /etc/nginx/api_gateway.conf;
```

现在，需要定义上游服务端点。[第 2 章](#)介绍了负载均衡，并讨论了 `upstream` 代码块。值得注意的是，`upstream` 在 `http` 上下文中有效，在 `server` 上下文中无效。必须在 `server` 代码块外部包含或设置下列内容：

```
upstream service_1 {
    server 10.0.0.12:80;
    server 10.0.0.13:80;
}
upstream service_2 {
    server 10.0.0.14:80;
    server 10.0.0.15:80;
}
```

可根据用例将内联服务声明为内含文件或按服务包含的文件。还存在应将服务定义为代理位置端点的情况；在这种情况下，建议将该端点定义为可供自始至终使用的变量。[第 5 章“可编程性和自动化”](#)讨论了从 `upstream` 代码块中自动添加和删除机器的实现方法。

在 `server` 代码块中为每项服务构建一个内部可路由的位置：

```
location = /_service_1 {
    internal;
    # 服务通用配置
    proxy_pass http://service_1/$request_uri;
}
location = /_service_2 {
    internal;
    # 服务通用配置
    proxy_pass http://service_2/$request_uri;
}
```

通过为这些服务定义内部可路由位置，可一次性定义通用于这些服务的配置，从而避免反复定义。

现在您需要构建负责为给定服务定义特定 URI 路径的 `location` 代码块。这些代码块将对请求进行适当的验证和路由。API 网关可以只基于路径路由请求，也可以为接受的每

个 API URI 定义特定规则。在后一种情况中，需要设计文件组织结构，并使用 NGINX `include` 导入配置文件。有关该概念的讨论请参阅[实操指南 1.6](#)。

为 API 网关新建目录：

```
mkdir /etc/nginx/api_conf.d/
```

在适合配置结构的路径上的一个文件中定义 `location` 代码块，构建服务用例的规范。使用 `rewrite` 指令将请求定向到先前配置的 `location` 代码块，然后 `location` 代码块将请求代理到服务。在下面的示例中，`rewrite` 指令指示 NGINX 重新处理 URL 发生更改的请求。该示例定义了特定于 API 资源的规则，限制了 HTTP 方法，然后使用 `rewrite` 指令将请求发送到了先前为该服务定义的内部通用代理位置：

```
location /api/service_1/object {
    limit_except GET PUT { deny all; }
    rewrite ^ /_service_1 last;
}
location /api/service_1/object/[^/]*$ {
    limit_except GET POST { deny all; }
    rewrite ^ /_service_1 last;
}
```

为每项服务重复执行此步骤。通过为用例有效组织文件和目录结构，进行逻辑划分。利用本书提供的信息，对 API `location` 代码块进行尽可能具体、严格的配置。

如果将单独文件用于上述 `location` 或 `upstream` 代码块，则需要确保已将它们包含到服务器上下文中：

```
server {
    listen 443 ssl;
    server_name api.company.com;
    # SSL 设置，请见第 7 章

    default_type application/json;

    include api_conf.d/*.conf;
}
```

利用[第 6 章](#)中描述的其中一种方法，或者使用下面的预共享 API 密钥（注意 `map` 指令只在 http 上下文中有效），启用身份验证来保护私有资源：

```
map $http_apikey $api_client_name {
    default "";

    "j7UqLLB+yRv2VTCXXDZ1M/N4" "client_one";
    "6B2kbyrrTiIN8S8JhSxb63R" "client_two";
    "KcVgIDSY4Nm46m3tXVY3vbgA" "client_three";
}
```

利用第 2 章中介绍的使用限制方法，通过 NGINX 保护后端服务免遭攻击。在 http 上下文中，定义一个或多个请求限制共享内存区：

```
limit_req_zone $http_apikey
    zone=limitbyapikey:10m rate=100r/s;
limit_req_status 429;
```

使用速率限制和身份验证保护给定上下文：

```
location /api/service_2/object {
    limit_req zone=limitbyapikey;

    # 考虑将这些 if 语句写入文件中，
    # 并且需要使用 include。
    if ($http_apikey = "") {
        return 401;
    }
    if ($api_client_name = "") {
        return 403;
    }

    limit_except GET PUT { deny all; }
    rewrite ^ /_service_2 last;
}
```

测试对 API 网关的一些调用：

```
$ curl -H "apikey: 6B2kbyrrTiIN8S8JhSxb63R" \
    https://api.company.com/api/service_2/object
```

详解

API 网关为接入 API 提供了入口点。这听起来有点笼统含糊，下面就让我们深入研究下许多不同层次上的集成点。需要通信（集成）的任何两个独立服务都应持有 API 版本契约。此类版本契约定义了服务的兼容性。API 网关会强制执行此类契约，在服务之间对请求进行身份验证、授权、转换和路由。

本节介绍了 NGINX 如何通过对传入请求进行验证、身份验证，以及将其定向到特定服务和限制其使用，发挥 API 网关的作用。该策略在微服务架构中广受欢迎，在微服务架构中，单个 API 产品会被分解给多个不同的服务。

现在，借助上述信息，您可以按照适合用例的确切规格来构建 NGINX 服务器配置。通过综合应用文中介绍的核心概念，您可以根据任何因素，对 URI 路径的使用进行身份验证和授权，路由或重写请求，限制使用，并定义有效请求。API 网关永远不会只有一款解决方案，因为其解决方案与用例紧密相关，具有无限可能性。

API 网关为运维团队和应用团队提供了一个终极协作空间，可帮助打造真正的 DevOps 组织。应用开发团队定义给定请求的有效性参数。而此类请求的交付通常由 IT 团队（网络、基础架构、安全和中间件团队）管理。API 网关充当这两层之间的接口。API 网关的构造需要来自各方的输入。这种配置应被置于某种源控制之下。许多现代源控制仓库都具有代码所有者的概念。此概念允许将某些文件设置为需要特定用户的批准。通过这种方式，团队不仅可以开展协作，还可以各自验证特定于各自部门的更改。

使用 API 网关时需要注意 URI 路径。在示例配置中，整个 URI 路径都被传递给上游服务器。这意味着 `service_1` 示例需要在 `/api/service_1/*` 路径上具有处理程序。如果要以这种方式执行基于路径的路由，则最好是该应用与其他应用没有相互冲突的路由。

如果确实存在相互冲突的路由，则可以采取下列其中一种解决办法。通过编辑代码解决冲突，或者通过向一个或两个应用添加 URL 前缀配置，将其中一个应用移到另一个上下文中。对于无法编辑的现成软件，您可以重写请求的 URI upstream。但是，如果应用在消息体中返回链接，则需要使用正则表达式（regex）来重写请求的消息体，然后再将其提供给客户端——应避免消息体中出现链接。

其他参考资料

《将 NGINX 部署为 API 网关》（电子书）

11.2 在 NGINX Plus 中使用 DNS SRV 记录

问题

希望将现有 DNS 服务器的 SRV 记录用作 NGINX Plus 发现上游服务器的源。

解决方案

在上游服务器上下文中将该服务指令的值设置为 http，以指示 NGINX 将 SRV 记录用作负载均衡池：

```
http {  
    resolver 10.0.0.2 valid=30s;  
  
    upstream backend {  
        zone backends 64k;  
        server api.example.internal service=http resolve;  
    }  
}
```

此功能为 NGINX Plus 独有。此配置使用 server 指令指示 NGINX Plus 解析来自位于 10.0.0.2 的 DNS 服务器的 DNS，并设置上游服务器池。server 指令的 resolve 参数指示该指令基于 DNS 记录 TTL 或 resolver 指令中的 valid override 参数定期重新解析域名。service=http 参数和值告诉 NGINX 该 SRV 记录包含 IP 和端口列表，并指示 NGINX 在这些 IP 和端口上进行负载均衡，仿佛它们是使用 server 指令配置的。

详解

随着对云基础架构的需求量和采用率的上升，动态基础架构正变得越来越受欢迎。自动缩放环境可横向扩缩容，增加或减少池中的服务器数量以满足负载需求。横向扩缩需要一个可以向池中添加或从池中删除资源的负载均衡器。借助 SRV 记录，您可以将保存服务器列表的责任转交给 DNS。对于容器化环境来说，这种配置别具吸引力，因为可能有容器在可变端口号上运行应用，也可能有容器在同一 IP 地址上运行应用。需要注意的是，UDP DNS 记录有效载荷限制为 512 个字节左右。

11.3 使用官方 NGINX 容器镜像

问题

您想要使用来自 Docker Hub 的 NGINX 镜像，以快速启动并运行。

解决方案

使用来自 Docker Hub 的 NGINX 镜像。该镜像包含默认配置。如需要更改配置，要么挂载本地配置目录，要么为镜像构建 Dockerfile 和 COPY 以更改配置。现在我们挂载一个卷，其中 NGINX 的默认配置提供静态内容来演示其单命令功能：

```
$ docker run --name my-nginx -p 80:80 \
-v /path/to/content:/usr/share/nginx/html:ro -d nginx
```

如果无法在本地找到，docker 命令就会从 Docker Hub 中拉取 `nginx:latest` 镜像。然后，该命令会将此 NGINX 镜像作为 Docker 容器运行，将 `localhost:80` 映射到 NGINX 容器的端口 80。它还会将本地目录 `/path/to/content/` 作为容器卷以只读模式安装到 `/usr/share/nginx/html/`。默认 NGINX 配置将此目录作为静态内容提供。当指定从本地机器映射到容器时，则将本地机器上的端口或目录映射到容器上的端口或目录。

详解

NGINX 通过 Docker Hub 和 Amazon Elastic Container Registry 提供官方容器镜像。官方容器镜像支持您在 Docker 中快速启动并运行首选应用交付平台 NGINX。在本节中，我们在容器中仅使用一个命令就完成了 NGINX 的快速启动和运行！我们在此示例中使用的官方 NGINX 容器镜像基于 Debian 而构建。不过，您也可以选择基于 Alpine Linux 的官方镜像、由 mainline 与最新 commit 构建而来的镜像以及使用已安装 Perl 模块构建的镜像。这些官方镜像的 Dockerfile 和源代码都可以在 GitHub 上找到。您可以通过构建自己的 Dockerfile 并在 FROM 命令中指明所基于的官方镜像名称，扩展官方镜像。您还可以将 NGINX 配置目录作为 Docker 卷安装，以覆盖 NGINX 配置，而无需修改官方镜像。

官方 NGINX 容器镜像以 root 用户身份运行 NGINX。在需要非特权用户运行服务的平台（如 OpenShift）上运行时，您可以使用官方非特权镜像 `nginxinc/nginx-unprivileged`。

其他参考资料

官方 NGINX 容器镜像，[NGINX](#)

[NGINX 非特权容器镜像](#)

11.4 创建 NGINX Dockerfile

问题

您想要创建 NGINX Dockerfile，以创建容器镜像。

解决方案

F5 公司负责维护 Dockerfile 并通过 GitHub 对其进行分发。您可以基于这些 Dockerfile 构建自己的 Dockerfile。这些代码仓库中包含大量脚本，可帮助安装 NGINX。如欲查找 [GitHub 代码库](#)，请点击此处。

从首选发行版的容器镜像的 FROM 命令行开始。本例使用的是 Debian 12。确保您的软件包管理器已更新，并安装代码库和签名密钥安装所需的必备软件包，以便您的软件包管理器能够将官方 NGINX 代码库用于您的发行版。使用 COPY 命令添加 NGINX 配置文件。COPY 命令会将本地目录中的文件添加到容器镜像中；使用该命令将默认 NGINX 配置替换为您自己的配置。或者，使用 EXPOSE 命令指示 Docker 暴露给定端口，或者在将镜像作为容器运行时手动执行此操作。将镜像实例化成容器时，使用 CMD 启动 NGINX。需要在前台运行 NGINX。为此，需要使用 -g "daemon off;" 启动 NGINX，或将 daemon off; 添加到配置中。您还可以按需将 NGINX 配置更改为将访问日志记录到 /dev/stdout 并将错误日志记录到 /dev/stderr，从而将日志添加到 Docker 守护进程中，这支持您基于所选择的 Docker 日志驱动程序，更轻松地访问日志。NGINX 提供的 Dockerfile 如下所示：

```
FROM debian:bookworm-slim

LABEL maintainer="NGINX Docker Maintainers <docker-maint@nginx.com>"

ENV NGINX_VERSION 1.25.3
ENV NJS_VERSION 0.8.2
ENV PKG_RELEASE 1~bookworm

RUN set -x \
# 先创建 nginx 用户/组，
# 在 docker 变体之间保持一致
&& groupadd --system --gid 101 nginx \
&& useradd --system --gid nginx --no-create-home --home \
/nonexistent --comment "nginx user" --shell /bin/false \
--uid 101 nginx \
&& apt-get update \
&& apt-get install --no-install-recommends --no-install-suggests \
-y gnupg1 ca-certificates \
&& \
NGINX_GPGKEY=573BFD6B3D8FBC641079A6ABABF5BD827BD9BF62; \
NGINX_GPGKEY_PATH=/usr/share/keyrings/nginx-archive-keyring.gpg; \
export GNUPGHOME="$({mktemp -d})"; \
found=''; \
for server in \
    hkp://keyserver.ubuntu.com:80 \
```

```

pgp.mit.edu \
; do \
    echo "Fetching GPG key $NGINX_GPGKEY from $server"; \
    gpg1 --keyserver "$server" --keyserver-options timeout=10 \
    --recv-keys "$NGINX_GPGKEY" && found=yes && break; \
done; \
test -z "$found" && echo >&2 "error: failed to fetch GPG key \
$NGINX_GPGKEY" && exit 1; \
gpg1 --export "$NGINX_GPGKEY" > "$NGINX_GPGKEY_PATH" ; \
rm -rf "$GNUPGHOME"; \
apt-get remove --purge --auto-remove -y gnupg1 && \
rm -rf /var/lib/apt/lists/* \
&& dpkgArch="$(dpkg --print-architecture)" \
&& nginxPackages=" \
    nginx=${NGINX_VERSION}-${PKG_RELEASE} \
    nginx-module-xslt=${NGINX_VERSION}-${PKG_RELEASE} \
    nginx-module-geoip=${NGINX_VERSION}-${PKG_RELEASE} \
    nginx-module-image-filter=${NGINX_VERSION}-${PKG_RELEASE} \
    nginx-module-njs=${NGINX_VERSION}+${NJS_VERSION}-${PKG_RELEASE} \
" \
&& case "$dpkgArch" in \
    amd64|arm64) \
# 上游官方构建的架构
        echo "deb [signed-by=$NGINX_GPGKEY_PATH] \
        https://nginx.org/packages/mainline/debian/ bookworm nginx" >> \
        /etc/apt/sources.list.d/nginx.list \
        && apt-get update \
        ;; \
    *) \
# 所在架构并非上游官方构建
# 使用发布的源代码包构建二进制文件
        echo "deb-src [signed-by=$NGINX_GPGKEY_PATH] \
        https://nginx.org/packages/mainline/debian/ \
        bookworm nginx" >> /etc/apt/sources.list.d/nginx.list \
        \
# 用于存储源代码和 .deb 文件的新目录
        && tempDir="$(mktemp -d)" \
        && chmod 777 "$tempDir" \
# (777 确保 APT 的 "_apt" 用户也可以访问它)
        \
# 用于构建依赖项的已安装软件包的保存列表
# 日后可彻底移除
        && savedAptMark="$(apt-mark showmanual)" \
        \
# 使用上游源代码包构建 .deb 文件
# (经过 apt-get 验证)
        && apt-get update \
        && apt-get build-dep -y $nginxPackages \
        && ( \
            cd "$tempDir" \
            && DEB_BUILD_OPTIONS="nocheck parallel=$(nproc)" \
            apt-get source --compile $nginxPackages \

```

```

) \
# 此处不删除 APT 列表, 因为它们会被重新下载
# 并稍后删除
\
# 重置 apt-mark 的 "manual" 列表, 以便 "purge --auto-remove"
# 移除所有构建依赖项
# (在此之前, 安装构建软件包,
# 这样无需重新下载任何重叠依赖项)
    && apt-mark showmanual | xargs apt-mark auto > /dev/null \
    && { [ -z "$savedAptMark" ] || apt-mark manual $savedAptMark; } \
\
# 创建临时本地 APT 软件仓库来进行安装
# (这样一来, APT 就能按照应有的方式处理依赖项解析)
    && ls -lAFh "$tempDir" \
    && ( cd "$tempDir" && dpkg-scanpackages . > Packages ) \
    && grep '^Package: ' "$tempDir/Packages" \
    && echo "deb [ trusted=yes ] file://$tempDir ./" > \
        /etc/apt/sources.list.d/temp.list \
# 解决以下 APT 问题, 方法是使用
# "Acquire::GzipIndexes=false" (覆盖
# "/etc/apt/apt.conf.d/docker-gzip-indexes")
# 无法打开文件
# /var/lib/apt/lists/partial/_tmp_tmp.ODWljPQfkE_.Packages -
# open (13: 权限被拒绝)
# ...
# 错误: 无法提取
# store:/var/lib/apt/lists/partial/_tmp_tmp.ODWljPQfkE_.Packages
# 无法打开文件
# /var/lib/apt/lists/partial/_tmp_tmp.ODWljPQfkE_.Packages -
# open (13: 权限被拒绝)
    && apt-get -o Acquire::GzipIndexes=false update \
    ;; \
esac \
\
&& apt-get install --no-install-recommends --no-install-suggests -y \
    $nginxPackages \
    gettext-base \
    curl \
&& apt-get remove --purge --auto-remove -y &&
rm -rf /var/lib/apt/lists/* /etc/apt/sources.list.d/nginx.list \
\
# 如有构建残留项,
# 则清除 (包括额外的不必要构建依赖项)
    && if [ -n "$tempDir" ]; then \
        apt-get purge -y --auto-remove \
        && rm -rf "$tempDir" /etc/apt/sources.list.d/temp.list; \
    fi \
# 将请求和错误日志转发到 docker 日志收集器
    && ln -sf /dev/stdout /var/log/nginx/access.log \
    && ln -sf /dev/stderr /var/log/nginx/error.log \
# 创建 docker-entrypoint.d 目录
    && mkdir /docker-entrypoint.d

```

```
COPY docker-entrypoint.sh /
COPY 10-listen-on-ipv6-by-default.sh /docker-entrypoint.d
COPY 15-local-resolvers.envsh /docker-entrypoint.d
COPY 20-envsubst-on-templates.sh /docker-entrypoint.d
COPY 30-tune-worker-processes.sh /docker-entrypoint.d
ENTRYPOINT ["/docker-entrypoint.sh"]

EXPOSE 80

STOPSIGNAL SIGQUIT

CMD ["nginx", "-g", "daemon off;"]
```

详解

当需要完全控制所安装的软件包和更新时，创建自己的 Dockerfile 大有帮助。通常的做法是保留自己的镜像仓库，以确保基础镜像稳定可靠，并且经过团队测试后才投入到生产环境中使用。

其他参考资料

[GitHub 上的官方 NGINX Docker 代码库](#)

11.5 构建 NGINX Plus 容器镜像

问题

您想要构建 NGINX Plus 容器镜像，以在容器化环境中运行 NGINX Plus。

解决方案

F5 公司维护着一个[博客](#)，并不断更新有关通过 Dockerfile 安装 NGINX Plus 的信息。您可以基于这些 Dockerfile 构建自己的 Dockerfile。

使用上述博客中介绍的 Dockerfile 构建 NGINX Plus 容器镜像。具体步骤与[实操指南 11.4 构建 NGINX 开源版 Dockerfile](#) 中步骤的相同；但是，需要下载名为 *nginx-repo.crt* 和 *nginx-repo.key* 的 NGINX Plus 软件仓库证书，并将它们与此 Dockerfile 保存在同一目录中。随后，使用此 Dockerfile 完成 NGINX Plus 的其余安装工作以供使用。

以下 `docker build` 命令使用 `--no-cache` 标记来确保无论何时进行此项构建，都能重新从 NGINX Plus 软件仓库中拉取 NGINX Plus 软件包以应用更新。如果允许在 NGINX Plus 上使用与先前构建相同的版本，则可以省略 `--no-cache` 标记。在本示例中，新容器镜像被标记为 `nginxplus`：

```
$ docker build --no-cache -t nginx-plus .
```

详解

通过为 NGINX Plus 创建自己的容器镜像，您可以按需配置 NGINX Plus 容器，并将其部署到任何 Docker 环境中。这支持您在容器化环境中充分利用 NGINX Plus 的强大功能和高级特性。此 Dockerfile 使用 Dockerfile `COPY` 命令来添加配置；您需要将配置添加到本地目录中。

其他参考资料

[“使用 Docker 部署 NGINX 和 NGINX Plus”](#)

11.6 使用 NGINX 中的环境变量

问题

您想要使用 NGINX 配置内的环境变量，以便将同一容器镜像用于不同的环境。

解决方案

使用通过 NGINX Perl 模块（如 `nginx:stable-perl`）构建的一种官方 NGINX 容器镜像变体。

使用 `load_module` 指令启用 `nginx_http_perl_module`。默认情况下，NGINX 会删除从父进程继承的所有环境变量，`TZ` 变量除外。您可以使用 `env` 指令保留变量，但这些变量不会暴露给配置。该示例在 NGINX 进程中保留了一个名为 `APP_DNS` 的环境变量，并使用 Perl 模块中的 `perl_set` 指令设置了一个可用于配置的环境变量。暴露给 NGINX 配置的环境变量名为 `$upstream_app`。然后，`$upstream_app` 变量将用作代理请求的域：

```
load_module /modules/  
ngx_http_perl_module.so;  
env APP_DNS;  
# ...
```

```

http {
    perl_set $upstream_app 'sub { return $ENV{"APP_DNS"}; }';
    server {
        # ...
        location / {
            proxy_pass https://$upstream_app;
        }
    }
}

```

详解

一种典型的 Docker 用法是利用环境变量来改变容器的运行方式。您可以使用 NGINX 配置中的环境变量将 NGINX Dockerfile 用于多种不同的环境。

11.7 来自 NGINX 的 NGINX Ingress Controller (Ingress 控制器)

问题

您正在 Kubernetes 上部署应用，需要一个 Ingress controller (Ingress 控制器)。

解决方案

确保可以访问 Ingress controller 镜像。对于 NGINX 开源版，您可以使用来自 Docker Hub 的 *nginx/nginx-ingress* 镜像。而对于 NGINX Plus，则需要从 F5 公司容器镜像仓库中拉取，或者构建自己的镜像并将其托管在私有容器镜像仓库中。您可以在 [NGINX 文档](#) 中查找有关如何构建和推送 NGINX Plus Kubernetes Ingress Controller 的说明。

访问 [GitHub 上 kubernetes-ingress 代码库](#) 中的 Kubernetes Ingress Controller 部署文件夹。以下命令均从代码库本地副本所在的目录运行。

为 Ingress controller 创建命名空间和服务 (service) 帐户；两者均命名为 `nginx-ingress`：

```
$ kubectl apply -f common/ns-and-sa.yaml
```

或者，您可以创建一个用于自定义 NGINX 配置的配置映射（所提供的配置映射为空；更多信息，请参阅有关 [“ConfigMap”](#) 和 [“注释”](#) 自定义的 NGINX 文档）：

```
$ kubectl apply -f common/nginx-config.yaml
```

如果在集群中启用基于角色的访问控制（RBAC），则创建集群角色并将其绑定到服务帐户。您必须是集群管理员才能执行此步骤：

```
$ kubectl apply -f rbac/rbac.yaml
```

现在部署 Ingress controller。此代码库中提供了两个示例部署：Deployment 和 DaemonSet。如要动态更改 Ingress controller 副本的数量，则使用 Deployment。使用 Daemonset 在每个节点或节点子集上部署 Ingress controller。

如要使用 NGINX Plus Deployment 清单，则必须更改 YAML 文件并指定自己的镜像仓库和镜像。

对于 NGINX Deployment:

```
$ kubectl apply -f deployment/nginx-ingress.yaml
```

对于 NGINX Plus Deployment:

```
$ kubectl apply -f deployment/nginx-plus-ingress.yaml
```

对于 NGINX DaemonSet:

```
$ kubectl apply -f daemon-set/nginx-ingress.yaml
```

对于 NGINX Plus DaemonSet:

```
$ kubectl apply -f daemon-set/nginx-plus-ingress.yaml
```

验证 Ingress controller 正在运行:

```
$ kubectl get pods --namespace=nginx-ingress
```

如果您创建了一个 DaemonSet，Ingress controller 的端口 80 和 443 就会被映射到容器所在节点上的相同端口。如要访问 Ingress controller，则使用 Ingress controller 所在节点上的端口和 IP 地址。如果部署了 Deployment，则继续执行后续步骤。

对于 Deployment 方法，有两种 Ingress controller pod 访问方法。一种方法是指示 Kubernetes 随机分配映射到 Ingress Controller Pod 的节点端口。这是使用 NodePort 类型创建服务。另一方法是使用 LoadBalancer 类型创建服务。当创建 LoadBalancer 类型服务时，Kubernetes 会为给定云平台构建负载均衡器，例如 Amazon Web Services (AWS)、Microsoft Azure 和 Google Cloud Platform。

如要创建 NodePort 类型服务，则使用以下命令：

```
$ kubectl create -f service/nodeport.yaml
```

如要静态地配置面向 pod 打开的端口，则更改 YAML 并将 nodePort: {port} 属性添加到每个打开的端口。

如要为 Google Cloud Compute 或 Azure 创建 LoadBalancer 类型服务，则使用以下代码：

```
$ kubectl create -f service/loadbalancer.yaml
```

如要为 AWS 创建 LoadBalancer 类型服务，则使用以下代码：

```
$ kubectl create -f service/loadbalancer-aws-elb.yaml
```

在 AWS 上，Kubernetes 启用代理协议，在 TCP 模式下创建典型的 ELB。您必须将 NGINX 配置为使用代理协议。为此，您可以将以下内容添加到前面提到的与 *common/nginx-config.yaml* 文件相关的 config 映射中：

```
proxy-protocol: "True"
real-ip-header: "proxy_protocol"
set-real-ip-from: "0.0.0.0/0"
```

然后，更新配置映射：

```
$ kubectl apply -f common/nginx-config.yaml
```

现在，您可通过两种方式访问 pod：通过其 NodePort，或通过向代表其创建的负载均衡器发起请求。

详解

在撰写本文时，Kubernetes 是容器编排和管理领域的领先平台。Ingress controller 是将流量路由到应用其余部分的边缘 pod。NGINX 非常适合这个角色，并且可通过注释简化配置。NGINX Ingress 项目通过 Docker Hub 镜像提供开箱即用的 NGINX 开源版 Ingress Controller。对于 NGINX Plus，您只需简单几步即可使用 F5 容器镜像仓库添加仓库证书和密钥。通过在 Kubernetes 集群中安装 NGINX Ingress Controller，您不仅可以畅享 NGINX 的所有特性，还可以使用 Kubernetes 联网和 DNS 路由流量等新增特性。

高可用性部署模式

12.0 简介

容错架构将系统分成多个相同的独立堆栈。使用 NGINX 等负载均衡器来分发负载，以优化配置。高可用性的核心理念是在多个活跃节点上进行负载均衡或 active-passive 故障转移。高可用性应用不会出现单点故障；每个组件，包括负载均衡器本身，都必须应用上述其中一种理念。对我们来说，这就意味着对 NGINX 实现高可用。NGINX 经专门设计，可配置为 active-active 或 active-passive 故障转移模式。本章详细介绍了如何运行多台 NGINX 服务器以确保负载均衡层的高可用性。

12.1 NGINX Plus HA（高可用性）模式

问题

您在本地部署中需要高可用性负载均衡解决方案。

解决方案

在一个或多个系统上，从 NGINX Plus 软件仓库中安装 `nginx-ha-keepalived` 包，利用 `keepalived` 实现 NGINX Plus 的 HA 模式：

```
$ sudo apt update
$ sudo apt install -y nginx-ha-keepalived
```

在每个系统上使用 nginx-ha-setup 脚本引导 keepalived 配置文件，并按照提示操作：

```
$ sudo nginx-ha-setup
```

查看 nginx-ha-setup 命令生成的配置文件：

```
$ sudo cat /etc/keepalived/keepalived.conf
```

```
global_defs {
    vrrp_version 3
}

vrrp_script chk_manual_failover {
    script "/usr/lib/keepalived/nginx-ha-manual-failover"
    interval 10
    weight 50
}

vrrp_script chk_nginx_service {
    script "/usr/lib/keepalived/nginx-ha-check"
    interval 3
    weight 50
}

vrrp_instance VI_1 {
    interface eth0
    priority 101
    virtual_router_id 51
    advert_int 1
    accept
    garp_master_refresh 5
    garp_master_refresh_repeat 1
    unicast_src_ip 172.17.0.2/16
    unicast_peer {
        172.17.0.4
    }
    virtual_ipaddress {
        172.17.0.3
    }
    track_script {
        chk_nginx_service
        chk_manual_failover
    }
    notify "/usr/lib/keepalived/nginx-ha-notify"
}
```

测试已配置的健康检查脚本，确保节点处于健康状态：

```
$ sudo /usr/lib/keepalived/nginx-ha-check
```

```
nginx is running.
```

在辅助节点上，将 VRRP 扩展统计信息和数据转储到文件系统，并查看输出结果：

```
$ sudo service keepalived dump
```

```
Dumping VRRP stats (/tmp/keepalived.stats)
and data (/tmp/keepalived.data)
```

```
$ sudo cat /tmp/keepalived.stats
```

```
VRRP Instance: VI_1
Advertisements:
  Received: 1985
  Sent: 0
  Became master: 0
  Released master: 0
Packet Errors:
  Length: 0
  TTL: 0
  Invalid Type: 0
  Advertisement Interval: 0
  Address List: 0
Authentication Errors:
  Invalid Type: 0
  Type Mismatch: 0
  Failure: 0
Priority Zero:
  Received: 0
  Sent: 0
```

在主节点上进行强制状态变更：

```
$ sudo service keepalived stop
```

```
Stopping keepalived: keepalived.
```

同样，在辅助节点上，将 VRRP 扩展统计信息和数据转储到文件系统，并查看输出结果：

```
$ sudo service keepalived dump
```

```
Dumping VRRP stats (/tmp/keepalived.stats)
and data (/tmp/keepalived.data)
```

```
$ sudo cat /tmp/keepalived.stats
```

```
VRRP Instance: VI_1
Advertisements:
```

```
Received: 1993
Sent: 278
Became master: 1
Released master: 0
Packet Errors:
  Length: 0
  TTL: 0
  Invalid Type: 0
  Advertisement Interval: 0
  Address List: 0
Authentication Errors:
  Invalid Type: 0
  Type Mismatch: 0
  Failure: 0
Priority Zero:
  Received: 0
  Sent: 0
```

请注意，统计文件现在报告称，最初是辅助节点的节点至少变成过主节点一次。

详解

nginx-ha-keepalived 包基于 keepalived，管理暴露给客户端的虚拟 IP 地址。该解决方案适用于可通过标准操作系统调用来控制 IP 地址的环境，通常不适用于通过对接云基础架构来控制 IP 地址的云环境。

keepalived 进程使用虚拟路由器冗余协议（VRRP）将短消息（通常被称为心跳消息）发送到备份服务器。如果备份服务器连续三个周期接收不到心跳消息，备份服务器就会启动故障转移，将虚拟 IP 地址切换到自己这里并成为主服务器。nginx-ha-keepalived 的故障转移功能经配置，可识别自定义故障情形。nginx-ha-setup 脚本仅用于提供基本配置。通过编辑 keepalived 的配置文件并重启服务，您可以进一步自定义 keepalived。

其他参考资料

[keepalived 文档](#)

12.2 通过 DNS 实现负载均衡器的负载均衡

问题

您希望在两台或多台 NGINX 服务器之间分发负载。

解决方案

通过向 DNS A 记录中添加多个地址，使用 DNS 轮询 NGINX 服务器。

详解

运行多个负载均衡器时，您可通过 DNS 分发负载。A 记录允许在单个 FQDN 下列出多个 IP 地址。DNS 将自动轮询所有列出的 IP。DNS 还为加权记录提供加权轮询，其工作方式与第 2 章中所描述的 NGINX 的加权轮询工作方式相同。这些功能帮助极大。但是，当 NGINX 服务器处理请求遇到故障时，记录可能不会被删除，这是个问题。DNS 提供商（Amazon Route 53 和 DynDNS）通过其 DNS 产品提供健康检查和故障转移功能，可帮助缓解这些问题。如果使用 DNS 帮助在 NGINX 上进行负载均衡，那么当某台 NGINX 服务器被标记为删除时，最好在删除上游服务器时遵循 NGINX 所使用的协议。首先，通过从 DNS 记录中删除服务器的 IP 地址，停止向其发送新连接，然后在停止或关闭服务之前允许连接耗尽。

12.3 在 EC2 上实现负载均衡

问题

您正在 AWS 上使用 NGINX，而 NGINX Plus HA（高可用性）模式不支持 Amazon IP。

解决方案

配置 NGINX 服务器的 Auto Scaling 组并将 Auto Scaling 组链接到目标组，然后将目标组连接到 NLB，从而将 NGINX 置于 AWS NLB 之后（请参阅[实操指南 10.5](#)）。或者，您也可以使用 AWS 控制台、命令行接口或 API，手动将 NGINX 服务器添加到目标组中。

详解

由于 EC2 IP 地址以不同的方式使用，AWS 不支持浮动虚拟 IP 地址，所以基于 keepalived 的 NGINX Plus HA 解决方案将无法在 AWS 上使用。这并非意味着 NGINX 无法在 AWS 云中实现高可用性；实际上，事实正好相反。亚马逊产品 AWS NLB 可以在多个被称作“可用区”的在物理上分隔的数据中心之间进行本地负载均衡，提供主动健康检查，并提供 DNS CNAME 端点。在 AWS 上实现 NGINX 高可用性的一种常见解

决方案是将 NGINX 层置于 NLB 之后。NGINX 服务器可按需自动添加到目标组中或从目标组中删除。NLB 无法替代 NGINX；NGINX 可提供许多 NLB 无法提供的功能，例如多种负载均衡方法、速率限制、高速缓存和七层路由。尽管 AWS ALB 能够基于 URI 路径和主机请求头执行七层负载均衡，但是无法提供 WAF 高速缓存、带宽限制等 NGINX 可提供的功能。如果 NLB 无法满足需求，还有许多其他解决方案可供选择。其中一种选择是 DNS 解决方案：AWS Route 53 提供健康检查和 DNS 故障转移。

12.4 NGINX Plus 配置同步

问题

您正在运行 HA（高可用性）NGINX Plus 层，需要实现服务器配置同步。

解决方案

使用 NGINX Plus 独有的配置同步功能。该功能的配置步骤如下：

从 NGINX Plus 软件包软件仓库中安装 `nginx-sync` 包。

使用 YUM 软件包管理器的 NGINX Plus：

```
$ sudo yum install nginx-sync
```

使用 APT 软件包管理器的 NGINX Plus：

```
$ sudo apt install nginx-sync
```

授予主机 SSH 访问权限作为对等机 root 权限。

为 root 权限生成 SSH 身份验证密钥对，并检索公钥：

```
$ sudo ssh-keygen -t rsa -b 2048
$ sudo cat /root/.ssh/id_rsa.pub
ssh-rsa AAAAB3Nz4rFgt...vgaD root@node1
```

获取主节点的 IP 地址：

```
$ ip addr
1: lo: mtu 65536 qdisc noqueue state UNKNOWN group default
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
```

```
2: eth0: mtu 1500 qdisc pfifo_fast state UP group default qlen \
    1000
    link/ether 52:54:00:34:6c:35 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.2/24 brd 192.168.1.255 scope global eth0
        valid_lft forever preferred_lft forever
    inet6 fe80::5054:ff:fe34:6c35/64 scope link
        valid_lft forever preferred_lft forever
```

`ip addr` 命令将转储机器接口信息。跳过环回接口，它通常显示在最前面。查找主接口 `inet` 后面的 IP 地址。在本示例中，IP 地址是 192.168.1.2。

将公钥分发到 `root` 用户在每个对等节点上的 `authorized_keys` 文件，并指定仅从主 IP 地址授权：

```
$ sudo echo 'from="192.168.1.2" ssh-rsa AAAAB3Nz4rFgt...vgaD \
    root@node1' >> /root/.ssh/authorized_keys
```

将以下命令行添加到 `/etc/ssh/sshd_config` 中并在所有节点上重载 `sshd`：

```
$ sudo echo 'PermitRootLogin without-password' >> \
    /etc/ssh/sshd_config
$ sudo service sshd reload
```

验证主节点上的 `root` 用户能否对每个对等节点进行 `ssh` 免密登录：

```
$ sudo ssh root@node2.example.com
```

在主机上使用以下配置创建配置文件 `/etc/nginx-sync.conf`：

```
NODES="node2.example.com node3.example.com node4.example.com"
CONFPATHS="/etc/nginx/nginx.conf /etc/nginx/conf.d"
EXCLUDE="default.conf"
```

本示例配置演示了该功能的三个常用配置参数：`NODES`、`CONFPATHS` 和 `EXCLUDE`。`NODES` 参数被设置为用空格隔开的主机名或 IP 地址的字符串；主节点会将其配置更改推送给对等节点。`CONFPATHS` 参数表示应同步的文件或目录。最后，您可以使用 `EXCLUDE` 参数将配置文件从同步进程中删除。在我们的示例中，主节点向名为 `node2.example.com`、`node3.example.com` 和 `node4.example.com` 的对等节点推送主要 `NGINX` 配置文件的配置更改并添加目录 `/etc/nginx/nginx.conf` 和 `/etc/nginx/conf.d`。如果同步进程发现名为 `default.conf` 的文件，由于该文件已被配置为 `EXCLUDE`，因此不会被推送给对等节点。

高级配置参数可以配置 NGINX 二进制文件、RSYNC 二进制文件、SSH 二进制文件、diff 二进制文件、lockfile 和备份目录的位置。还有一个参数使用 sed 为给定文件提供模板。如欲了解有关高级参数的更多信息，请参阅“[NGINX 集群配置同步](#)”文档。

对配置进行测试：

```
$ nginx-sync.sh -h # display usage info
$ nginx-sync.sh -c node2.example.com # compare config to node2
$ nginx-sync.sh -C # compare primary config to all peers
$ nginx-sync.sh # sync the config & reload NGINX on peers
```

详解

这项 NGINX Plus 独有功能可帮助高效管理 HA 配置中的多台 NGINX Plus 服务器，您只需更新主节点，将配置同步到所有其他对等节点即可。配置同步的自动化可降低在配置同步过程中发生错误的风险。为避免将不良配置发送给对等节点，nginx-sync.sh 应用提供了多项保障措施，包括测试主节点的配置，在对等节点上创建配置备份，以及在重载之前验证对等节点的配置。尽管使用配置管理工具或 Docker 对配置进行同步是更好的选择，但如果您尚不能通过这些方式管理环境，NGINX Plus 配置同步特性将会是不错的选择。

12.5 与 NGINX Plus 的状态共享和区域同步

问题

您希望 NGINX Plus 在大量高可用性服务器之间同步其共享内存区。

解决方案

配置区域同步，然后使用 sync 参数配置 NGINX Plus 共享内存区：

```
stream {
    resolver 10.0.0.2 valid=20s;

    server {
        listen 9000;
        zone_sync;
        zone_sync_server nginx-cluster.example.com:9000 resolve;
        # ... 安全措施
    }
}
```

```

http {
    upstream my_backend {
        zone my_backend 64k;
        server backends.example.com resolve;
        sticky learn zone=sessions:1m
            create=$upstream_cookie_session
            lookup=$cookie_session
            sync;
    }

    server {
        listen 80;
        location / {
            proxy_pass http://my_backend;
        }
    }
}

```

详解

zone_sync 模块是 NGINX Plus 的一项独有功能，能够让 NGINX Plus 真正实现集群。如上述配置所示，必须将一台 stream 服务器设为配置为 zone_sync。在本示例中，我们选用了侦听端口 9000 的服务器。NGINX Plus 与 zone_sync_server 指令定义的其他服务器通信。您可以为此指令设置一个域名，使域名解析到多个 IP 地址以构成动态集群，或者静态定义一系列 zone_sync_server 指令以避免单点故障。应限制对区域同步服务器的访问；您可以为 zone_sync 模块设置特定 SSL/TLS 指令以进行机器身份验证。

将 NGINX Plus 配置为集群的好处是，您可以对共享内存区进行速率限制、粘性学习（sticky-learn）会话和键值（key-value）存储等方面的同步。在所提供的示例中，sticky learn 指令在末端添加了 sync 参数。在本示例中，用户基于名为 session 的 cookie 绑定到上游服务器。在没有 zone_sync 模块的情况下，如果用户向不同的 NGINX Plus 服务器发出请求，则可能会丢失会话。如果存在 zone_sync 模块，所有 NGINX Plus 服务器都将能够感知到会话及其绑定到的上游服务器。

高级活动监控

13.0 简介

为了确保应用以最佳性能和精度运行，您需要清晰地了解有关其活动的监控指标。NGINX 提供了多种监控选项，例如 stub 状态或高级监控仪表盘以及 NGINX Plus 中的 JSON feed。NGINX Plus 活动监控提供有关请求、上游服务器池、高速缓存、健康状况等的洞察。通过使用 OpenTelemetry，还可进一步实现应用集成可观测性。本章详细介绍了使用 NGINX 进行监控的功能及其所带来的可能性。

13.1 启用 NGINX stub 状态

问题

您想要对 NGINX 进行基本的监控。

解决方案

启用 NGINX HTTP 服务器内 location 代码块中的 stub_status 模块：

```
location /stub_status {
    stub_status;
    allow 127.0.0.1;
    deny all;
    # 视情况设置 IP 限制
}
```

通过向 stub 状态发送请求来测试配置：

```
$ curl localhost/stub_status
Active connections: 1
server accepts handled requests
1 1 1
Reading: 0 Writing: 1 Waiting: 0
```

详解

stub_status 模块支持对 NGINX 服务器进行一些基本的监控。返回的信息包括活跃连接数、已接受的连接总数、已处理的连接数和服务请求数。此外，还显示了当前正处于读入、写入或等待状态的连接数。所提供的信息为全局信息，并非特定于定义 stub_status 指令的父 server。这意味着您可以将 stub 状态托管在受保护的 server 上。出于安全原因，除本地流量以外，我们拦截了对监控功能的所有访问。该模块将活跃连接数作为嵌入式变量提供，以用于日志和其他地方。这些变量包括 \$connections_active、\$connections_reading、\$connections_writing 以及 \$connections_waiting。

13.2 启用 NGINX Plus 监控仪表盘

问题

您想要有关流经 NGINX Plus 服务器的流量的深度指标。

解决方案

使用实时活动监控仪表盘：

```
server {
    # ...
    location /api {
        api write=on;
        # 限制访问 API 的指令
        # 见第 7 章
    }

    location = /dashboard.html {
        root /usr/share/nginx/html;
    }
}
```

NGINX Plus 配置提供 NGINX Plus 状态监控仪表盘。此配置设置了 HTTP 服务器，以提供 API 和状态仪表盘。状态仪表盘从 `/usr/share/nginx/html` 目录中作为静态内容提供。该仪表盘会向位于 `/api/` 的 API 发起请求，以实时检索和显示状态。

详解

NGINX Plus 提供高级状态监控仪表盘。此状态仪表盘提供有关 NGINX 系统状态的详细信息，例如活跃连接数、正常运行时间、上游服务器池信息等。有关该控制台的图示，请参见图 13-1。

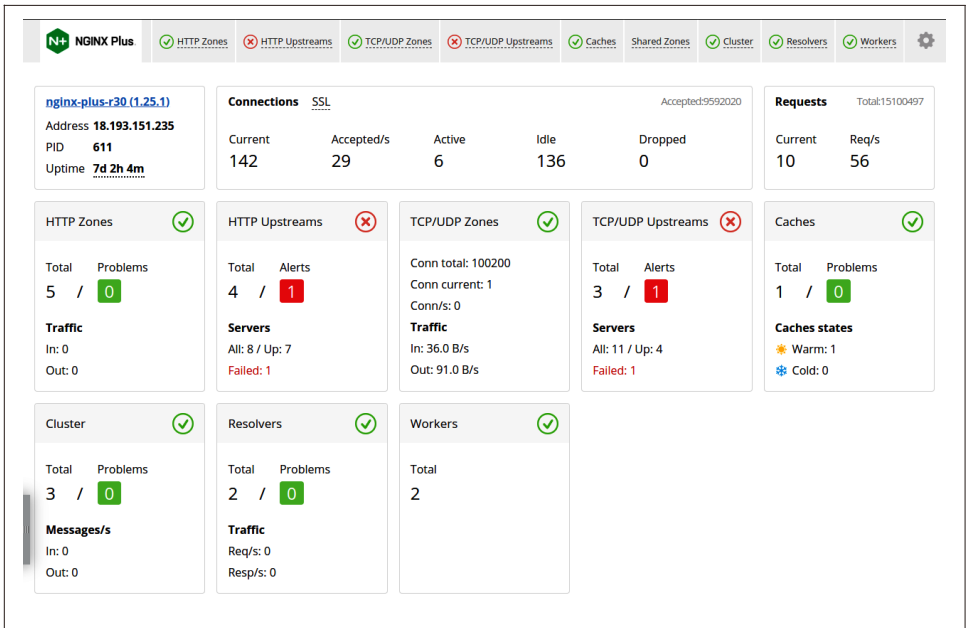


图 13-1. NGINX Plus 状态仪表盘

该状态仪表盘的登录界面提供了对整个系统的概览。点击 HTTP Zones (HTTP 区域) 选项卡，即可获得有关 NGINX 配置中所有 HTTP 服务器的详细信息，包括 1xx~5xx 响应数和总响应数，以及每秒请求数和当前吞吐量（参见图 13-2）。HTTP Upstreams (HTTP 上游) 选项卡显示了有关上游服务器状态的详细信息，包括服务器故障期间服务的请求数、状态代码处理的响应数，以及其他统计信息，例如健康检查通过或失败次数。TCP/UDP Zones (TCP/UDP 区域) 选项卡详细显示了流经 TCP 或 UDP 串流的流量和连接数。TCP/UDP Upstreams (TCP/UDP 上游) 选项卡显示了 TCP/UDP 上游池

中每台上游服务器所服务的请求数，以及健康检查通过和失败的详细信息和响应时间。Caches（高速缓存）选项卡显示了有关高速缓存占用空间、已服务、写入和绕过流量以及命中率的信息。NGINX 状态仪表盘在监控应用和流量的关键指标方面发挥着重要作用。

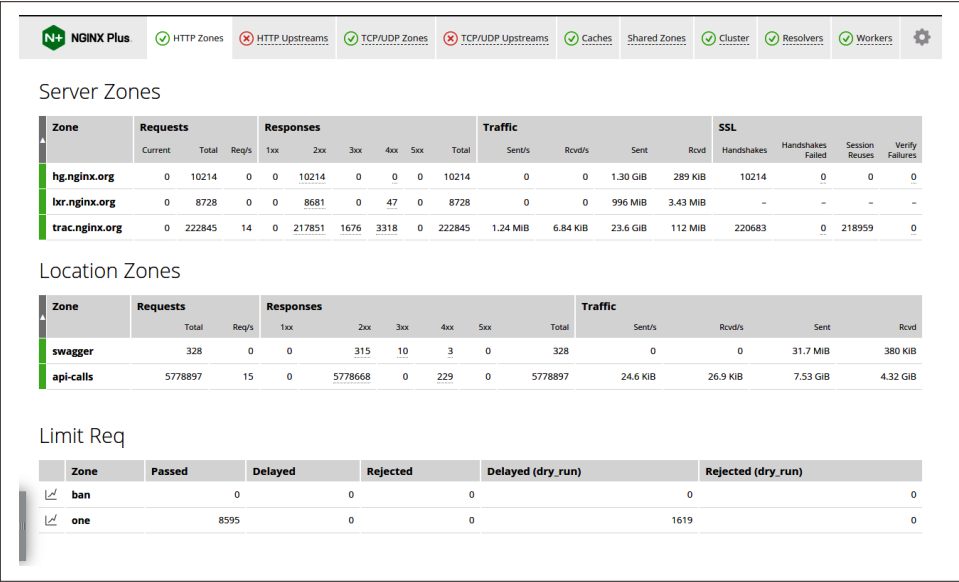


图 13-2. NGINX Plus 状态仪表盘 HTTP 区域页面

其他参考资料

NGINX Plus 状态仪表盘演示

13.3 使用 NGINX Plus API 收集指标

问题

您想要通过 API 访问 NGINX Plus 状态仪表盘提供的详细指标。

解决方案

利用 RESTful API 收集指标。这些示例通过 json_jp 导出输出结果，以便于读取：

```
$ curl "https://demo.nginx.com/api/9/" | json jq
[
  "nginx",
  "processes",
  "connections",
  "slabs",
  "http",
  "stream",
  "resolvers",
  "ssl",
  "workers"
]
```

curl 调用向 API 顶层发起请求，以显示 API 的其他部分。

如要获取有关 NGINX Plus 服务器的信息，则使用 `/api/{version}/nginx` URI：

```
$ curl "https://demo.nginx.com/api/9/nginx" | json jq
{
  "address" : "10.3.0.7",
  "build" : "nginx-plus-r30",
  "generation" : 1,
  "load_timestamp" : "2023-08-15T11:38:35.603Z",
  "pid" : 622,
  "ppid" : 621,
  "timestamp" : "2023-08-22T20:49:21.717Z",
  "version" : "1.25.1"
}
```

如要限制 API 返回的信息，则使用下列参数：

```
$ curl "https://demo.nginx.com/api/9/nginx?fields=version,build" \
| json jq
{
  "build" : "nginx-plus-r30",
  "version" : "1.25.1"
}
```

您可以从 `/api/{version}/connections` URI 获取请求连接数统计：

```
$ curl "https://demo.nginx.com/api/9/connections" | json jq
{
  "accepted" : 9884385,
  "active" : 4,
  "dropped" : 0,
  "idle" : 127
}
```

您可以从 `/api/{version}/http/requests` URI 收集请求数：

```
$ curl "https://demo.nginx.com/api/9/http/requests" | json jq
{
  "total" : 52107833,
  "current" : 2
}
```

您可以使用 `/api/{version}/http/server_zones/{httpServerZoneName}` URI 检索有关特定服务器区域的统计数据：

```
$ curl "https://demo.nginx.com/api/9/http/server_zones/hg.nginx.org" \
| json jq
{
  "discarded" : 0,
  "processing" : 0,
  "received" : 308357,
  "requests" : 10633,
  "responses" : {
    "1xx" : 0,
    "2xx" : 10633,
    "3xx" : 0,
    "4xx" : 0,
    "5xx" : 0,
    "codes" : {
      "200" : 10633
    },
    "total" : 10633
  },
  "sent" : 1327232700,
  "ssl" : {
    "handshake_timeout" : 0,
    "handshakes" : 10633,
    "handshakes_failed" : 0,
    "no_common_cipher" : 0,
    "no_common_protocol" : 0,
    "peer_rejected_cert" : 0,
    "session_reuses" : 0,
    "verify_failures" : {
      "expired_cert" : 0,
      "no_cert" : 0,
      "other" : 0,
      "revoked_cert" : 0
    }
  }
}
```

仪表盘可清晰地显示 API 返回的所有数据。这些数据富有深度，按逻辑有序呈现。您可以在本节实操指南的结尾找到有关资源的链接。

详解

NGINX Plus API 可返回有关 NGINX Plus 服务器的多方面统计信息。您可以收集有关 NGINX Plus 服务器及其进程、连接和 slab 的信息。还可以查看有关 NGINX 内 http 和 stream 服务器的信息，包括服务器、上游、上游服务器和键值存储，以及有关 HTTP 缓存区的信息和统计数据。这能够为您或第三方指标聚合器提供有关 NGINX Plus 服务器状态的深入视图。

其他参考资料

[“NGINX HTTP API 模块文档”](#)

[NGINX API REST UI](#)

[“指标 API 使用” 教程](#)

13.4 在 NGINX 中使用 OpenTelemetry

问题

您希望支持 OpenTelemetry (OTel) 的链路追踪收集器集成 NGINX。

解决方案

从 NGINX 软件包仓库安装 NGINX OTel 模块。

使用 YUM 软件包管理器的 NGINX Plus:

```
$ sudo yum install -y nginx-plus-module-otel
```

使用 APT 软件包管理器的 NGINX Plus:

```
$ sudo apt install -y nginx-plus-module-otel
```

使用 YUM 软件包管理器的 NGINX 开源版:

```
$ sudo yum install -y nginx-module-otel
```

使用 APT 软件包管理器的 NGINX 开源版:

```
$ sudo apt install -y nginx-module-otel
```

加载 NGINX OTel 动态模块并配置支持 OTel 的接收器:

```
load_module modules/nginx_otel_module.so;
# ...
http {

    otel_exporter {
        endpoint localhost:4317;
    }

    server {
        listen 127.0.0.1:8080;

        location / {
            otel_trace on;
            otel_trace_context inject;

            proxy_pass http://backend;
        }
    }
}
```

NGINX 从传入请求继承链路追踪上下文, 记录 span, 并通过将 otel_trace_context 指令设置为 propagate, 将链路追踪传播到后端服务器:

```
server {
    location / {
        otel_trace on;
        otel_trace_context propagate;
        proxy_pass http://backend;
    }
}
```

您可以从传入请求继承链路追踪上下文, 并只记录 10% 流量的 span, 方法是只启用 otel_trace 与 split_clients:

```
# 链路追踪 10% 的请求
split_clients "$otel_trace_id" $ratio_sampler {
    10% on;
    * off;
}

server {
    location / {
        otel_trace $ratio_sampler;
        otel_trace_context propagate;

        proxy_pass http://backend;
    }
}
```

在某些情况下，例如调试问题时，您可能需要确保对每个请求进行链路追踪。NGINX 的 `map` 函数可用于创建 OR 逻辑门。在下面的示例更新中，我们将使用 `map` 函数为变量赋值 `on` 或 `off`。`map` 将基于名为 `ENABLE_OTEL_TRACE` 的 HTTP 请求头。如果请求基于 10% `split_clients` 进行了分流，或者带有 `ENABLE_OTEL_TRACE` 请求头，则使用另一个 `map` 生成一个变量，其值等于 `on`：

```
# 链路追踪 10% 的请求
split_clients "$otel_trace_id" $ratio_sampler {
    10% on;
    * off;
}

# 当 ENABLE_OTEL_TRACE 请求头通过值
map "$http_enable_otel_trace" $has_trace_header {
    default on;
    '' off;
}

# 或语句开启 otel_trace 时始终进行链路追踪
# 但前提是上述用例支持 otel_trace
map "$ratio_sampler:$has_trace_header" $request_otel {
    off:off off;
    on:on on;
    on:off on;
    off:on on;
}

server {
    location / {
        otel_trace $request_otel;
        otel_trace_context propagate;

        proxy_pass http://backend;
    }
}
```

或者，如果您使用的是 NGINX Plus，则可以利用 NGINX Plus API 和键值存储启用或禁用某些会话的 100% 链路追踪：

```
# 通过键值存储开启动态 100% 链路追踪
keyval "otel.trace" $trace_switch zone=otel_kv;

# 链路追踪 10% 的请求
split_clients "$otel_trace_id" $ratio_sampler {
    10% on;
    * off;
}

# 或者通过语句开启 otel_trace
```

```
# 但前提是上述用例支持 otel_trace
map "$trace_switch:$ratio_sampler" $request_otel {
    off:off    off;
    on:on      on;
    on:off     on;
    off:on     on;
}

server {
    location / {
        otel_trace $request_otel;
        otel_trace_context propagate;

        proxy_pass http://backend;
    }
    location /api {
        api write=on;
    }
}
```

详解

OpenTelemetry 提供了一套 SDK、API 和工具，用于检测应用以生成和收集遥测数据。这些数据被流式传输到 Jaeger 和 Prometheus 等系统，以便工程师通过应用栈链路追踪调用，从而确保精确的可观测性。NGINX OTel 模块与这些遥测系统集成，便于收集请求数据、设置请求的 `tracestate` 和 `traceparent` 请求头，然后将这些数据发送到 OpenTelemetry 接收器。

在本节中，我们动态加载了 NGINX `ngx_otel_module`，并提供了一个接收器来接收发送的遥测数据。`otel_trace_context` 指令可配置为 `extract`（提取）、`inject`（拒绝）、`propagate`（传播）或 `ignore`（忽略）`traceparent` 和 `tracestate` 请求头，在此处被配置为 `propagate`。`propagate` 参数结合使用 `extract` 和 `inject` 方法，其中 `extract` 从 HTTP 请求头中提取 `tracestate` 和 `traceparent`，而 `inject` 则为上游请求创建和设置新值。使用 `propagate` 时，仅当 HTTP 请求头尚未设置时，才创建新的 ID。

在繁忙的系统中，记录所有请求的链路追踪数据可能有点困难。该示例显示了如何使用 `split_clients` 模块仅发送 10% 请求的链路追踪数据。它经进一步改进后，允许开发人员通过设置请求头来启用链路追踪，从而确保对其请求进行链路追踪。这些选项与 `map` 模块相结合来创建 OR 逻辑门，仅当两者都返回 `off` 时，链路追踪才会设置为 `off`。

其他参考资料

[NGINX Plus OTel 模块文档](#)

[隆重推出 NGINX Plus R29 \(支持 OpenTelemetry\)](#)

[OpenTelemetry 文档](#)

[GitHub 上的 NGINX OpenTelemetry 项目](#)

[Split Clients Module 文档](#)

[NGINX Plus API 模块文档](#)

[NGINX Plus 键值模块文档](#)

[NGINX 开源版 OpenTelemetry 模块](#)

13.5 Prometheus Exporter 模块

问题

您正在使用 Prometheus 监控将 NGINX 部署到环境中，需要 NGINX 统计数据。

解决方案

使用 NGINX Prometheus Exporter 获取 NGINX 统计数据，并将这些数据发送给 Prometheus。

NGINX Prometheus Exporter 模块使用 GoLang 编写而成，在 [GitHub](#) 上作为二进制文件分发，也在 [Docker Hub](#) 上作为预构建容器镜像提供。

默认情况下，为 NGINX 启动该 exporter 程序，并且该 exporter 程序只收集 stub_status 信息。如要为 NGINX 开源版运行该 exporter 程序，首先需确保启用 stub 状态（如未启用，请参阅[实操指南 13.1](#)，详细了解如何启用）。然后运行以下 Docker 命令：

```
$ docker run -p 9113:9113 nginx/nginx-prometheus-exporter:0.8.0 \
  -nginx.scrape-uri http://{nginxEndpoint}:8080/stub_status
```

如要将该 exporter 程序与 NGINX Plus 一起使用，以利用 NGINX Plus API 收集更多数据，则必须使用标记来切换该 exporter 程序的上下文。您可参阅[实操指南 13.2](#)，了解如何打开 NGINX Plus API。使用以下 Docker 命令为 NGINX Plus 环境运行该 exporter 程序：

```
$ docker run -p 9113:9113 nginx/nginx-prometheus-exporter:0.8.0 \
  -nginx.plus -nginx.scrape-uri http://{nginxPlusEndpoint}:8080/api
```

详解

Prometheus 是一种极为常见的指标监控解决方案，在 Kubernetes 生态系统中应用广泛。NGINX Prometheus Exporter 模块虽然是一个极其简单的组件，但支持在 NGINX 和常用监控平台之间进行预构建集成。使用 NGINX 时，`stub_status` 提供的数据虽不多，但很重要，可帮助您了解 NGINX 节点当前的工作量。利用 NGINX Plus API 可以收集更多有关 NGINX Plus 服务器的统计数据，`exporter` 程序会将所有这些数据都发送给 Prometheus。无论哪种情况，收集的信息都可用作宝贵的监控数据，而且都会被发送给 Prometheus。您只需接通电源，坐享 NGINX 统计数据提供的洞察即可。

其他参考资料

[NGINX Prometheus Exporter GitHub](#)

[NGINX `stub\_status` 模块文档](#)

[NGINX Plus API 模块文档](#)

[NGINX Plus 实时活动监控](#)

利用访问日志、错误日志和请求跟踪进行调试和故障排除

14.0 简介

日志记录是了解应用的基础。使用 NGINX 您可有效控制对您和您的应用有意义的日志信息。NGINX 允许您根据不同的上下文将日志以不同的格式拆分到不同的文件中，并更改错误日志的日志级别，以更深入地了解当前状况。NGINX 支持对接 Syslog，天生就具备了将日志以流的形式输出给集中式日志服务器的能力。NGINX 还支持为请求发起全链路跟踪。在本章中，我们将介绍访问和错误日志，基于 Syslog 协议的流式传输，以及利用 NGINX 所生成的请求标识符的实现端到端请求跟踪。

14.1 配置访问日志

问题

您想要配置访问日志格式，以将内置变量添加到请求日志中。

解决方案

配置访问日志格式：

```
http {  
    log_format  geoproxy
```

```

        '[time_local] $remote_addr '
        '$realip_remote_addr $remote_user '
        '$proxy_protocol_server_addr $proxy_protocol_server_port '
        '$request_method $server_protocol '
        '$scheme $server_name $uri $status '
        '$request_time $body_bytes_sent '
        '$geoip_city_country_code3 $geoip_region '
        '"$geoip_city" $http_x_forwarded_for '
        '$upstream_status $upstream_response_time '
        '$http_referer" "$http_user_agent";

    # ...
}

```

该日志格式配置被命名为 `geoproxy`，使用了大量内置变量来演示 NGINX 日志记录功能。该配置可输出发起请求时服务器上的本地时间、打开连接的 IP 地址，以及客户端的 IP 地址，因为 NGINX 根据 `geoip_proxy` 或 `realip_header` 指令知道这些信息。NGINX 默认不包含 GeoIP 模块；因此，NGINX 需要在配置该模块后使用源代码进行构建。请参阅[实操指南 3.2](#)，了解如何安装 GeoIP 模块。

当 `server` 下的 `listen` 指令使用了 `proxy_protocol` 参数时，带 `$proxy_protocol_server_` 前缀的变量可用于从 PROXY 协议头中获取服务端相关信息。`$remote_user` 表示通过 Basic 认证的用户名；后面还包含请求方法、协议版本和协议类型，例如 HTTP 或 HTTPS。当然也记录了匹配的 `server` 上下文名称、请求 URI 和响应状态码。

记录的统计信息包括以毫秒为单位的处理时间以及发送给客户端的消息体大小。此外，有关国家、地区和城市的信息也被记录在内。X-Forwarded-For HTTP 请求头可表明请求是否来自正被其它代理服务器转发。`upstream` 模块支持一些用于显示上游服务器响应状态和上游请求响应时间的内置变量。最后，有关客户端请求来源和客户端所用浏览器的信息也被记录在内。

可选参数 `escape` 可指定字符串转义类型，可选值有：`default`、`json` 和 `none`。`none` 表示禁用转义字符。`default` 时，字符 `"`、`\` 以及其他值小于 32 或大于 126 的字符用 `\xxx` 来转义。如果找不到变量值，则使用连字符 (`-`) 替代。对于 `json` 转义，JSON 字符串中所有不允许的字符都将被转义，如：字符 `"` 和 `\` 用 `\"` 和 `\\` 来表示，值小于 32 的字符用 `\n`、`\r`、`\t`、`\b`、`\f` 或 `\u00XX` 来表示。

此日志配置的日志条目如下所示：

```
[25/Nov/2016:16:20:42 +0000] 10.0.1.16 192.168.0.122 Derek  
GET HTTP/1.1 http www.example.com / 200 0.001 370 USA MI  
"Ann Arbor" - 200 0.001 "-" "curl/7.47.0"
```

如要使用此日志格式，则使用 `access_log` 指令，并将日志文件路径和格式名称 `geoproxy` 用作参数，其中后者是指本解决方案第一步中创建的日志格式名称：

```
server {  
    access_log /var/log/nginx/access.log geoproxy;  
    # ...  
}
```

`access_log` 指令使用日志文件路径和格式名称作为参数。该指令在许多上下文中都有效，可根据上下文采用不同的日志路径和/或日志格式。`buffer`、`flush` 和 `gzip` 等参数可配置日志写入日志文件的频率以及文件是否被压缩。为了不与 `if` 代码块混淆，还可在 `access_log` 指令中利用 `if` 参数设置输出条件，如果条件值为 0 或空字符串，则访问将不被记录。

详解

NGINX 中的 `log` 模块允许您为许多不同的场景配置日志格式，以根据需要记录到不同日志文件中。在实际运用中，为不同上下文配置不同的日志格式非常有用，您可以在日志中使用不同的模块和内置变量，或者集中提供所有必要信息。日志也能记录以 JSON 格式和 XML 格式创建的字符串。这些日志将帮助您了解流量模式、客户端使用情况和客户端来源等。此外，访问日志还可以帮助您发现与上游服务器或特定 URI 相关的响应延迟和问题。访问日志还可用于在测试环境中解析和回放流量模式，以模仿真实的用户交互。日志在应用故障排除、调试或市场分析中发挥着至关重要的作用。

14.2 配置错误日志

问题

您想要配置错误日志，以更深入地了解 NGINX 服务器问题。

解决方案

使用 `error_log` 指令定义日志路径和日志级别：

```
error_log /var/log/nginx/error.log warn;
```

在配置 `error_log` 指令时，路径是必选的，但日志级别是可选的，默认为 `error`。除 `if` 语句以外，该指令在所有上下文中都有效。日志级别包括 `debug`（调试）、`info`（信息）、`notice`（注意）、`warn`（警告）、`error`（错误）、`crit`（严重）、`alert`（警报）和 `emerg`（紧急）。上述日志级别的排列顺序也是严重程度从低到高的顺序。`debug` 日志级别仅在 NGINX 配置了 `--with-debug` 标记时可用。

详解

当配置文件无法正常工作，您应首先查看错误日志。错误日志也是定位由 FastCGI 等应用服务器所产生的错误的利器。您还可以使用错误日志来调试 worker 进程连接、内存分配、客户端 IP 和服务器。错误日志虽然不支持自定义日志格式，但也遵循特定格式——先列出日期，随后列出日志级别和具体消息。

14.3 转发日志到 Syslog

问题

您想要将日志转发到 Syslog 侦听器，以便将日志聚合到集中式日志服务中。

解决方案

使用 `error_log` 和 `access_log` 指令将日志发送到 Syslog 侦听器：

```
error_log syslog:server=10.0.1.42 info;
```

```
access_log syslog:server=10.0.1.42,tag=nginx,severity=info geoproxy;
```

`error_log` 和 `access_log` 指令的 `syslog` 参数后面紧跟冒号和一些参数选项。这些选项包括必要的日志服务标识，如：需要连接的 IP 地址、DNS 名称或 Unix socket 的必选服务器标记，以及 `facility`、`severity`、`tag` 和 `nohostname` 等可选标记。`server` 选项在指定 IP 地址或 DNS 名称时设置端口号，默认为 UDP 514。`facility` 选项是指日志消息的 `facility` 值，取 Syslog RFC 标准定义的 23 个值之一；默认值为 `local7`。`tag` 选项表示消息的标记，默认值为 `nginx`。`severity` 选项默认为 `info`，表示所发送

消息的日志级别。nohostname 标记禁止将 hostname 字段添加到 syslog 消息头中，不取值。

详解

Syslog 是一种在单台服务器或服务器集群上发送和收集日志消息的标准协议。在多个主机上运行相同服务的多个实例时，将日志发送到集中位置有助于调试。这被称作聚合日志。聚合日志允许您在一个地方查看日志，无需切换服务器，并通过时间戳将日志文件合并在一起。常见的日志聚合堆栈有 Elasticsearch、Logstash 和 Kibana，统称为 ELK 堆栈。NGINX 可使用 access_log 和 error_log 指令，轻松将这些日志传输到 Syslog 侦听器。

14.4 调试配置

问题

您从 NGINX 服务器收到预料之外的响应。

解决方案

调试您的配置，并谨记以下提示：

- NGINX 通过查找特定匹配规则来处理请求。这虽然增加了手动配置的难度，但却是运行 NGINX 的最有效方法。有关 NGINX 如何处理请求的更多信息，请访问“其他参考资料”部分中的文档链接。
- 您可以打开调试日志记录。对于调试日志记录，需确保 NGINX 软件包带有 --with-debug 标记。大多数常用软件包都带有此标记，但如果是自行构建的软件包或最简化软件包，则您至少要运行 nginx-V 确认一下。确保开启调试后，您可将 error_log 指令的日志级别设置为 debug:error_log/var/log/nginx/error.log debug。
- 您可以开启对特定连接的调试。debug_connection 指令在 events 上下文中有效，并将 IP 或 CIDR 范围用作参数。该指令可以被多次声明，以添加多个 IP 地址或 CIDR 范围进行调试。这可帮助您调试生产问题，并避免因调试所有连接而降低性能。

- 您可以只对特定虚拟服务器进行调试。error_log 指令在主 http、mail、stream、server 和 location 上下文中都有效，您可以只在需要的上下文中设置 debug 日志级别。
- 您可以启用内核转储并从中获取回溯。您可通过操作系统或 NGINX 配置文件启用内核转储。更多相关信息请参阅“其他参考资料”部分中的管理员指南。
- 您可以开启 rewrite_log 指令：rewrite_log on 记录重写语句的执行过程。

详解

NGINX 工具规模庞大，配有诸多强大功能。但是，这也有可能带来极大不便。调试时，请您务必要弄清如何通过配置跟踪请求；如果遇到问题，您可添加调试日志级别以寻求帮助。调试日志虽繁杂冗长，但它可清晰地显示 NGINX 正如何处理请求，以及配置中存在问题的地方。

其他参考资料

NGINX 请求处理

《NGINX 调试管理员指南》

NGINX rewrite_log 模块文档

14.5 请求跟踪

问题

您想要将 NGINX 日志与应用日志相关联，以了解请求的端到端处理过程。

解决方案

使用 request 识别变量，并将其传递给应用以进行记录：

```
log_format trace '$remote_addr - $remote_user [$time_local] '
                '$request' $status $body_bytes_sent '
                '$http_referer' "$http_user_agent" '
                '$http_x_forwarded_for' $request_id';

upstream backend {
    server 10.0.0.42;
}

server {
    listen 80;
```

```

# 将 X-Request-ID 请求头添加到对客户端的响应中
add_header Request-ID $request_id;
location / {
    proxy_pass http://backend;
    # 将 X-Request-ID 请求头发送到应用
    proxy_set_header X-Request-ID $request_id;
    access_log /var/log/nginx/access_trace.log trace;
}
}

```

在本示例配置中，我们配置了名为 `trace` 的 `log_format`，并在日志中使用了 `$request_id` 变量。还在发起上游请求时，通过使用 `proxy_set_header` 指令将请求 ID 添加到请求头里，将此 `$request_id` 变量传递给了上游应用。此外，还通过使用 `add_header` 指令，在响应头中设置请求 ID，将请求 ID 传回客户端。

详解

该功能在 NGINX Plus R10 和 NGINX 开源版 1.11.0 中可用，`$request_id` 变量提供了一个随机生成的 32 位十六进制字符串，这些字符可以用来唯一性标识请求。通过将此标识符传递给客户端和应用，您可以将日志与请求相关联。在前端客户端，您可通过响应头收到该请求唯一性标识字符串，并且可以利用它在日志中搜索与请求相关联的条目。您还需要在应用日志中捕获和记录此请求唯一性标头，从而在日志之间建立真正的端到端关联。基于这项功能，NGINX 让跨应用栈跟踪请求变为可能。

其他参考资料

实操指南 13.4 “OpenTelemetry for NGINX”

15.0 简介

NGINX 调优是一种艺术。无论何种类型的服务器或应用，其性能调优都取决于许多可变量，包括但不限于环境、用例、需求和相关物理组件。瓶颈驱动型性能调优十分常见，意指遇到瓶颈后才进行测试，确定瓶颈，改进限制，并不断重复，直至满足性能需求。在本章中，我们建议在系统调优时，使用自动化工具进行测试，并收集和衡量测试结果。本章还介绍了如何通过连接调优保持客户端和上游服务器的长连接，以及如何通过优化操作系统调优满足更多并发连接需求。

15.1 使用压测工具实现测试自动化

问题

您希望使用压测工具实现测试自动化，以确保测试一致性和可重复性。

解决方案

使用 HTTP 压测工具，例如 Apache JMeter、Locust、Gatling 或团队标准化的任何测试工具。为压测工具创建配置，对您的 Web 应用做全面测试，包括对服务进行测试。查看从测试中收集的指标，以建立基线。缓慢增加模拟的并发用户数，以模拟典型的生产使用情况并确定改进点。对 NGINX 进行调优并不断重复此流程，直至实现预期性能。

详解

通过使用自动化测试工具来定义测试，可通过一致的测试，构建 NGINX 调优指标。必须能够重复测试并衡量性能优劣以进行科学分析。在对 NGINX 配置进行任何调整之前，先进行测试以建立基线，这样才能衡量配置更改是否实现了性能优化。对每次更改进行衡量，有助于确定性能得以提升的根源。

15.2 控制浏览器缓存

问题

您希望通过在客户端缓存内容来提高性能。

解决方案

使用客户端的 cache-control HTTP 请求头：

```
location ~* \.(css|js)$ {  
    expires 1y;  
    add_header Cache-Control "public";  
}
```

这个 location 代码块指定客户端可以缓存 CSS 和 JavaScript 文件的内容。expires 指令指示客户端将缓存资源的有效期设置为一年。add_header 指令将 HTTP 响应头 Cache-Control 添加到响应中，值为 public，允许沿途的所有缓存服务器缓存资源。如果将值设置为 private，则只允许客户端缓存资源。

详解

缓存性能有许多影响因素，其中磁盘速度最为重要。NGINX 配置中有很多指令可以帮助提升缓存性能。其中一种方法是设置 HTTP 响应头，让客户端只缓存响应但是不向 NGINX 发送请求，这样只需读取缓存便可。

15.3 保持客户端长连接

问题

您想要增加可在单个客户端连接上发起的请求数以及空闲连接可保持打开状态的时长。

解决方案

使用 `keepalive_requests` 和 `keepalive_timeout` 指令，更改可在单个连接上发起的请求数以及空闲连接可保持打开状态的时长。

```
http {  
    keepalive_requests 320;  
    keepalive_timeout 300s;  
    # ...  
}
```

`keepalive_requests` 指令默认值为 100，`keepalive_timeout` 指令默认值为 75 秒。

详解

一般情况下，单个连接上的默认请求数能够满足客户端需求，因为现代浏览器能够为每台服务器（根据 FQDN）打开多个连接。与同一域名的并发开放连接数通常仍限制为小于 10，因此就这一点而言，仍会发生在单个连接上发送多个请求的情况。内容交付网络常用的 HTTP/1.1 创建多个域名指向内容服务器，并以编码的方式轮换所用域名，从而使浏览器打开更多连接。如果前端应用不断轮询后端应用以进行更新，那么这些连接优化可能会很有帮助，因为允许更多请求和保持更长时间的连接可减少需要建立的连接数量。

15.4 保持上游长连接

问题

您希望保持上游服务器长连接，以通过连接复用提高性能。

解决方案

在 upstream 上下文中使用 keepalive 指令，保持上游服务器连接开放以供复用：

```
proxy_http_version 1.1;
proxy_set_header Connection "";

upstream backend {
    server 10.0.0.42;
    server 10.0.2.56;

    keepalive 32;
}
```

upstream 上下文中的 keepalive 指令会为每个 NGINX worker 进程保持打开状态的连接开启高速缓存。该指令表示每个 worker 进程可保持打开状态的最大空闲连接数。在 upstream 代码块上使用的代理模块指令在确保 keepalive 指令正常用于上游服务器连接方面发挥着不可或缺的作用。proxy_http_version 指令指示代理模块使用 HTTP 版本 1.1，以允许在单个保持打开状态的连接上连续发送多个请求。proxy_set_header 指令指示代理模块删除默认请求头 close，以允许连接保持打开状态。

详解

您需要保持上游服务器长连接，以缩短建立连接所需的时间，并允许 worker 进程直接将请求分发到空闲连接上进行处理。需要注意的是，长连接的数量可以超过 keepalive 指令中指定的连接数量，因为长连接和空闲连接并不等同。keepalive 连接数应尽量少，以允许上游服务器接受其他接入连接。该 NGINX 调优技巧可通过减少连接建立开销，提升服务器性能。

15.5 响应缓冲

问题

您希望在内存中为上游服务器和客户端之间的响应启用缓冲区，以避免将响应写入临时文件。

解决方案

调整代理缓冲区设置，以允许 NGINX 将响应内容写入内存缓冲区：

```
server {  
    proxy_buffering on;  
    proxy_buffer_size 8k;  
    proxy_buffers 8 32k;  
    proxy_busy_buffer_size 64k;  
    # ...  
}
```

`proxy_buffering` 指令的值可以是 `on` 或 `off`，默认是 `on`。`proxy_buffer_size` 表示用于读取来自代理服务器的响应头部和响应头的缓冲区大小，默认为 4K 或 8K，具体取决于平台。`proxy_buffers` 指令包含两个参数：缓冲区数量和缓冲区大小。默认情况下，`proxy_buffers` 指令被设置为 8 个缓冲区，依据平台不同单个缓冲区容量为 4k 或 8k。`proxy_busy_buffer_size` 指令限定了繁忙缓冲区的大小，繁忙缓冲区可支持 NGINX 在还未完全读取上游服务响应内容的情况下直接响应客户端。繁忙缓冲区的大小默认为代理缓冲区或正常缓冲区大小的两倍。如果禁用代理缓冲区，那么发生故障时，将无法把请求发送给下一台上游服务器，因为请求内容已由 NGINX 发送。

详解

代理缓冲区能显著提升代理服务性能，具体取决于响应内容的大小。调整缓冲区设置可能会产生不利影响，因此您务必要参考响应消息体的平均大小并反复进行全面测试。非必要情况下不要设置过大缓冲区，因为这会占用大量的 NGINX 内存。为了优化性能，您可以只为已知会返回大型响应消息体的特定位置设置大型缓冲区。

其他参考资料

[NGINX proxy_request_buffering 模块文档](#)

15.6 访问日志的缓冲

问题

当系统处于高负载状态时，您希望启用日志缓冲，以降低 NGINX worker 进程发生阻塞的可能性。

解决方案

设置访问日志的缓冲区大小和刷新时间：

```
http {  
    access_log /var/log/nginx/access.log main buffer=32k  
    flush=1m gzip=1;  
}
```

`access_log` 指令的 `buffer` 参数表示内存缓冲区的大小，在写入磁盘之前日志数据可保存在内存缓冲区中。`access_log` 指令的 `flush` 参数设置在写入磁盘之前，日志可保存在缓冲区中的最长时间。当使用 `gzip` 时，日志数据在写入日志之前会进行压缩，具有 1（最快轻度压缩）到 9（最慢极限压缩）九个压缩级别。

详解

将日志数据缓冲到内存中可能只是一种很小的优化手段。但是，对于存在大量请求的站点和应用来说，这会显著影响磁盘和 CPU 的使用率。当 `access_log` 指令中包含 `buffer` 参数时，如果日志条目无法再进入缓冲区，日志会直接写入磁盘。如果同时包含 `flush` 参数和 `buffer` 参数，当缓冲区中的数据超过指定期限时，日志也会写入磁盘。如果跟踪日志，您可能会发现启用缓冲时，延迟可达到 `flush` 参数指定的时长。

15.7 操作系统调优

问题

为应对尖峰负载或高流量站点，您想要优化操作系统以接受更多连接。

解决方案

检查 `net.core.somaxconn` 的内核设置，该参数表示内核可排队等待 NGINX 处理的最大连接数。如果将其设置为大于 512 的值，则需要设置 NGINX 配置中 `listen` 指令的 `backlog` 参数以进行匹配。当需要检查此内核设置时，内核日志会给予明确提示。NGINX 会非常快速地处理连接，因此对于大多数用例，无需更改此设置。

通常更需要调大可打开文件描述符（open file descriptor）上限。在 Linux 中会为每个连接打开一个文件句柄；因此，如果将 NGINX 用作代理或负载均衡器，由于需要与上游服务建立连接，NGINX 可能会为每个连接打开两个文件句柄。为了满足大量连接需求，可能需要使用内核选项 `sys.fs.file_max` 调大系统的文件描述符上限；而对于系统用户，确保 NGINX 按照 `/etc/security/limits.conf` 文件中的配置运行。此外，可能还需要更改 `worker_connections` 和 `worker_rlimit_nofile` 的值。它们都是 NGINX 配置中的指令。

开启更多临时端口以支持更多连接。当 NGINX 充当反向代理或负载均衡器时，每个上游连接都会为响应流量打开一个临时端口。根据系统配置，服务器可能不会将所有临时端口都打开。如要确认打开端口的数量，请查看内核设置 `net.ipv4.ip_local_port_range` 的设置。该设置表示所能使用的临时端口范围。通常将此内核设置设为从 1024 到 65535。1024 是 TCP 注册端口结束的位置，65535 是动态或临时端口结束的位置。请记住，下限应高于最高的开放侦听服务端口号。

详解

当您为了满足大量连接需求而开始调优时，应首先优化操作系统。您可以根据特定用例，对内核进行多方面的优化。但是，内核调优不应只是一时兴起，而是应根据其性能表现来衡量更改，以确保更改有所帮助。如前所述，您可以根据内核日志中记录的消息来确定何时开始优化内核，或者当 NGINX 在错误日志中给予明确提示时，开始优化内核。

第 1 章 (1-8)

- 高级软件包工具 (APT) , 2
- alias 指令, 8
- AlmaLinux, 2
- APT (高级软件包工具) , 2
- 身份验证, 63-75
 - NGINX Plus 证书, 3
- CentOS, 2
- 证书
 - 安装 NGINX Plus, 3
- 命令, 5
- 配置, 6
 - (另请参阅“身份验证”“可编程性”)
 - 使用 includes 使配置文件清晰简洁, 6
- curl 命令
 - 请求测试, 4
- 将 NGINX 作为守护进程启动, 4
- Debian, 1-2
- FastCGI 虚拟服务器, 7
- 文件和目录, 4
- include 指令, 6
- index 指令, 8
- 安装, 1-4
 - 在 Debian/Ubuntu 上, 1-2
 - NGINX Plus, 3
- 在 RedHat/Oracle Linux/AlmaLinux/Rocky Linux 上, 2
 - 验证, 3
- listen 指令
 - 提供静态内容, 8
- location 块, 8
- location 指令, 8
- lsb_release 命令, 2
- NGINX, 1-8
 - 命令, 5
 - 文件、目录和命令, 4-6
 - 使用 includes 使配置文件清晰简洁, 6

- 安装, 1-4
 - 提供静态内容, 7-8
- nginx 命令, 4
- NGINX GPG 软件包, 2
- NGINX Plus, 1
- 安装, 3
- ps 命令, 4
- RedHat Enterprise Linux (RHEL) , 2
- 重载配置, 6
- RHEL (RedHat Enterprise Linux) , 2
- Rocky Linux, 2
- root 指令, 8
- 安全套接字层 (SSL) 配置, 7
- server_name 指令, 8
- SSL (安全套接字层) 配置, 7
- 提供静态内容, 7-8
- server 块
 - 通过 HTTP 提供静态文件, 7
- Ubuntu, 1-2
- 统一资源标识符 (URI) , 8
- YUM 软件包管理器, 3

第 2 章 (9-24)

- 主动健康检查, 10, 22-23
- 连接清空, 20
- 数据报, 14
- DNS SRV 记录, 14
- drain 参数, 20
- 匹配代码块时所用的 expect 指令, 23
- 通用哈希负载均衡, 15, 16
- 哈希指令, 16
- 健康检查, 10
 - 主动健康检查, 10, 22-23
 - 被动健康检查, 10, 21
 - 数据流, 22
- health_check 指令, 22
- 横向扩展, 9

- HTTP
 - 健康检查, 22
 - IP 哈希负载均衡, 16
 - 负载均衡, 10
 - sticky cookie 指令, 17
 - 相对于 stream 上下文, 12
- http 块, 12
- IP 哈希负载均衡, 15, 16
- ip_hash 指令, 17
- 负载均衡策略: 最少连接, 15
- 负载均衡策略: 最短时间, 16
- least_conn 指令, 15
- least_time 指令, 16
- listen 指令
 - UDP 负载均衡, 13, 14
- 负载均衡, 9-24
- Amazon EC2, 141
 - 客户端/服务器绑定, 17-20
 - 连接清空, 20
 - 健康检查, 10, 21-23
 - HTTP 服务器, 10
 - 方法, 14-17
 - 慢启动, 24
 - TCP 服务器, 11-13, 22
 - UDP 服务器, 13-14
- location 块, 22
- 流媒体服务器的 match 代码块, 22
- 网络时间协议 (NTP) 服务器, 13
- NGINX Plus, 1
 - 客户端/服务器绑定, 17-20
 - 连接清空, 20
 - 健康检查, 10, 22-23
 - 负载均衡策略: 最短时间, 16
- NGINX Plus API, 20
 - (另请参阅“可编程性”)
 - 连接清空, 20
- NTP (网络时间协议) 服务器, 13
- OpenVPN 服务, 13
- 被动健康检查, 10, 21
- proxy_responses 指令, 14
- proxy_timeout 指令, 14
- 随机指令, 16
- 随机负载均衡, 16
- require 指令, 23
- reuseport 参数, 13, 14
 - TCP 负载均衡, 11
- 轮询负载均衡, 15
- match 代码块中的 send 指令, 23
- server 指令, 11
- server slow_start 指令, 24
- 通过 NGINX Plus 对服务器进行添加和删除, 20
- server_drain 指令, 20
- 会话状态, 9, 16, 18, 20
- 负载均衡中的慢启动, 24
- 有状态与无状态应用, 9
- sticky_cookie 指令, 17
- sticky_learn 指令, 18
- sticky_route 指令, 19
- stream 代码块, 12
- stream 健康检查, 22
- stream 模块, 11-14
- TCP 协议
 - 负载均衡, 11-13, 22
- upstream 代码块, 10, 11
- 上游模块, 11
- 用户数据报协议 (UDP)
 - 负载均衡, 13-14, 22

第 3 章 (25-36)

- A/B 测试, 25-26
- 访问控制, 171
 - (另请参阅“安全控制”)
 - 基于国家或地区的限制, 30-31
- APT 软件包管理器
 - GeoIP 安装, 27, 28
- AWS 弹性负载均衡 (AWS ELB), 32
- Azure (请参阅“Microsoft Azure”)
- Microsoft Azure, 32
- 限制带宽, 35
- 蓝绿部署, 26
- 无类别网路由 (CIDR), 32
- 连接
 - 流量管理限制, 32-33
- 灰度发布部署, 26
- 弹性负载均衡 (AWS ELB), 32
- 转发请求头, 32
- GeoIP 模块, 27-30
- GeoIP2, 27
 - geoip_city 指令, 29
 - geoip_country 指令, 28
 - geoip_proxy_recursive 指令, 31
 - geoip2_proxy 指令, 31
- 谷歌负载均衡器, 32
- HTTP
 - limit_conn_zone 指令, 33
- IP 地址
 - 查找原始客户端, 31
 - 限制连接数, 32-33
 - 限制请求速率, 34-35
- 限制带宽, 35
- 限制连接数, 32-33
- 限制请求速率, 34-35
- limit_conn 指令, 32-33
- limit_conn_dry_run 指令, 33, 35

- limit_conn_zone 指令, 33
- limit_rate 指令, 36
- limit_rate_after 指令, 36
- limit_req 指令, 34
- limit_req_log_level 指令, 35
- Amazon EC2, 141
 - AWS ELB, 32
- load_module 指令, 28
- Microsoft Azure, 32
- 网络地址转换 (NAT) , 33
- NGINX, 1-8
 - nginx-module-geoip, 27
- NGINX Plus, 1
 - nginx-plus-module-geoip, 27
- nginx-module-geoip 包, 27
- nginx-plus-module-geoip 包, 27
- proxy_pass 指令, 26
- 限速模块, 34-35
- 代理
 - 查找原始客户端 IP 地址, 31
- 安全控制, 77-97
 - IP 地址限制, 32-35
- 共享内存区, 34
- split_clients 模块, 25
- server 块
 - 基于国家/地区的访问限制, 31
- 流量管理, 25-36
 - A/B 测试, 25-26
 - 基于国家/地区的访问限制, 30-31
 - GeoIP 模块和数据库, 27-30
 - 限制连接数, 32-33
 - 通过预定义键来限制请求速率, 34-35
 - 查找原始客户端 IP 地址, 31-32
- X-Forwarded-For 请求头, 32
- YUM 软件包管理器, 3
 - GeoIP 安装, 27, 28

第 4 章 (37-44)

- 绕过缓存, 41
- 缓存, 37-44
 - 绕过缓存, 41
 - 哈希键, 39
 - 使对象无效, 42
 - 锁定缓存, 40
 - 内存缓存区, 37-38
 - 性能调优, 37
 - 将文件分段以提高效率, 43-44
 - 使用陈旧缓存, 40
- CDN (内容交付网络) , 37
- 客户端缓存, 37
- 内容交付网络 (CDN) , 37

- 调试, 159-165
 - 绕过缓存, 42
- 使用过期缓存, 40
- 哈希键, 39
- HTML5 视频, 44
- proxy_cache_path 的 inactive 参数, 38
- 使缓存对象无效, 42
- proxy_cache_path 的 levels 参数, 38
- 锁定缓存, 40
- proxy_cache_path 的 max_size 参数, 38
- 内存缓存区, 37-38
- NGINX Plus, 1
 - 使缓存对象无效, 42
- 性能调优, 167-173
 - 缓存, 37
- proxy_cache 指令, 38
- proxy_cache_bypass 指令, 41
- proxy_cache_key 指令, 39, 44
- proxy_cache_lock 指令, 40
- proxy_cache_lock_age 指令, 40
- proxy_cache_lock_timeout 指令, 40
- proxy_cache_path 指令, 37-38
- proxy_cache_purge 指令, 43
- proxy_cache_use_stale 指令, 40
- 安全控制, 77-97
 - 缓存哈希键, 39
- 将文件分段以提高效率, 43-44
- slice 指令, 43-44
- 缓存切片, 43-44
- 使用陈旧缓存, 40
- 用于视频交付的 Cache Slice 模块, 44

第 5 章 (45-62)

- Ansible, 56-57
- Ansible 自动化平台, 57
- Ansible Galaxy, 57
- APT 软件包管理器
 - 提供 JavaScript 功能的 NJS 模块, 52
- 身份验证, 63-75
 - JWT, 52
- C 模块, 55
- C 编程语言, 54
- Chef, 58-59
- 无类域间路由 (CIDR) , 51
- cluster-aware 键值存储, 49, 51
- 配置, 6
 - (另请参阅“身份验证”“可编程性”)
 - 使用 Consul 模板, 59
- 连接清空, 47-49
- Consul 模板, 59-61
- consul-template 守护进程, 60

- curl 命令
 - 连接清空, 47-49
 - 键值存储, 50
- 防火墙, 57
- 安装, 1-4
 - 使用 Ansible, 56-57
 - 使用 Chef, 58-59
 - njs 模块, 51-54
- JavaScript, 51-54
- Jinja2 模板语言, 57
- JSON Web 令牌 (JWT) , 52
- 键值存储, 49-51
- keyval 指令, 50
- keyval_zone 指令, 50
- Lua 模块, 54
- 内存缓存区, 50
- NGINX, 1-8
 - 使用通用编程语言进行扩展, 54-56
- NGINX App Protect WAF 模块, 57
- NGINX JavaScript (njs) 模块, 51-54
- NGINX Plus, 1
 - 动态环境配置, 45-49
 - 键值存储设置, 49-51
- NGINX Plus API, 45-49
 - (另请参阅“可编程性”)
 - 支持添加和删除服务器, 45-49
- nginx_config 资源, Chef, 58
- nginx_config 角色, Ansible, 56
- nginx_site 资源, Chef, 58
- ngx 对象, Lua 模块, 55
- njs (NGINX JavaScript) 模块, 51-54
- Perl 模块, 55
- perl_set 指令, 55
- 可编程性, 45-60
 - 使用 Ansible 安装, 56-57
 - 使用 Chef 安装和配置, 58-59
 - Consul 模板, 59-61
 - 动态环境配置, 45-49
 - 扩展 NGINX, 54-56
 - 键值存储设置, 49-51
- 提供 JavaScript 功能的 NJS 模块, 51-54
- Python, 57
- 通过 NGINX Plus 对服务器进行添加和删除, 45-49
- 流量管理, 25-36
 - NGINX Plus 键值存储, 49-51
- 故障排除 (请参阅“调试”)
- 键值存储中的 type 参数, 51
- Web 应用防火墙 (WAF) , 57
- YAML 文件
 - Ansible 配置, 57
- YUM 软件包管理器, 3
 - 提供 JavaScript 功能的 NJS 模块, 52

第 6 章 (63-76)

- 身份验证, 63-75
 - HTTP 基础, 63-65
 - JWKs, 68-69
 - JWK 缓存, 71
 - JWT, 66-67, 70
 - OIDC 身份验证供应商, 69
 - SAML 身份验证供应商, 72-75
 - 子请求, 65-66
- auth_basic 指令, 64
- auth_basic_user_file 指令, 64
- auth_jwt 指令, 66, 67
- auth_jwt_key_file 指令, 66, 70
- auth_jwt_key_request 指令, 71
- auth_request 指令, 65
- auth_request_set 指令, 66
- 使用 crypt() (C 函数) 实现身份验证, 64
- curl 命令
 - HTTP 身份验证测试, 64
- 加密, 67
- htpasswd 命令, 64
- HTTP
 - 身份验证, 63-67
- http_auth_request_module, 65
- 身份验证供应商 (IdP)
 - OIDC, 69
 - SLO, 74
- Internal_redirect 指令, 66
- JSON Web 密钥集 (JWKs) , 71
- JSON Web 密钥 (JWK) , 68-69
- JSON Web 签名, 71
- JSON Web 令牌 (JWT) , 66-67, 70
- JavaScript, 72-75
- 键值存储, 70
- JWK 文件的 kty 属性, 68
- LDAP (轻量级目录访问协议) , 64
- 轻量级目录访问协议 (LDAP) , 64
- 内存缓存区, 71
- NGINX, 1-8
 - 身份验证, 63-66
- NGINX Plus, 1
 - 身份验证, 66-75
- nginx_http_internal_redirect_module, 66
- njs (NGINX JavaScript) 模块, 72-75
- OpenID Connect (OIDC) 身份验证供应商, 69
- openssl 命令, 64
- openssl passwd 命令, 64
- 身份验证密码, 63-65
- proxy_pass_request_body 指令, 66
- SAML 身份验证, 72-75
- SAML 单点注销 (SLO) , 74

- 安全控制, 77-97
 - JWK, 69
- 身份验证中的子请求, 65-66
- YUM 软件包管理器, 3
 - SAML 身份验证, 72
- 第 7 章 (77-98)**
- 访问控制, 171
 - (另请参阅“安全控制”)
 - 基于 IP, 77
- Access-Control-Allow-Origin 请求头, 79
- add_header 指令, 79
- F5 高级应用安全性, 97
- allow 指令, 78
- App Protect 模块, 93
- app_protect_* 指令, 94
- app_protect_enable 指令, 95
- app_protect_policy_file 指令, 95
- app_protect_security_log 指令, 95
- 构建自动拦截列表, 93-94
- AWS 弹性负载均衡 (AWS ELB), 89
- 缓存, 37-44
 - SSL 会话缓存, 82
- 证书
 - 客户端加密证书, 79-82
 - 安装 NGINX Plus, 3
 - 上游加密, 83
- 客户端加密, 79-82
- 集群级速率限制的构建, 93-94
- 配置, 6
 - (另请参阅“身份验证”“可编程性”)
 - 客户端加密配置, 80-82
 - NGINX App Protect WAF 模块, 94-97
- 使用 GeoIP 模块实现基于国家或地区的限制, 90-91
- 跨域资源共享 (CORS), 78-79
- deny 指令, 78
- DoS 模块, 93
- 动态 DDoS 缓解, 92-94
- 弹性负载均衡 (AWS ELB), 89
- ECC (椭圆曲线密码学) 格式的密钥, 82
- 加密, 79-83
- NGINX App Protect 的 enforcementMode 策略, 95
- 使用过期日期保护地址, 85-86
- 生成过期链接, 86-88
- 防火墙, 94-97
- GeoIP 模块, 90-91
- 哈希摘要, 85
- Python 的 hashlib 库, 85
- HSTS (HTTP 严格传输安全协议), 90
- HTTP
 - 访问模块, 77
 - 代理模块 SSL 规则, 83

- http SSL 模块, 81
- HTTP 严格传输安全协议 (HSTS), 90
- HTTPS
 - 重定向, 88-89
 - 上游加密, 83
- IP 地址
 - 基于 IP, 77
- 键值存储, 94
- keyval_zone 指令, 94
- limit_req_zone 指令, 94
- load_module 指令, 94
- location 块, 84
- 日志记录
 - app_protect_security_log 指令, 95
- md5 哈希, 84
- ModSecurity 3.0 NGINX 模块, 77
- NGINX Plus, 1
 - 动态 DDoS 缓解, 92-94
 - NGINX App Protect WAF 模块, 94-97
- ngx_http_ssl_module, 80
 - HTTP 代理模块 SSL 规则, 83
- ngx_stream_ssl_module, 80
- NGINX App Protect WAF 模块, 94-97
- openssl 命令, 85
- proxy 指令, 83
- proxy_pass 指令, 83
- proxy_ssl_certificate 指令, 83
- proxy_ssl_certificate_key 指令, 83
- proxy_ssl_crl 指令, 83
- proxy_ssl_protocols 指令, 83
- Python, 85, 87
- CORS 请求方法, 78-79
- RSA 格式密钥, 82
- satisfy 指令, 92
- 密钥, 84-85
- 安全链接模块, 84, 85
- secure_link 指令, 86
- secure_link_md5 指令, 86
- secure_link_secret 指令, 84
- 安全控制, 77-97
 - App Protect 模块, 93
 - 身份验证 (请参阅“身份验证”)
 - 客户端加密, 79-82
 - 允许 CORS, 78-79
 - 使用 GeoIP 模块实现基于国家或地区的限制, 90-91
 - DoS 模块, 93
 - 动态 DDoS 缓解, 92-94
 - 使用过期方法保护地址, 85-88
 - HSTS, 90
 - HTTPS 重定向, 88-89
 - IP 地址限制, 77
 - 保护 location 代码块, 84

- NGINX App Protect WAF 模块, 94-97
 - 提供多种方法, 92
 - 密钥, 84-85
 - 上游加密, 83
- SSL 模块, 80
- SSL/TLS
 - 客户端加密, 79-82
 - HTTPS 重定向, 89
 - 上游加密, 83
- ssl_certificate 指令, 80
- ssl_certificate_key 指令, 80
- stream 访问模块, 77
- stream SSL 模块, 81
- Strict-Transport-Security 请求头, 90
- sync 参数
 - DDoS 攻击缓解, 94
- 上游加密, 83
- Web 应用防火墙 (WAF), 77
- X-Forwarded-Proto 请求头, 89

第 8 章 (99-104)

- TCP 协议
 - 队头阻塞问题, 99
- HTTP/2, 99-100
- 用户数据报协议 (UDP)
 - QUIC, 99
- 配置, 6
 - (另请参阅“身份验证”“可编程性”)
 - HTTP/2, 99-100, 102-104
 - HTTP/3 over QUIC, 99, 100-101
- Alt-Svc 请求头, 100
- http2 指令, 100
- nghttp 实用程序, 100
- listen 指令
 - quic 参数, 101
- ssl_protocol 指令, 101
- gRPC 方法调用, 102-104
- grpc_pass 指令, 102
- SSL/TLS
 - HTTP/2 注意事项, 100
 - QUIC 协议, 99, 100-101
 - 代理 gRPC 连接, 102
- Amazon EC2, 141
 - gRPC 调用, 103
- location 指令, 103
- 加密, 103

第 9 章 (105-108)

- Flash Video (FLV) 格式, 105
- NGINX, 1-8
 - MP4 和 FLV 流媒体, 105
- HLS (HTTP Live Stream) 模块, 106

- HTTP Live Stream (HLS) 模块, 106
- NGINX Plus, 1
 - 使用 HLS 和 HDS 处理流媒体, 106-108
- f4f 模块, 107
- HDS (HTTP Dynamic Streaming), 107
- hls_buffers 指令, 107
- HTTP Dynamic Streaming (HDS), 107
- f4f_buffer_size 指令, 108
- MP4 格式
 - 带宽限制, 108
 - 在 NGINX 中使用 FLV, 105
 - 使用 HLS 处理 MP4, 106
- mp4_limit_rate 指令, 108
- mp4_limit_rate_after 指令, 108
- 流量管理, 25-36
 - 限制每个客户端的带宽, 108
- 限制带宽, 108
- 流媒体, 105-108
 - 使用 NGINX Plus 限制带宽, 108
 - HDS, 107
 - 使用 HLS 处理 MP4, 106
 - MP4 和 FLV, 105
- 使用 HLS 和 HDS 处理流媒体, 106-108

第 10 章 (109-120)

- IaaS (基础设施即服务), 109
- 基础设施即服务 (IaaS), 109
- 配置, 6
 - (另请参阅“身份验证”“可编程性”)
 - AMI 配置管理, 110
- HashiCorp 中的 Packer, 110
- 虚拟机 (VM)
 - 使用 Packer 进行云配置, 110
 - 在 Azure 上创建 NGINX 镜像, 111-112
 - Google Cloud 镜像, 113
- Amazon EC2, 141
 - AWS NLB, 115-117
 - Azure, 117
 - 不经过.....直接路由到节点, 113-114
- Amazon EC2 UserData, 111
- GCP (Google Cloud Platform), 112
- Google Cloud Platform (GCP), 112
- Google Compute Image, 113
- Google Cloud 镜像, 113
- Amazon Route 53 DNS 服务, 114
- 根据地理位置进行 NGINX 节点路由, 114
- 健康检查, 10
 - Amazon Route 53, 114
- Route 53 DNS 服务, 114
- Auto Scaling 组, 114, 115-117
- 轮询负载均衡, 114
- Microsoft Azure, 32

- 通过 Microsoft Azure 上的规模集 (scale set) 进行
 负载均衡, 117
- 虚拟机, 111-112
- VMSS, 117
- 虚拟机规模集 (VMSS) , 117
- server 指令, 118
- proxy_pass 指令, 118
- upstream 代码块, 118
- 证书
 - Google App Engine, 118
- DNS
 - Google App Engine 代理, 118-119
- Google Compute Cloud, 118
- Google Compute Engine, 118
- resolver 指令, 118
- 云部署, 109-119
 - AWS 上的自动配置, 109-111
 - 使用 Google App Engine 代理, 118-119
 - 通过 Azure 上 NGINX 规模集 (scale set) 进行负载
 均衡, 117
 - 使用 GCP 构建机器镜像, 112
 - NLB sandwich, 115-117
 - 不经过负载均衡器直接路由到节点, 113-114
 - Azure 中的虚拟机, 111-112
- Amazon 弹性计算云 (Amazon EC2) , 110-111
- Amazon 机器镜像 (AMI) , 110-111
- Amazon Web Services (AWS) 上的自动配置, 110-111
- AWS (Amazon Web Services) 上的自动配置, 110-111
- 在 Azure 上创建 NGINX 镜像, 111-112
- 故障转移
 - Amazon Route 53, 113-114
- AWS 网络负载均衡器 (AWS NLB) , 115-117
- 网络负载均衡器 (NLB) , 115-117
- NLB (网络负载均衡器) , 115-117
- VMSS (虚拟机规模集) , 117
- Google App Engine, 118-119
- 微服务架构, 119

第 11 章 (121-136)

- Amazon EC2, 141
 - 在容器化环境中, 121
 - DNS SRV 记录, 127
- 日志记录
 - 在容器化环境中, 121
- least_time 指令, 121
- Crilly, Liam, 122
- 将 NGINX 部署为 API 网关 (Crilly) , 122
- upstream 代码块, 123
- 在外部设置上游服务, 123
- include 指令, 123
- location 块, 124
- 预共享 API 密钥, 124

- NGINX 作为 API 网关时所用的 rewrite 指令, 124
- 安全控制, 77-97
 - NGINX 作为 API 网关, 124
- 限制请求速率, 125
- 共享内存区, 125
- 统一资源标识符 (URI) , 126
- 微服务架构, 126
- 代码所有者, 126
- NGINX Plus, 1
 - DNS SRV 记录, 126
 - 容器镜像, 132
 - Kubernetes ingress controller (Kubernetes Ingress
 控制器) , 134
- server 指令的 resolve 参数, 127
- Auto Scaling 组, 127
- resolver 指令, 127
- server 指令, 127
- 来自 Docker Hub 的 NGINX 镜像, 127
- 从 Docker Hub 中拉取 NGINX 镜像, 127
- 使用 Dockerfile 创建镜像, 127, 128-133
- NGINX, 1-8
 - 作为 API 网关, 122-126
 - 容器镜像, 127, 128-132
- resolver 指令中的 valid override 参数, 127
- Alpine Linux, 128
- Amazon Elastic Container Registry, 128
- docker 命令, 128
- OpenShift, 128
- load_module 指令, 133
- Perl 模块, 133
- perl_set 指令, 133
- docker build 命令, 133
- nginx_http_perl_module, 133
- nginx_ingress 命名空间和服务账户, 134
- 访问控制, 171
 - (另请参阅“安全控制”)
 - 基于角色的访问控制, 135
- Kubernetes DaemonSet, 135
- Ingress controller (Ingress 控制器) 的 Deployment 与
 DaemonSet 方法, 135
- Kubernetes 的 LoadBalancer 服务 (service) 类型, 135
- Kubernetes 负载均衡器中的 NodePort 服务 (service)
 类型, 135
- RBAC (基于角色的访问控制) , 135
- 基于角色的访问控制 (RBAC) , 135
- YAML 文件
 - NGINX Plus Deployment 清单, 135
- Kubernetes PROXY 协议, 136
- 容器, 121-136
 - NGINX 作为 API 网关, 122-126
 - NGINX Plus 容器镜像, 132
 - 使用 Dockerfile 创建 NGINX 镜像, 128-132

- NGINX 中的环境变量, 133
- Kubernetes ingress controller (Kubernetes Ingress 控制器), 134-136
- 官方 NGINX 镜像, 127
- Docker
 - 镜像构建, 128-132
- ingress controller (Ingress 控制器), 134-136
- Kubernetes, 134-136

第 12 章 (137-146)

- Amazon EC2, 141
- 高可用性, 137
- 故障转移
 - active-passive 故障转移, 137
- HA (高可用性) 部署模式, 137-145
 - 配置同步, 142-144
 - 通过 DNS 实现负载均衡器的负载均衡, 141
 - 在 EC2 上实现负载均衡, 141
 - HA (高可用性) 模式, 137-140
 - 与 zone sync 共享状态, 144-145
- keepalived, 137
- 安装, 1-4
 - HA (高可用性) keepalived, 137-140
 - nginx-sync 包, 142
- nginx-ha-keepalived 包, 137-140
- nginx-ha-setup 脚本, 138, 140
- 分发到 NGINX 节点, 140
- HA (高可用性) 部署模式, 140
- 虚拟机 (VM)
 - 虚拟路由器冗余协议 (VRRP), 140
- IP 地址
 - keepalived, 140
 - EC2 和虚拟地址, 140
 - 向 DNS A 记录中添加多个地址, 141
 - 通过 DNS 实现负载均衡, 141
- Route 53 DNS 服务, 141
- DNS
 - Amazon Route 53 服务, 141
 - 分发到 NGINX 节点, 140
- Amazon Route 53 DNS 服务, 141
- Auto Scaling 组, 141
- 可用区, 141
- DynDNS, 141
- 轮询负载均衡, 141
- FQDN (完整域名), 141
- 健康检查, 10
 - DNS 负载均衡, 141
 - EC2 负载均衡, 141
- Amazon 弹性计算云 (Amazon EC2), 141
- AWS 网络负载均衡器 (AWS NLB), 141
- 云部署, 109-119
 - NLB sandwich, 141

- AWS NLB, 141
- 网络负载均衡器 (NLB), 141
- NLB (网络负载均衡器), 141
- ALB (应用负载均衡器), 142
- 应用负载均衡器 (ALB), 142
- APT 软件包管理器
 - 配置同步, 142, 145
- AWS 应用负载均衡器 (AWS ALB), 142
- CONFPATHS 参数, 143
- EXCLUDE 参数, 143
- ip addr 命令, 143
- 配置同步时使用的 NODES 参数, 143
- Docker
 - 配置管理, 144
- nginx-sync.sh 应用, 144
- sync 参数
 - 区域配置, 144
- stream 服务器, 145
- zone_sync 模块, 145
- zone_sync_server 指令, 145
- 共享内存区, 144-145
- 配置, 6
 - (另请参阅“身份验证”“可编程性”)
 - HA (高可用性) 部署模式的同步, 142-144
- 区域同步, 144-145

第 13 章 (147-158)

- NGINX, 1-8
 - 启用 stub 状态, 147-148
- stub_status 模块, 147-148
- 活动监控, 147-158
 - 日志记录 (请参阅“日志记录”)
 - 指标收集, 150-153
 - 监控仪表盘, 148-150
 - 用于链路追踪收集器的 OTel 集成, 153-156
 - Prometheus Exporter 模块, 157-158
 - stub 状态, 147-148
- NGINX Plus API, 20, 45-49
 - (另请参阅“可编程性”)
 - 指标收集, 150-153
 - OTel 集成特性, 155
 - 服务器统计数据, 151-153
- stub_status 指令, 148
- 健康检查, 10
 - TCP/UDP, 149
- curl 命令
 - 收集指标, 151
- OpenTelemetry (OTel), 153-156
- APT 软件包管理器
- OpenTelemetry for NGINX, 153
- 安装, 1-4
 - OTel 模块, 153

- otel_trace_context 指令, 154, 156
- map 模块, 155, 156
- ngx_otel_module, 156
- OTel propagate 参数, 156
- Prometheus Exporter 模块使用 GoLang 编写而成, 157
- stub_status 模块, 157
- Kubernetes, 158

第 14 章 (159-166)

- 访问控制, 171
 - (另请参阅“安全控制”)
 - 日志, 159-161
- 日志记录
 - 访问日志, 159-161, 171
 - 聚合日志, 162
 - 错误日志, 161
- 调试, 159-165
 - 访问日志配置, 159-161
 - 错误日志配置, 161
 - 请求跟踪, 164
 - 服务器配置, 163-164
 - 转发到系统日志 (Syslog) 侦听器, 162
- Kubernetes PROXY 协议, 160
- 上游模块, 160
- 定义日志记录格式时所用的 escape 参数, 160
- geoproxy 日志格式配置, 160
- listen 指令
 - 访问日志配置, 159-161
- escape 参数, 160
- proxy_protocol 参数, 160
- Real IP 模块, 160
- X-Forwarded-For 请求头, 160
- GeoIP 模块, 160
- access_log 指令, 161, 162
- 配置错误日志, 161
- 日志模块, 161
- error_log 指令, 162, 163
- Syslog, 162
- Syslog 侦听器, 162
- syslog 参数, 162
- IP 地址
 - 连接调试, 163
- 无类域间路由 (CIDR), 163
- debug_connection 指令, 163
- 打开调试日志记录, 163
- 配置, 6
 - (另请参阅“身份验证”“可编程性”)
 - 调试配置, 163-164
- 核心转储的启用, 164
- 根据请求 ID 追踪请求, 164
- rewrite_log 指令, 164
- proxy_set_header 指令, 165

第 15 章 (167-172)

- 性能调优, 167-173
 - 访问日志缓冲, 171
 - 使用压测工具实现测试自动化, 167
 - 缓冲响应, 170
 - 缓存, 168
 - 保持开放状态的客户端连接, 169
 - 保持上游连接开放, 169
 - 操作系统调优, 172
- 自动化 (请参阅“可编程性”)
- 瓶颈驱动型性能调优, 167
- 压测工具, 167
- 缓存, 37-44
 - 性能调优, 168
- add_header 指令, 168
- 客户端缓存, 168
- Cache-Control 响应头, 168
- expires 指令, 168
- 客户端连接管理, 169
- 连接
 - 客户端连接, 169, 170
 - 调试, 163
 - 上游连接, 169
- 上游连接, 169, 170
- keepalive_requests 指令, 169
- keepalive_timeout 指令, 169
- proxy_set_header 指令, 170
- keepalive 指令, 170
- proxy_http_version 指令, 170
- 访问控制, 171
 - (另请参阅“安全控制”)
 - 日志, 171
- 访问日志, 171
- 缓冲访问日志, 171
- 代理
 - 缓冲响应, 171
- proxy_buffering 指令, 171
- proxy_buffers 指令, 171
- proxy_busy_buffer_size 指令, 171
- access_log 指令, 172
- listen 指令的 backlog 参数, 172
- access_log 指令的 buffer 参数, 172
- access_log 指令的 flush 参数, 172
- net.core.somaxconn 内核设置, 172
- listen 指令
 - 操作系统调优, 172
- 启用临时端口, 173
- 用于操作系统调优的文件描述符数, 173
- 内核调优, 173
- net.ipv4.ip_local_port_range 内核设置, 173
- 调大可打开文件描述符上限, 173
- sys.fs.file_max 内核选项, 173

关于作者

Derek DeJonghe 对技术充满热情。他在 Web 开发、系统管理和网络方面拥有扎实的专业背景和丰富的行业经验，对现代 Web 架构有着全面的了解。Derek 领导了一支站点可靠性和云解决方案工程师团队，在一家专业咨询集团工作了十年，创建了可自我修复的自动扩展基础架构。

他目前正专注于筹建一个专用孵化器，以帮助初创企业启用云环境和开发工具。Derek 利用自己在云技术和开发方面的丰富领导经验，帮助初创企业制定上市战略，并助力其团队成员研发未来技术。Derek 在弹性云架构方面成绩卓著，革新了云部署安全性和可维护性，可充分满足客户需求。

书末题署

《NGINX 完全指南》封面上的动物是欧亚猞猁 (*Lynx lynx*)，是最大的猞猁物种，分布范围广泛，从西欧延伸到中亚。猞猁的耳尖生有黑色耸立簇毛，两颊毛发浓密而粗糙。皮毛颜色从黄灰色到灰褐色，腹部为白色。这只猞猁通身布满黑斑，与南部地区的亚种相比，北部地区的亚种更灰白，斑点更少。

与其他猞猁物种不同，欧亚猞猁捕食较大的有蹄类动物，例如鹿、麋，甚至是驯养的绵羊。成年猞猁每天消耗两到五磅肉，可长达一周食用单一食物。

欧亚猞猁在二十世纪中叶濒临灭绝，后来经过坚持不懈的保护，降级为无危物种。O'Reilly 封面上的许多动物都濒临灭绝，但对世界意义重大。

封面插画由 Karen Montgomery 参考 Shaw 著作《Zoology (动物学)》中的黑白版画绘制而成。该系列由 Edie Freedman、Ellie Volckhausen 和 Karen Montgomery 共同设计。封面字体为微软雅黑系列字体，Neusa Next Std Bold 和 Arial 粗体。正文字体为微软雅黑和 Arial 字体，标题字体为微软雅黑粗体，Proxima Nova 粗体和 Arial 粗体，代码字体为 Dalton Maag's Ubuntu Mono。